

Corso di Ingegneria del Software

Prof. Ing. Andrea Calvagna Ph.D.

A.A. 2024/2025 - canale M-Z



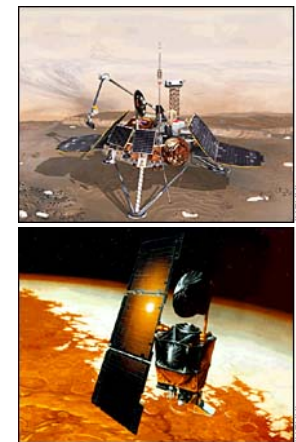
What is Software Engineering?

What happens?



"Better, Faster, Cheaper"

- In Sept. '99, NASA lost both the Mars Polar Lander and the Climate Orbiter.
- Later investigations determined software errors were to blame.
 - Orbiter: Component reuse error.
 - Lander: Precondition violation.



USS Yorktown



“After a crew member mistakenly entered a zero into the data field of an application, the computer system proceeded to divide another quantity by that zero. *The operation caused a buffer overflow, in which data leaked from a temporary storage space in memory, and the error eventually brought down the ship's propulsion system.* The result: *the Yorktown was dead in the water for more than two hours.*”

Warrantees?

LIMITED WARRANTY. Microsoft warrants that (a) the SOFTWARE PRODUCT will perform substantially in accordance with the accompanying written materials for a period of ninety (90) days from the date of receipt, ...

LIMITATION OF LIABILITY. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, *IN NO EVENT SHALL MICROSOFT OR ITS SUPPLIERS BE LIABLE FOR ANY SPECIAL, INCIDENTAL, INDIRECT, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING, ...) ARISING OUT OF THE USE OF ... THE SOFTWARE PRODUCT... MICROSOFT'S ENTIRE LIABILITY ... SHALL BE LIMITED TO THE GREATER OF THE AMOUNT ACTUALLY PAID BY YOU FOR THE SOFTWARE PRODUCT OR U.S. \$5.00; PROVIDED...*

Automotive Analogy

- “If the automobile had followed the same development as the computer, a Rolls-Royce would today cost \$100, get a million miles per gallon, and ...

Automotive Analogy

- “If the automobile had followed the same development as the computer, a Rolls-Royce would today cost \$100, get a million miles per gallon, and... explode once a year killing everyone inside.”

Space Shuttle Software

- Cost: \$10 Billion, millions of dollars more than planned
- Time: 3 years late
- Quality: First launch of Columbia was cancelled because of a synchronization problem with the Shuttle's 5 onboard computers.
 - Error was traced back to a change made 2 years earlier when a programmer changed a delay factor in an interrupt handler from 50 to 80 milliseconds.
- Substantial errors still exist.
 - Astronauts are supplied with a book of known software problems "Program Notes and Waivers".

Group Activity: What is Software Engineering?

- Come up with a group answer to the question:
 - How would you define it or characterize it?
- Choose someone to report the group answer(s)

What is Software Engineering?

a raw and incomplete definition

- The discipline that studies:
 - The **process** of building a software **product**.
- Some questions to put SE in perspective:
 - What are the sizes of some typical software products?
 - How many people would it take to build these in 1 year? 2?
 - What would you do if a bug could cost lives and \$2 billion?
 - What would you do if a delay could cost \$100's or millions?

Challenges of Software Engineering

- What are the fundamental challenges?
 - Requirements can unexpectedly **change** over time
 - software system can be very **complex** systems
 - **defects** in software are always inevitable

History

- The field of software engineering was born in 1968 in response to chronic failures of large software projects to meet schedule and budget constraints – Recognition of "the software crisis"
- Term became popular after NATO Conference in Garmisch Partenkirchen (Germany), 1968

Early Use of the term "software engineering"

Margaret Hamilton got the Presidential Medal of Freedom, November 22, 2016



"Software during the early days of [the Apollo space missions] was treated like a stepchild and not taken as seriously as [hardware] ... I fought to bring the software legitimacy so that it (and those building it) would be given its due respect"

– Margaret Heafield Hamilton

From a 2014 interview

Software Engineering: an appropriate definition

Software Engineering is a collection of

- **techniques**,
- **methodologies** and
- **tools** that help with
- the **production** of
 - a high **quality** software system
 - with a given **budget**
 - before a given **deadline**
- while **change** occurs.

Pratiche manuali
Concetti astratti e ideologie
Strumenti

Qualità?
Costi per cose e persone
mentre il tempo scorre...

Software quality?

- Criteri operativi per valutare la qualità
 - **Correttezza**: il sistema software raggiunge lo scopo ed è conforme alle specifiche
 - Il sistema software fa quello che il cliente vuole? (**Validazione** funzionale)
 - Il sistema software soddisfa le specifiche? (**Verifica** di correttezza tecnica)
 - **Efficienza, dependability** (sicurezza e affidabilità), **usabilità**

Maintainability: la più costosa

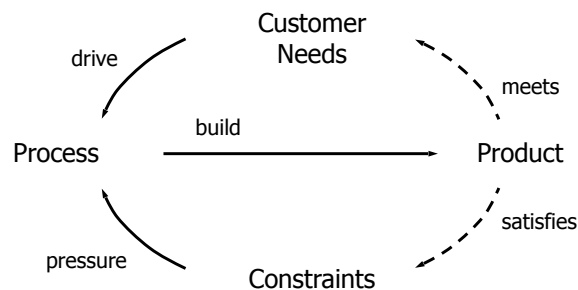
- **Manutenibilità:** un sistema software è intrinsecamente modificabile, poiché non ha parti fisiche
- Se un sistema software è di successo sarà necessario cambiarlo
 - Per adattarlo ad una realtà che cambia (mutate o nuove esigenze)
- Le richieste di estensione aumentano al crescere del successo
- Poiché di successo, il sistema software sopravvive all'hardware per cui era stato sviluppato inizialmente, generando una nuova esigenza di adattamento alla nuova piattaforma

Software Engineering: riformuliamo la definizione

- Software engineering
 - meets **customer** needs...
 - by applying systematic, quantifiable **processes**...
 - to the development and evolution of software **products** and services...
 - subject to resource and other external **constraints**

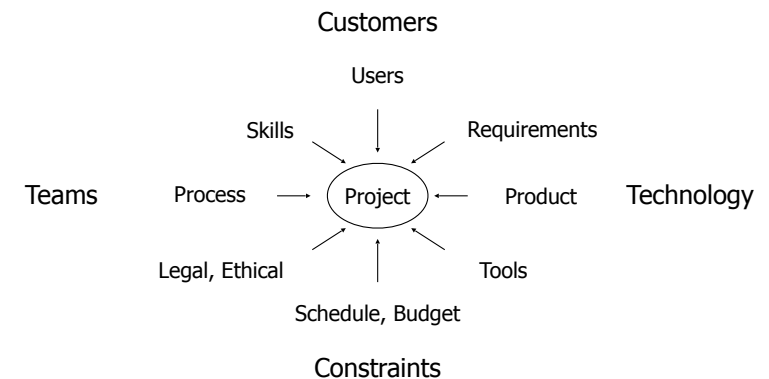
Software Process

Developers can control the Process (how they work) and Product (what they build)



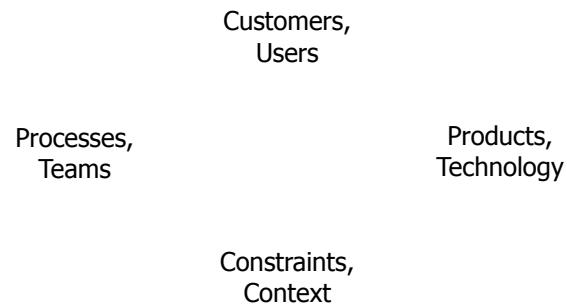
Representative Sample of Driving Forces

Drivers are related; e.g., teams use processes, products embody technology



Drivers (Forces) in the Definition

A significant change related to any one of them affects all aspects of a project



Example: Customer Needs as a Driver

- Company A wants to respond quickly to customer needs
 - Incremental features go from idea to deployment in roughly a week
 - Developers/teams working in parallel deploy many updates a day
- The pace of deployment affects every aspect of the company
 - Organization structure: No time for a separate testing organization
 - Software architecture: It must accommodate changes gracefully

Example: Constraints as a Driver

- Company B is highly regulated, the cost of a misstep is high
 - It tests 3rd party software in its own lab before deployment
- The high cost of lab testing affects the pace of deployment
 - The company wants to batch changes, so it can test them together
 - It asks suppliers to provide semi-annual releases for deployment

Examples together: The Pace of Deployment

- Over 1,000x difference in the pace for companies A and B
- Company A deploys small changes 10 or more times a day
- Over 100 days between deployments for Company B
- Both companies are doing what they believe is right for them

Consequences of the Magnitude of a Change

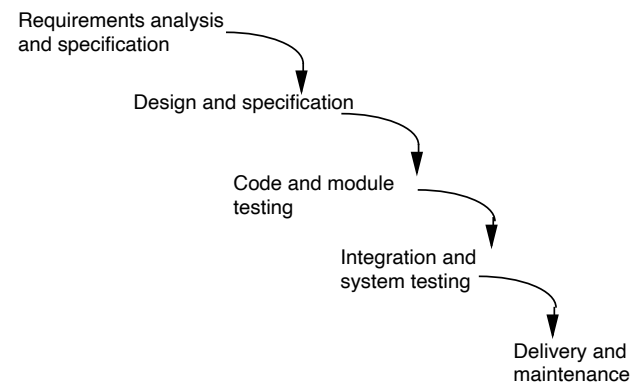
- Advice on scaling from Jeff Dean of Google
 - With a 10x change in any aspect of a software project
 - The old solution may not be optimal or may not work at all
 - With a 100x change
 - May require an entirely new approach
 - Maybe even a new organizational structure

What is a software process?

- A set of activities and their relationships to each other to support the development or evolution of a software system
- Typical questions:
 - Which activities should I select for the software project?
 - What are the dependencies between activities?
 - How should I schedule the activities?

The software lifecycle (a preview)

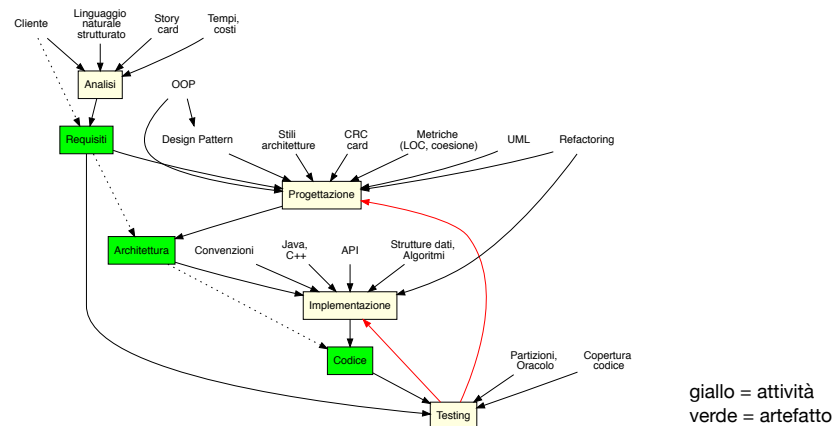
Waterfall model



Software Process

- Generic activities in all software processes are:
 - **Analysis and Specification** - what the system should do and its development constraints
 - **Development** - production of the software system actual code
 - **Validation** - checking that the software is what the customer wants
 - **Maintenance/Evolution** - changing the software in response to changing demands.

The software development process (enhanced model)



Role of software engineer

- Programming skill not enough
 - understand requirements and write specifications
 - derive models and reason about them
 - master software coding
 - operate at various abstraction levels
 - member of a team: communication skills, management skills

Scientist vs Engineer

- **Computer Scientist**
 - Proves theorems about algorithms, designs languages, defines knowledge representation schemes
 - Has infinite time...
- **Engineer**
 - Develops a solution for an application-specific problem for a client
 - Uses computers & languages, tools, techniques and methods
- **Software Engineer**
 - Works in multiple application domains
 - Has only 3 months...
 - ...while changes occurs in requirements and available technology

15 min break



Challenges of Software Engineering

•What are the fundamental challenges?

- Requirements can unexpectedly **change** over time -> organizzazione
- software system can be very **complex** systems -> astrazione
- **defects** in software are always inevitable -> misurazione

Contenuti del corso

- Programmazione object oriented e funzionale in Java
- Notazioni standard: Unified Modelling Language
- Modellazione e Progettazione, Design Patterns
- Tecniche di Refactoring
- Processi di sviluppo dei sistemi software
- Misurazione della correttezza del software
- Misurazione della qualità del software

Lezioni

- **Partecipazione fortemente consigliata:** si impara di più, e si ascolta da un esperto, è possibile interagire con domande, durante e a fine lezione
- **Ricevimento:** in studio dopo la lezione. Oppure mandare un messaggio su Teams per un appuntamento
- **Per rendere efficace lo studio:** non limitarsi alle slides, non usare solo Internet, studiare dai libri indicati. Le slide non bastano!
- **Esercitarsi:** con il codice fornito, usare i concetti spiegati e i tool consigliati

Comunicazione

- Materiale, slides e altro su gruppo TEAMS di **Ingegneria del Software A-Z**, canale **General/files/SE Corso MZ 2025**
- Codice mostrato a lezione su **<https://github.com/amcalvagna>**
- Canal su Teams e su Telegram (Ingegneria del Software) per avvisi rapidi sul corso (lezioni, esami)

Modalità Esami

- Test a risposte chiuse (teoria), test a risposta aperta (implementare codice, disegnare alcuni diagrammi UML), orale
- Progetto opzionale, da concordare (dopo pasqua)

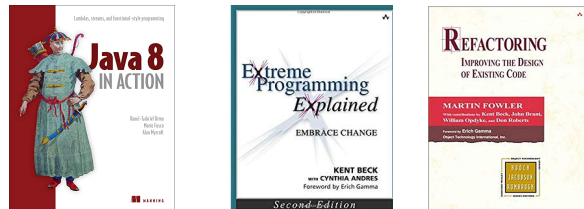
Libri utilizzati

- Ravi Sethi - Software Engineering: Basic Principles and Best Practices - Cambridge University Press.
- Martin Fowler - UML Distilled - Pearson Education
- Erich Gamma et al. - Design Patterns: Elements of Reusable Object-Oriented Software - Addison-Wesley
- Dan S. Myers - Data Structures and Algorithms in Java - Cambridge University Press.



Libri Consigliati

- Raul Urma, Fusco, Mycroft - Java 8 in Action - Manning
- Kent Beck - Extreme Programming Explained - Addison-Wesley
- Martin Fowler - Refactoring: Improving the design of existing code - Addison-Wesley



Lezioni

Aula 24					
1° anno		2° anno	3° anno	1° anno Mag.	2° anno Mag.
3° anno CV1	3° anno CV2	3° anno CV3	3° anno CV4	3° anno CV5	3° anno CV6
Lun	Mar	Mer	Gio	Ven	
08:00 - 09:00	INGEGNERIA DEL SOFTWARE M-Z (Calvagna)		INGEGNERIA DEL SOFTWARE M-Z (Calvagna)		
09:00 - 10:00					
10:00 - 11:00	RETI DI CALCOLATORI M-Z (Rundo / Santoro)	SISTEMI OPERATIVI M-Z (Pavone)	RETI DI CALCOLATORI M-Z (Rundo / Santoro)	SISTEMI OPERATIVI M-Z (Pavone)	
11:00 - 12:00					
12:00 - 13:00					
13:00 - 14:00					
14:00 - 15:00		SISTEMI OPERATIVI A-L (Di Raimondo)		SISTEMI OPERATIVI A-L (Di Raimondo)	
15:00 - 16:00					
16:00 - 17:00					
17:00 - 18:00					
18:00 - 19:00					

Java

Java is one of the world's most popular programming languages

- ▶ Widely used in enterprise software development
- ▶ Has a large library of built-in features
- ▶ Good performance and portability
- ▶ Default language for Android mobile devices and Minecraft, among many others

Java belongs to the family of **object-oriented languages**

- ▶ Programs are organized into units called **classes** and **objects**
- ▶ We'll discuss these in detail later in the class

Java origins

Java was developed in the early 1990s by a team at Sun Microsystems led by James Gosling

- ▶ Originally called Oak after a tree outside Gosling's office
- ▶ Intended for programming smart TVs and other devices

Quickly pivoted to the emerging Web after the first release in 1995

If you have experience with Python or JavaScript, Java will have similarities, but with an older, more verbose style

- ▶ If your experience is with C/C++, Java will feel very similar

HelloWorld.java

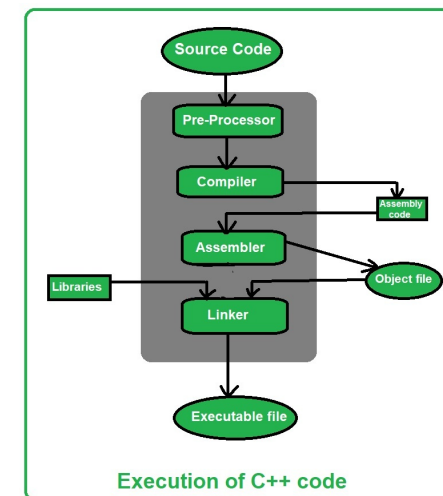
```
import java.time.LocalDate;

/**
 * Classe che stampa sullo schermo un messaggio e la data corrente
 * Tutto il codice di questo file deve essere contenuto in una classe
 * con lo stesso nome del file che la contiene
 */
public class HelloWorld { // definizione classe:
    // dichiarazione e assegnazione campi
    private static final String msg = "Lezione di Ingegneria del Software";
    private static final LocalDate d = LocalDate.now();

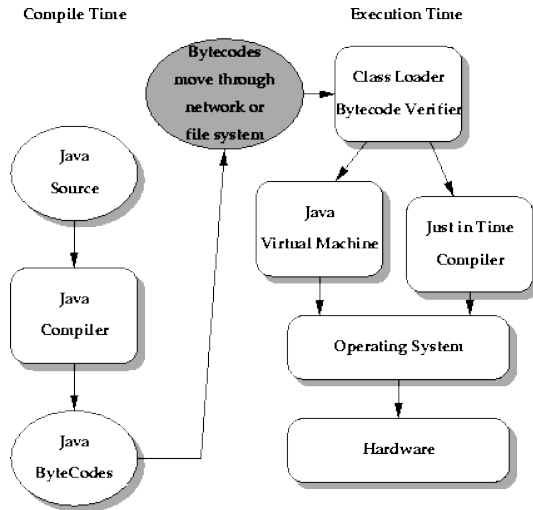
    /**
     * Metodo da cui inizia l'esecuzione del programma
     */
    @param args parametri passati al metodo all'avvio della classe
    /**
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println(msg);
        System.out.println(d);
    }
}
```

• Il codice della classe HelloWorld deve essere salvato sul file HelloWorld.java, compilato con `javac HelloWorld.java` ed eseguito con `java HelloWorld`

Output:
Hello World
Lezione di Ingegneria del Software
2025-03-03



Java Program Environment



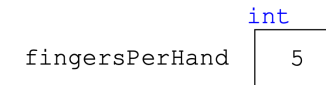
Java variables and data types

Every Java variable has a declared type

```
int fingersPerHand = 5;
```

A variable can only be assigned values of its type

Think of a variable as a name bound to a box that holds a value



The eight primitive types

Type	Format	Range	Example Values
<code>int</code>	32-bit signed integer	-2^{31} to $2^{31} - 1$	10, 0, -101
<code>short</code>	16-bit signed integer	-2^{16} to $2^{16} - 1$	10, 0, 32767
<code>long</code>	64-bit signed integer	-2^{64} to $2^{64} - 1$	12345L, -6789L
<code>byte</code>	8-bit signed integer	-128 to 127	0, -101
<code>double</code>	64-bit floating point	$\pm 1.7\text{E}308$	-1.0, 3.14, 2e100
<code>float</code>	32-bit floating point	$\pm 3.4\text{E}38$	-1.0f, 3.14F
<code>char</code>	16-bit Unicode	A single text character	'z', '0', 'n'
<code>boolean</code>	Logical value	N/A	true, false

Object types

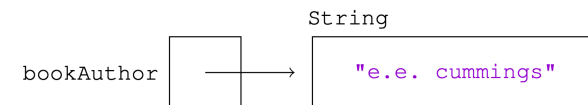
Anything that isn't a primitive type is an *object type*

► Object types start with a capital letter

String is the object type representing a sequence of text characters

```
String bookAuthor = "e.e. cummings";
```

The variable is the *object reference* that points to the underlying object in memory



Reading user input with Scanner

Scanner is a built-in class that can read input from the user console and from files

Import it at the top of your program

```
import java.util.Scanner;
```

Creating a Scanner is more complex than a primitive type variable

```
Scanner input = new Scanner(System.in);
```

Creating a Scanner

```
Scanner input = new Scanner(System.in);
```

The left side declares a `Scanner` variable named `input`

The right side has two parts

- ▶ The `new` keyword: create a new object
- ▶ A call to the constructor method, which initializes the object

The constructor always has the same name as the class

- ▶ `Scanner`'s constructor takes one input: the source to read from
- ▶ `System.in` is the standard system input

A full Scanner example

```
1 // Reading user input with Scanner
2
3 import java.util.Scanner;
4
5 public class ScannerDemo {
6     public static void main(String[] args) {
7         // Create a new Scanner -- System.in is the standard input
8         Scanner input = new Scanner(System.in);
9
10        // Read a String using the nextLine method
11        System.out.println("What is your name?");
12        String name = input.nextLine();
13
14        // Print a message using the name
15        System.out.printf("Hello, " + name + ".");
16    }
17 }
```

Exceptions

What happens if you input a non-numeric value, like "fifteen"? The result is an `InputMismatchException`

Exception in thread "main" java.util.InputMismatchException

at java.base/java.util.Scanner.throwFor(Scanner.java:939)

at java.base/java.util.Scanner.next(Scanner.java:1594)

at java.base/java.util.Scanner.nextDouble(Scanner.java:2564)

at Smoots.main(Smoots.java:15)

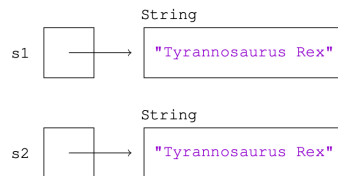
Exceptions are Java's way of handling run-time errors that occur while the program is executing

- ▶ It's possible to "catch" an exception and handle it gracefully—more on this later

Comparing Objects

```
// Construct two new Strings with the same characters
String s1 = new String("Tyrannosaurus Rex");
String s2 = new String("Tyrannosaurus Rex");

// Test if the two strings are equal
if (s1 == s2) {
    System.out.println("Equal");
} else {
    System.out.println("Not equal");
}
```



Comparing Objects

`==` tests if the two objects are *the same object in memory*

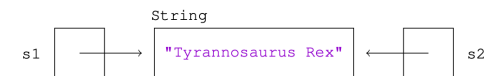
- This is called *reference equality*

In the previous example, the two strings are not equal

- `s1` and `s2` point to different objects in memory, even though they have the same characters

Here, `s1 == s2` would be `true`

```
String s1 = new String("Tyrannosaurus Rex");
String s2 = s1;
```



Comparing Objects

Use the `equals` method to test for *value equality*: do the objects have equal data?

- This is the more typical concept of equality
- For strings, `equals` tests if they have the same characters in the same sequence

```
// Two strings with the same characters
String s1 = new String("Tyrannosaurus Rex");
String s2 = new String("Tyrannosaurus Rex");

// Test for value equality
if (s1.equals(s2)) {
    System.out.println("Equal");
} else {
    System.out.println("Not equal");
}
```

Identificatori e Puntatori

C/C++

- `T a;` //alloca una struct T
- `T a();` //alloca una istanza di T
- `T *b;` //alloca un puntatore a T

- `new T();` //all. din. una istanza di T;
- `b++;` //legale
- `a = *b;` //legale
- `a->m();` //accesso interno a T

Java

- `T a;` //alloca l'id. a di tipo T
- `T a();` //illegale
- `T *b;` //illegale

- `new T();` //all. din. una istanza di T;
- `a++;` //illegale
- `a = *b;` //illegale. `a=new T();`
- `a.m();` //accesso interno a T

Differenze C++ vs Java

- Il significato del separatore punto . È lo stesso di $\rightarrow$ in C++
 - Gli identificatori in Java sono tutti puntatori
- Non ci sono che oggetti, nulla può essere definito al di fuori di un costrutto di classe (o interfacce)
- Non serve il punto e virgola alla fine dei blocchi dichiarativi delle classi {}
- In una Classe metto contestualmente sia l'astrazione (l'ADT) che la sua implementazione
- È fortemente tipizzato (strong type checking)
 - I tipi ai lati di un assegnamento devono essere formalmente compatibili a tempo di compilazione
- Controlla anche il tipo dinamico
 - un identificatore punta a oggetti di tipo diverso in momenti diversi dell'esecuzione
 - Il tipo effettivo a runtime deve essere compatibile col tipo a cui lo assegni

Differenze C++ vs Java

- Non supporta l'ereditarietà multipla
 - Problema del diamante
 - Possibile con le interfacce
 - La radice della gerarchia delle classi in Java è unica ed è **la classe Object**
 - Libreria standard java è nel dominio java.*
 - Una sola classe per file (*), con lo stesso nome
 - Il main va definito per forza dentro una classe
 - Gli "include" indicano interi file (header), gli import indicano gruppi o singole classi per nome composto
- (*) salvo le nidificate

Parole chiave di Java

- **class** permette di definire un tipo, e quindi le sue istanze
- **final** definisce un campo o una variabile che non può essere assegnata più di una volta (una costante). Una classe final non può essere ereditata, un metodo final non può essere ridefinito (override)
- **import** indica dove trovare la definizione di una classe che sarà usata nel seguito
- **new** permette di creare un'istanza di una classe
- **private** e **public** indicano l'accessibilità di classi, campi e metodi
- **static** è usata per dichiarare un campo o un metodo appartenente alla classe (e non all'istanza)
- **void** indica che il metodo non ritorna alcun valore
- Tipi usati: **String**, per rappresentare insiemi di caratteri; **LocalDate**, per accedere alla data attuale; **System** per scrivere sullo schermo

Writing static methods

Java methods are named blocks of code that take inputs and return outputs

- ▶ You may be familiar with Python or JavaScript functions
- ▶ Method is the preferred term in Java

We'll start by writing static methods, which work like their counterparts in other languages

They are globally unique code blocks, immediately available for execution

```
public static double circleArea(double radius) {  
    return Math.pow(radius, 2) * Math.PI;  
}
```

Static Method declaration:

- ▶ The keywords `public` and `static`
- ▶ The return type, which specifies the type of value returned by the method;
- ▶ The name, using the same rules as variable names
- ▶ A list of input parameters, each with a type

```
public static void main(String[] args){...}
```

`void` is the special return type for methods that don't return anything

So `main` is a static method that takes an array of `String` as input and returns nothing

Hello World in Java

```
import java.time.LocalDate; // indica dove trovare la classe LocalDate
```

```
public class HelloWorld { // dichiara classe HelloWorld
    private static final String msg = "Lezione di Ingegneria del
Software";
    private LocalDate d; // dichiara campo d di tipo LocalDate
```

```
    public static void main(String[] args) {
        System.out.println("Hello World"); // scrive su schermo
        System.out.println(msg);
        final HelloWorld world = new HelloWorld(); // crea oggetto
        world.printDate(); // chiama metodo
    }
```

```
    private void printDate() { // metodo
        d = LocalDate.now(); // chiama metodo static now
        System.out.println(d);
    }
}
```

HelloWorld	
- msg: String	
- d: LocalDate	
+ main(args: String[*])	
- printDate()	

Output
Hello World
Lezione di Ingegneria del Software
2025-03-05

Programmazione in Java



Java

Java is one of the world's most popular programming languages

- ▶ Widely used in enterprise software development
- ▶ Has a large library of built-in features
- ▶ Good performance and portability
- ▶ Default language for Android mobile devices and Minecraft, among many others

Java belongs to the family of **object-oriented languages**

- ▶ Programs are organized into units called **classes** and **objects**
- ▶ We'll discuss these in detail later in the class

Java origins

Java was developed in the early 1990s by a team at Sun Microsystems led by James Gosling

- ▶ Originally called Oak after a tree outside Gosling's office
- ▶ Intended for programming smart TVs and other devices

Quickly pivoted to the emerging Web after the first release in 1995

If you have experience with Python or JavaScript, Java will have similarities, but with an older, more verbose style

- ▶ If your experience is with C/C++, Java will feel very similar

HelloWorld.java

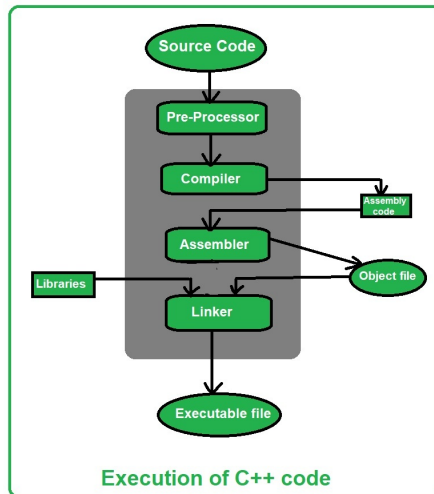
```
import java.time.LocalDate;

/**
 * Classe che stampa sullo schermo un messaggio e la data corrente
 * Tutto il codice di questo file deve essere contenuto in una classe
 * con lo stesso nome del file che la contiene
 */
public class HelloWorld { // definizione classe:
    // dichiarazione e assegnazione campi
    private static final String msg = "Lezione di Ingegneria del Software";
    private static final LocalDate d = LocalDate.now();

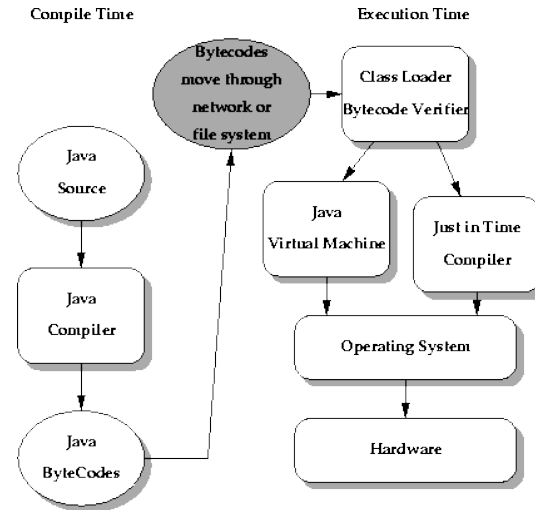
    /**
     * Metodo da cui inizia l'esecuzione del programma
     *
     * @param args parametri passati al metodo all'avvio della classe
     */
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println(msg);
        System.out.println(d);
    }
}
```

• Il codice della classe HelloWorld deve essere salvato sul file HelloWorld.java, compilato con `javac HelloWorld.java` ed eseguito con `java HelloWorld`

Output:
Hello World
Lezione di Ingegneria del Software
2025-03-03



Java Program Environment



Java variables and data types

Every Java variable has a declared type

```
int fingersPerHand = 5;
```

A variable can only be assigned values of its type

Think of a variable as a name bound to a box that holds a value

```
int
fingersPerHand [ 5 ]
```

The eight primitive types

Type	Format	Range	Example Values
int	32-bit signed integer	-2^{31} to $2^{31} - 1$	10, 0, -101
short	16-bit signed integer	-2^{16} to $2^{16} - 1$	10, 0, 32767
long	64-bit signed integer	-2^{64} to $2^{64} - 1$	12345L, -6789L
byte	8-bit signed integer	-128 to 127	0, -101
double	64-bit floating point	$\pm 1.7E308$	-1.0, 3.14, 2e100
float	32-bit floating point	$\pm 3.4E38$	-1.0f, 3.14F
char	16-bit Unicode	A single text character	'z', '0', 'n'
boolean	Logical value	N/A	true, false

Object types

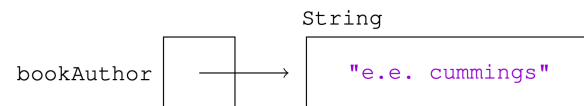
Anything that isn't a primitive type is an *object type*

- ▶ Object types start with a capital letter

String is the object type representing a sequence of text characters

```
String bookAuthor = "e.e. cummings";
```

The variable is the *object reference* that points to the underlying object in memory



Reading user input with Scanner

Scanner is a built-in class that can read input from the user console and from files

Import it at the top of your program

```
import java.util.Scanner;
```

Creating a Scanner is more complex than a primitive type variable

```
Scanner input = new Scanner(System.in);
```

Creating a Scanner

```
Scanner input = new Scanner(System.in);
```

The left side declares a **Scanner** variable named input

The right side has two parts

- ▶ The **new** keyword: create a new object
- ▶ A call to the constructor method, which initializes the object

The constructor always has the same name as the class

- ▶ **Scanner**'s constructor takes one input: the source to read from
- ▶ **System.in** is the standard system input

A full Scanner example

```
1 // Reading user input with Scanner
2
3 import java.util.Scanner;
4
5 public class ScannerDemo {
6     public static void main(String[] args) {
7         // Create a new Scanner -- System.in is the standard input
8         Scanner input = new Scanner(System.in);
9
10        // Read a String using the nextLine method
11        System.out.println("What is your name?");
12        String name = input.nextLine();
13
14        // Print a message using the name
15        System.out.printf("Hello, " + name + ".");
16    }
17 }
```

Exceptions

What happens if you input a non-numeric value, like "fifteen"? The result is an InputMismatchException

Exception in thread "main" java.util.InputMismatchException

at java.base/java.util.Scanner.throwFor(Scanner.java:939)

at java.base/java.util.Scanner.next(Scanner.java:1594)

at java.base/java.util.Scanner.nextDouble(Scanner.java:2564)

at Smoots.main(Smoots.java:15)

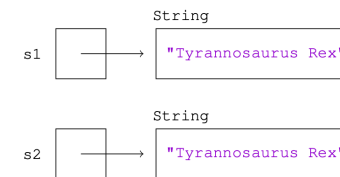
Exceptions are Java's way of handling run-time errors that occur while the program is executing

- It's possible to "catch" an exception and handle it gracefully—more on this later

Comparing Objects

```
// Construct two new Strings with the same characters
String s1 = new String("Tyrannosaurus Rex");
String s2 = new String("Tyrannosaurus Rex");

// Test if the two strings are equal
if (s1 == s2) {
    System.out.println("Equal");
} else {
    System.out.println("Not equal");
}
```



Comparing Objects

== tests if the two objects are *the same object in memory*

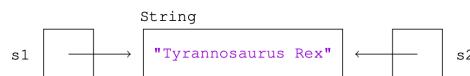
- This is called *reference equality*

In the previous example, the two strings are not equal

- s1 and s2 point to different objects in memory, even though they have the same characters

Here, s1 == s2 would be **true**

```
String s1 = new String("Tyrannosaurus Rex");
String s2 = s1;
```



Comparing Objects

Use the `equals` method to test for *value equality*: do the objects have equal data?

- This is the more typical concept of equality
- For strings, `equals` tests if they have the same characters in the same sequence

```
// Two strings with the same characters
String s1 = new String("Tyrannosaurus Rex");
String s2 = new String("Tyrannosaurus Rex");

// Test for value equality
if (s1.equals(s2)) {
    System.out.println("Equal");
} else {
    System.out.println("Not equal");
}
```

Identificatori e Puntatori

C/C++

- `T a;` //alloca una struct T
- `T a();` //alloca una istanza di T
- `T *b;` //alloca un puntatore a T

- `new T();` //all. din. una istanza di T;
- `b++;` //legale
- `a = *b;` //legale
- `a->m();` //accesso interno a T

Java

- `T a;` //alloca l'id. a di tipo T
- `T a();` //illegale
- `T *b;` //illegale

- `new T();` //all. din. una istanza di T;
- `a++;` //illegale
- `a = *b;` //illegale. `a=new T();`
- `a.m();` //accesso interno a T

Differenze C++ vs Java

- Il significato del separatore punto . È lo stesso di `—>` in C++
 - Gli identificatori in Java sono tutti puntatori
- Non ci sono che oggetti, nulla può essere definito al di fuori di un costrutto di classe (o interfacce)
- Non serve il punto e virgola alla fine dei blocchi dichiarativi delle classi { }
- In una Classe metto contestualmente sia l'astrazione (l'ADT) che la sua implementazione
- È fortemente tipizzato (strong type checking)
 - I tipi ai lati di un assegnamento devono essere formalmente compatibili a tempo di compilazione
- Controlla anche il tipo dinamico
 - un identificatore punta a oggetti di tipo diverso in momenti diversi dell'esecuzione
 - Il tipo effettivo a runtime deve essere compatibile col tipo a cui lo assegna

Differenze C++ vs Java

- Non supporta l'ereditarietà multipla
 - Problema del diamante
 - Possibile con le interfacce
 - La radice della gerarchia delle classi in Java è unica ed è **la classe Object**
 - Libreria standard java è nel dominio java.*
 - Una sola classe per file (*), con lo stesso nome
 - Il main va definito per forza dentro una classe
 - Gli "include" indicano interi file (header), gli import indicano gruppi o singole classi per nome composto
- (*) salvo le nidificate

Writing static methods

Java methods are named blocks of code that take inputs and return outputs

- ▶ You may be familiar with Python or JavaScript functions
- ▶ Method is the preferred term in Java

We'll start by writing static methods, which work like their counterparts in other languages

They are globally unique code blocks, immediately available for execution

```
public static double circleArea(double radius) {  
    return Math.pow(radius, 2) * Math.PI;  
}
```

Static Method declaration:

- ▶ The keywords `public` and `static`
- ▶ The return type, which specifies the type of value returned by the method;
- ▶ The name, using the same rules as variable names
- ▶ A list of input parameters, each with a type

```
public static void main(String[] args){...}
```

`void` is the special return type for methods that don't return anything

So `main` is a static method that takes an array of `String` as input and returns nothing

Hello World in Java

```
import java.time.LocalDate; // indica dove trovare la classe LocalDate
```

```
public class HelloWorld { // dichiara classe HelloWorld
    private static final String msg = "Lezione di Ingegneria del
Software";
    private LocalDate d; // dichiara campo d di tipo LocalDate
```

```
    public static void main(String[] args) {
        System.out.println("Hello World"); // scrive su schermo
        System.out.println(msg);
        final HelloWorld world = new HelloWorld(); // crea oggetto
        world.printDate(); // chiama metodo
    }
```

```
    private void printDate() { // metodo
        d = LocalDate.now(); // chiama metodo static now
        System.out.println(d);
    }
}
```

HelloWorld
- msg: String
- d: LocalDate
+ main(args: String[*])
- printDate()

Output
Hello World
Lezione di Ingegneria del Software
2025-03-05

Parole chiave di Java

- **class** permette di definire un tipo, e quindi le sue istanze
- **final** definisce un campo o una variabile che non può essere assegnata più di una volta (una costante).
Una classe `final` non può essere ereditata, un metodo `final` non può essere ridefinito (override)
- **import** indica dove trovare la definizione di una classe che sarà usata nel seguito
- **new** permette di creare un'istanza di una classe
- **private** e **public** indicano l'accessibilità di classi, campi e metodi
- **static** è usata per dichiarare un campo o un metodo appartenente alla classe (e non all'istanza)
- **void** indica che il metodo non ritorna alcun valore
- Tipi usati: **String**, per rappresentare insiemi di caratteri; **LocalDate**, per accedere alla data attuale;
System per scrivere sullo schermo

Il paradigma Object Oriented

Caratteristiche peculiari del paradigma Object Oriented

- Incapsulamento
- Ereditarietà
- Polimorfismo
 - Sintatticamente compare solo un nuovo costrutto: "class {}"
 - Algoritmi + strutture dati = ...programmi? → classi
- Cosa è una classe?
 - che differenza c'è con un modulo?
 - con un oggetto?
 - è un tipo ?
- E' uno strumento **sintattico** nato per supportare **astrazioni** ed una codifica più **robusta e riusabile**, il cui significato concettuale (la semantica) può variare leggermente col linguaggio di programmazione considerato

Attività in classe

- Implementare in C una coda LIFO
 - Struttura dati lineare e omogenea (array)
 - dimensione e tipo parametrici
- Tempo: 5 minuti

Stack Example

```
typedef int T;  
/* const int MAX STACK = 100; */  
#define MAX STACK 100  
T stack[MAX STACK];  
int top = 0;
```

T item;

```
stack[top++] = item; // push  
...  
item = stack[--top]; // pop
```

Stack: versione minimale

- Molto efficiente e prestante in termini di velocità di esecuzione ma...
- Tipo di elementi nello stack generico, ma di un solo tipo!
 - Typedef crea un solo alias
- Un solo stack in tutto il sistema: è una variabile globale
- Lo stato dello stack è totalmente esposto
- **Il codice è semplicemente strutturato ma....per niente astratto!**

Stack con (pseudo) incapsulamento

- **Incapsulamento** (data hiding)
 - Uso un **modulo** separato per definire lo stato e il protocollo
- **Lo stato dello stack resta “nascosto” dentro l’implementazione**
- **Implementazione è modulare e riusabile, ma...**

Data Hiding Implementation in C

```
/* File stack.h */
```

```
/* Type of Stack element. */  
typedef int T;
```

```
/* Stack interface. */  
int create (int size);  
int destroy (void);  
void push (T new item);  
void pop (T *old top);  
void top (T *cur top);  
int is empty (void);  
int is full (void);
```

Data Hiding Implementation in C

```
/* File stack.c */  
#include <stdlib.h>  
#include "stack.h"
```

```
/* Hidden within this file. */  
static int top_ , size_ ;  
static T *stack_ ;
```

```
int create (int size) {  
    top_ = 0; size_ = size;  
    stack_ = malloc (size * sizeof (T));  
    return stack_ == 0 ? -1 : 0;  
}  
void destroy (void) { free ((void *) stack_); }  
void push (T item) { stack_ [top_ ++] = item; }  
void pop (T *item) { *item = stack_ [--top_]; }  
void top (T *item) { *item = stack_ [top_ -1]; }  
int is empty (void) { return top_ == 0; }  
int is full (void) { return top_ == size_ ; }
```

Use case

```
#include "stack.h"
void foo (void) {
    T i;
    push (10); /* Oops, forgot to call create! */
    push (20);
    pop (&i);
    destroy ();
}
```

Main problems:

1. The programmer must call create first and destroy last!
Could use first-time-in flag...
2. There is only one stack and only one type of stack
3. Name space pollution
4. Non-reentrant

Stack come ADT

- Definizione di uno stack come **Abstract Data Type**
 - Dichiaro lo stato come un TIPO astratto con Typedef
 - Integra la specifica della sua interfaccia
- **Ho molteplici istanze di Stack**, non è più una variabile globale
- Conflitto nello spazio dei nomi... risolto (manualmente)
- Il codice è rientrante
- Tipo di elementi nello stack generico

Data Abstraction Implementation in C

An ADT Stack interface in C: /* File stack.h */

```
/* Type of Stack element. */
typedef int T;

/* Type definition for Stack. */
typedef struct {
    size_t top_, size_;
    T *stack_;
} Stack;

/* Stack protocol interface. */
int Stack_create (Stack *s, size_t size);
void Stack_destroy (Stack *s);
void Stack_push (Stack *s, T item);
void Stack_pop (Stack *, T *item);

/* Must call before pop'ing */
int Stack_is_empty (Stack *);

/* Must call before push'ing */
int Stack_is_full (Stack *);
/*...*/
```

Data Abstraction Implementation in C

An ADT Stack implementation in C: /* File stack.C */

```
#include "stack.h"

int Stack_create (Stack *s, size_t size) {
    s->top_ = 0; s->size_ = size;
    s->stack = malloc (size * sizeof (T));
    return s->stack_ == 0 ? -1 : 0;
}

void Stack_destroy (Stack *s) {
    free ((void *) s->stack_);
    s->top_ = 0; s->size_ = 0; s->stack_ = 0;
}

void Stack_push (Stack *s, T item) {
    s->stack_ [s->top_++] = item;
}

void Stack_pop (Stack *s, T *item) {
    *item = s->stack_ [--s->top_];
}

int Stack_is_empty (Stack *s) {
    return s->top_ == 0;
}
```

Data Abstraction Implementation in C

Use case:

```
void foo (void){
    Stack s1, s2, s3; /* Multiple stacks! */
    T item;

    /* Pop'd empty stack */
    Stack pop (&s2, &item);

    /* Forgot to call Stack create! */
    Stack push (&s3, 10);

    /* Destroy uninitialized stack! */
    Stack destroy (&s1);
}
```

Main problems with Data Abstraction in C

1. No guaranteed initialization/termination
2. Still only one type of stack supported
3. Too much overhead due to function calls (si può usare "inline")
4. No generalized error handling...
5. The C compiler does not **enforce** information hiding e.g.,
s1.top = s2.stack [0]; /* Violate abstraction */
s2.size = s3.top ; /* Violate abstraction */

Data Abstraction Implementation in OOP (C++)

Question: how to we get encapsulation and more than one stack?
Answer: define a Stack ADT using Object orientation

```
// File Stack.h
typedef int T;

class Stack {
public:
    Stack (size t size);
    ~Stack (void);
    void push (const T &item);
    void pop (T &item);
    int is_empty (void) const;
    int is_full (void) const;
// ...
private:
    size_t top_, size_;
    T *stack_;
};
```

Data Abstraction Implementation in OOP (C++)

```
//Stack.cpp

int Stack::is_empty (void) const {
    return this->top_ == 0;
}

int Stack::is_full (void) const {
    return this->top_ == this->size_ ;
}

Stack::Stack (size t size) : top (0), size (size), stack (new T[size]) {}

Stack::~~Stack (void) {
    delete [] this->stack_ ;
}

void Stack::push (const T &item) {
    this->stack_ [this->top_ ++] = item;
}

void Stack::pop (T &item) {
    item = this->stack_ [--this->top_ ];
}
```

Data Abstraction Implementation in OOP

```
#include "Stack.h"

void foo (void) {
    Stack s1 (1), s2 (100);
    T item;

    if (!s1.is full ())
        s1.push (473);
    if (!s2.is full ())
        s2.push (2112);
    if (!s2.is empty ())
        s2.pop (item);
    // Access violation caught at compile-time!
    s2.top = 10;
    // Termination is handled automatically.
}
```

Benefits of OO Data Abstraction Implementation

1. Data hiding and data abstraction, e.g.,
Stack s1 (200);
s1.top = 10 // Error flagged by compiler!
2. The ability to declare multiple stack objects
Stack s1 (10), s2 (20), s3 (30);
3. Automatic initialization and termination
{
 // Constructor automatically called.
 Stack s1 (1000);
 //...
 // Destructor automatically called
}

Drawbacks with OO Data Abstraction Implementation:

1. Error handling is obtrusive
Use exception handling to solve this
2. The example is limited to a single type of stack element (int in this case)
We can use "parameterized types" to remove this limitation

Exception Handling Implementation

exception handling separates error handling from normal processing

```
// File Stack.h

typedef int T;
class Stack {
public:
    class Underflow { /* ... */ };
    class Overflow { /* ... */ };
    Stack (size_t size);
    ~Stack (void);
    void push (const T &item) throw (Overflow);
    void pop (T &item) throw (Underflow);
    // ...
private:
    size_t top_, size_;
    T *stack_;
};
```

Exception Handling Implementation

exception handling separates error handling from normal processing

```
//Stack.C

void Stack::push (const T &item) throw (Stack::Overflow){
    if (this->is_full ()) throw Stack::Overflow ();
    this->stack_[this->top_++] = item;
}

void Stack::pop (T &item) throw (Stack::Underflow){
    if (this->is_empty ()) throw Stack::Underflow ();
    item = this->stack_[--this->top_];
}
```

Exception Handling Implementation

Use case:

```
#include "Stack.h"

void foo (void) {
    Stack s1 (1), s2 (100);
    try {
        T item;
        s1.push (473);
        s1.push (42); // Exception, push'd full stack!
        s2.pop (item); // Exception, pop'd empty stack!
        s2.top = 10; // Access violation caught!
    } catch (Stack::Underflow e) {
        // Handle underflow...
    } catch (Stack::Overflow e) {
        // Handle overflow...
    }
    // Termination is handled automatically.
}
```

Template Implementation in C++

A parameterized type Stack class interface using C++

```
// typedef int T;
template <class T>

class Stack {
public:
    Stack (size_t size);
    ~Stack (void)
    void push (const T &item);
    void pop (T &item);
    int is_empty (void);
    int is_full (void);
    //...
private:
    size_t top_, size_;
    T *stack_;
};
```

Template Implementation in C++

A parameterized type Stack class implementation using C++

```
template <class T>

Stack<T>::Stack (size_t size) : top (0), size (size), stack (new T[size]) {}
Stack<T>::~~Stack (void) {
    delete [] this->stack_;
}

Stack<T>::push (const T &item) {
    this->stack_[this->top_++] = item;
}

void Stack<T>::pop (T &item) {
    item = this->stack_[--this->top_];
}
```

Implementazione in JAVA dello stack

```
class Stack<T> {
    private int top, size;
    private T[] stack;
    private String type;

    public Stack (int size) {
        this.top = 0;
        this.size = size;
        this.stack= (T[]) new Object[size];
        //int [] i = new int[size]; / array id declaration and allocation
        //int x[10]; //error
    }

    public void push (T item) throws StackOverflowException{
        if (this.is_full()) throw new StackOverflowException(this);
        this.stack[this.top++] = item;
    }

    public T pop ()throws StackUnderflowException{
        if (this.is_empty()) throw new StackUnderflowException(this);
        return this.stack[--this.top];
    }

    public boolean is_empty () { return this.top == 0; }
    public boolean is_full () { return this.top == this.size; }
    //...

    void setType(String type){this.type = type;}
    String getType(){return this.type;}
};
```

Implementazione in JAVA dello stack

```
public static void main(String []args){
    Stack s = new Stack<Integer>(10);
    Stack<Integer> s1 = new Stack<Integer>(10);
    Stack<String> s2 = new Stack<String>(10);
    s1.setType("Stack<Integer>");
    s2.setType("Stack<String>");

    try{
        //s.pop();
        s.push(10);
        System.out.println(s.pop());

        s1.push(20);
        System.out.println(s1.pop());

        s2.push("ciao");
        System.out.println(s2.pop());
        s2.pop();
    }catch(StackOverflowException e){
        System.out.println(e.getMessage());
    }catch(StackUnderflowException e){
        System.out.println(e.getMessage());
    }
}
```

Conclusioni

• Caratteristiche peculiari del paradigma Object Oriented

- Incapsulamento
- Ereditarietà
- Polimorfismo

• Ma cosa è una classe?

• Un tipo ?

• Un Oggetto?

• Un ADT ?

• E' solo uno strumento di codifica: il suo significato concettuale (la semantica) varia col linguaggio di programmazione considerato

Glossary of OO Terms and definitions

an **ADT** is only a **specification** of properties, a rule that concrete objects may or not happen to obey. it is a model abstracting **many** real-world entities.

an **Object** is a **model** of a **unique** entity in the real world, and an instance is its **working** dual. Each instance, like its corresponding object, is unique and distinguishable from all the others, including those of same **type**, through its "identity" (i.e., the *this* reference).

a **Module** is a **unique** collection of **programs**, usable to realize an object instance.

an **Instance** is last stage: a program already allocated in memory and all is ready for execution. they are also usually called objects, informally, since they realize an Object model in the context of software systems.

a **Class** is a programming language **syntactical artifact** whose semantics could be an ADT, an Object, a Module, or a combination of those. In Java is a template implementation for object instances creation.

- **Class-Attribute**: a compile-time allocated (static) identifier, stored once in a Class, and shared by all its instances. The referenced object instance (which must also be static) gets shared too
- **Instance-Attribute**: an identifier *declared* in a Class, but run-time allocated inside all its instances, as separate copies. Thus, they can reference private instances of objects.
- **Class-Method**: a method globally available for execution. The receiver of the method call is the defining Class, thus instance-attributes can't be used inside (they are not yet allocated).
- **Instance-Method**: an object method which can be called only through an Instance of its defining class, so that it will be able to operate on that receiver's instance-attributes.
- **call dispatch**: determining which code to execute in response to a message sent to an identifier. This can be done statically (early binding) if identifier has an associated static type, or dynamically (late binding) at run-time, based on the type of the currently referenced instance.
- **self/this**: is a *pseudo variable* referencing to the *receiver* instance of a method call (as an additional, *hidden*, formal parameter to a function call): it is the context in which the function has to be executed (execution class instance).
- **super**: is a *dispatch qualifier*. it is not an identifier (doesn't actually references nothing). it just means to start searching for a corresponding method (to bind to an operation call) from the parent class of the current object. The current object is the one actually defining (not executing) the method where super is used. put simply, is always the parent context (class) with respect to method definition

15 min break



Object Oriented Design

To design reusable object-oriented software you must:

- find pertinent objects,
- factor them into classes at the right granularity,
- define class interfaces and inheritance hierarchies,
- establish key relationships among them.

Your design should be specific to the problem at hand but also general enough to address future problems and requirements.

Object oriented design: un esempio pratico

- Riusciamo a sviluppare un componente software che risulti: riusabile, modificabile e corretto?
- Consideriamo un componente estremamente piccolo (potrebbe far parte di un sistema più grande)
- Descrizione dei requisiti
 - Dati vari file contenenti valori numerici, con un valore per ciascuna riga del file
 1. Leggere da ciascun file la lista di valori
 2. Tenere solo i valori non duplicati
 3. Calcolare la somma dei numeri letti dal file (non duplicati)
 4. Calcolare il massimo fra i numeri letti

```

import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.io.LineNumberReader;
import java.util.ArrayList;
import java.util.List;

public class CalcolaImporti { // classe Java vers 0.0.1
    private final List<String> importi = new ArrayList<>();
    // List e ArrayList sono tipi della libreria Java
    private float totale;

    public float calcola(String c, String n) throws IOException { // metodo
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        // lettura file tramite le API Java: File, FileReader, LineNumberReader

        totale = 0;
        while (true) {
            final String riga = f.readLine(); // legge una riga dal file
            if (null == riga) break; // esce dal ciclo
            importi.add(riga); // aggiunge in lista
            totale += Float.parseFloat(riga); // converte da String a float
        }
        f.close(); // chiude file
        return totale; // restituisce totale al chiamante
    }
}

```

Progressi

- Passi implementati
 - lettura da file
 - calcolo del totale
- Da fare
 - controlli su valori unici
 - estrazione del massimo

```

public class CalcolaImporti { // classe Java vers 0.0.2
    private final List<String> importi = new ArrayList<>();
    private float totale, massimo;

    public float calcola(String c, String n) throws IOException {
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        // lettura file tramite le API Java: File, FileReader, LineNumberReader

        totale = massimo = 0;
        while (true) {
            final String riga = f.readLine(); // legge una riga dal file
            if (null == riga) break; // esce dal ciclo
            importi.add(riga); // aggiunge in lista
            float x = Float.parseFloat(riga); // converte da String a float
            totale += x; // aggiorna totale
            if (massimo < x) massimo = x; // aggiorna massimo
        }
        f.close(); // chiude file
        return totale; // restituisce il totale al chiamante
    }
}

```

Progressi

- Passi implementati
 - lettura da file
 - calcolo del totale
 - estrazione del massimo
- Da fare
 - controlli su valori unici

```

public class CalcolaImporti { // classe Java vers 0.1

    private final List<String> importi = new ArrayList<>();
    private float totale, massimo;

    public float calcola(String c, String n) throws IOException {
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        // lettura file tramite le API Java: File, FileReader, LineNumberReader

        totale = massimo = 0;
        while (true) {
            final String riga = f.readLine(); // legge una riga dal file
            if (null == riga) break;          // esce dal ciclo
            if (!importi.contains(riga)) {    // se non presente
                importi.add(riga);           // aggiunge in lista
                float x = Float.parseFloat(riga); // converte da String a float
                totale += x;                 // aggiorna totale
                if (massimo < x) massimo = x; // aggiorna massimo
            }
        }
        f.close(); // chiude file
        return totale; // restituisce il totale al chiamante
    }
}

```

Librerie Java

- Riepilogo di alcuni tipi e metodi di librerie Java utilizzati:
- **List**, interfaccia utile a tenere una sequenza di elementi
- **ArrayList**, implementazione di **List**, la sua dimensione cresce automaticamente
- **add()**, metodo di **List**, aggiunge un elemento alla fine della lista
- **contains()**, metodo di **List**, ritorna true se la lista contiene l'elemento specificato
- **parseFloat(String s)**, metodo di **Float**, ritorna un nuovo float con il valore specificato nel parametro stringa s, o ritorna un'eccezione se la stringa non contiene un numero
- **readLine()**, metodo di **LineNumberReader**, ritorna una stringa contenente la linea del file, o null se si raggiunge la fine del file

Progressi

- Passi implementati
 - lettura da file
 - calcolo del totale
 - estrazione del massimo
 - controlli su valori unici
- Il codice è conforme alla programmazione OO?

```

public class CalcolaImporti { // classe Java vers 0.1

    private final List<String> importi = new ArrayList<>();
    private float totale, massimo;

    public float calcola(String c, String n) throws IOException {
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        totale = 0;
        massimo = 0;
        while (true) {
            String riga = f.readLine(); // leggiFile()
            if (null == riga) break;
            if (!importi.contains(riga)) { // inserisci()
                importi.add(riga);
                float x = Float.parseFloat(riga); // calcolaSomma()
                totale += x;
                if (massimo < x) massimo = x; // calcolaMassimo()
            }
        }
        f.close();
        return totale; // getSomma()
    }
}

```

Spaghetti Code

- Metodi lunghi, senza parametri, e che usano variabili globali
- Flusso di esecuzione determinato dall'implementazione interna all'oggetto, non dai chiamanti
- Interazioni minime fra oggetti
- Nomi classi e metodi indicano la programmazione procedurale
- Ereditarietà e polimorfismo non usati, riuso impossibile
- Gli oggetti non mantengono lo stato fra le invocazioni
- Cause: inesperienza con OOP, nessuna progettazione

```
public class CalcolaImporti { // classe Java vers 0.2

    private final List<String> importi = new ArrayList<>();
    private float totale, massimo;

    public float calcola(String c, String n) throws IOException {
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        String riga;
        while (true) {
            riga = f.readLine();
            if (null == riga) break;
            if (!importi.contains(riga))
                importi.add(riga);
        }
        f.close();
        // calcola totale
        totale = 0;
        for (int i = 0; i < importi.size(); i++)
            totale += Float.parseFloat(importi.get(i));
        // calcola massimo
        massimo = Float.parseFloat(importi.get(0));
        for (int i = 1; i < importi.size(); i++)
            if (massimo < Float.parseFloat(importi.get(i)))
                massimo = Float.parseFloat(importi.get(i));
        return totale;
    }
}
```

```
public class CalcolaImporti { // classe Java vers 0.2

    private final List<String> importi = new ArrayList<>();
    private float totale, massimo;

    public float calcola(String c, String n) throws IOException {
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        String riga;
        while (true) {
            riga = f.readLine();
            if (null == riga) break;
            if (!importi.contains(riga))
                importi.add(riga);
        }
        f.close();
        totale = 0;
        for (int i = 0; i < importi.size(); i++) {
            totale += Float.parseFloat(importi.get(i));
        }
        massimo = Float.parseFloat(importi.get(0));
        for (int i = 1; i < importi.size(); i++)
            if (massimo < Float.parseFloat(importi.get(i)))
                massimo = Float.parseFloat(importi.get(i));
        return totale;
    }
}
```

Problemi?

- Il metodo `calcola` di entrambe le versioni è **spaghetti code** (un antipattern)
- Il codice è monolitico: fa troppe cose in un unico flusso. Non è un codice Object-Oriented. Conseguenze: non si può riusare, né verificarne la correttezza
 1. Come verificare che tutti i valori del file siano stati letti? Si dovrà modificare il metodo. Non è una soluzione, si dovrebbe poter verificare il comportamento del metodo dall'esterno
 2. Analogamente per verificare il calcolo del totale e del massimo, in più punti si dovrebbero aggiungere alcuni controlli
 3. Non si riesce a modificare facilmente o riusare il codice. Per es. se si volessero conservare tutti i valori letti, quali ulteriori effetti provoca la modifica?
- Quindi: difficoltà di comprensione e modifiche che coinvolgono varie operazioni

```

public class Pagamenti { // Pagamenti vers 1.1
    private List<String> importi = new ArrayList<>();
    private float totale, massimo;
    public void leggiFile(String c, String n) throws IOException {
        LineNumberReader f = new LineNumberReader(new FileReader(new File(c, n)));
        String riga;
        while (true) {
            riga = f.readLine();
            if (null == riga) break;
            inserisci(riga);
        }
        f.close();
    }
    public void inserisci(String riga) {
        if (!importi.contains(riga)) importi.add(riga);
    }
    public void calcolaSomma() {
        for (String v : importi) // enhanced for
            totale += Float.parseFloat(v);
    }
    public void calcolaMassimo() {
        for (String v : importi)
            if (massimo < Float.parseFloat(v))
                massimo = Float.parseFloat(v);
    }
    public void svuota() {
        importi = new ArrayList<>();
        totale = massimo = 0;
    }
    public float getMassimo() {
        return massimo;
    }
    public float getSomma() {
        return totale;
    }
}

```

Pagamenti
- importi: List<String> - totale: float - massimo: float
+ leggiFile(c: String, n: String) + inserisci(riga: String) + calcolaSomma() + calcolaMassimo() + svuota() + getMassimo(): float + getSomma(): float

```

// chiamate da un'altra classe
public static void main(String[] args) {
    Pagamenti p = new Pagamenti();
    try {
        p.leggiFile("csv", "importi");
    } catch (IOException e) {}
    p.svuota();
    p.calcolaSomma();
    p.calcolaMassimo();
}

```

Considerazioni sul codice

- Si sta usando bene il paradigma di programmazione ad Oggetti (OOP)
 - Ogni metodo ha una sola piccola responsabilità
 - Il flusso di chiamate ai metodi è indipendente dai singoli algoritmi
 - Posso riusare (richiamandoli) i servizi offerti dai metodi
- Inoltre, sto usando il **paradigma Command e Query**
 - I metodi Query restituiscono un risultato (si vede dal parametro di ritorno), e non modificano lo stato del sistema
 - I metodi Command (o modificatori) cambiano lo stato del sistema ma non restituiscono un valore
 - I metodi query si possono chiamare liberamente, senza preoccupazioni sulla modifica dello stato, mentre si deve stare più attenti quando si chiamano i metodi command
- **Enhanced for** indica che si vogliono gli elementi della lista, uno per ogni passata, si può usare con i tipi che implementano Iterable

Metriche

- Classe CalcolaImporti (vers. 0.1)
 - Metodi 1, LOC 26 (di cui 5 linee sono per i vari import)
- Classe CalcolaImporti (vers. 0.2)
 - Metodi 1, LOC 29
- Classe Pagamenti (vers 1.1)
 - Metodi 7, LOC 43 (media 6 LOC per metodo)
- Confronto con sistemi software open source (valori approssimativi) JUnit (JU), JHotDraw (JHD):
 - JU LOC 22K, Classi 231, Metodi 1200, Attributi 265, media 18
 - JHD LOC 28K, Classi 600, Metodi 4814, Attributi 1151, media 3

Convalida (Testing)

```

// test per la classe Pagamenti
import java.io.IOException;

public class TestPagamenti { // per Pagamenti vers 1.2
    private Pagamenti pgm = new Pagamenti();

    private void initLista() {
        pgm.svuota();
        pgm.inserisci(321.01f);
        pgm.inserisci(531.7f);
        pgm.inserisci(1234.5f);
    }

    public void testSommaValori() {
        initLista();
        if (pgm.calcolaSomma() == 2087.21f) // test che si autovaluta
            System.out.println("OK test somma val");
        else System.out.println("FAILED test somma val");
    }
}

```

Convalida (Testing)

```
...
public void testLeggiFile() {
    pgm.svuota();
    try {
        pgm.leggiFile("csvfiles", "Importi.csv");
        System.out.println("OK test leggi file");
    } catch (IOException e) {
        System.out.println("FAILED test leggi file");
    }
}

public void testMaxValore() {
    initLista();
    pgm.calcolaMassimo();
    if (pgm.calcolaMassimo() == 1234.5f)
        System.out.println("OK test massimo val");
    else System.out.println("FAILED test massimo val");
}
...

public static void main(String[] args) {
    TestPagamenti t1 = new TestPagamenti();
    t1.testLeggiFile();
    t1.testSommaValori();
    t1.testMaxValore();
}
```

Conclusioni

• Key points

- Correttezza del codice: test
- Antipattern Spaghetti Code
- Ciascun metodo ha un unico compito
- Per sistemi complessi bisogna Modellare ad oggetti: identificare e definire correttamente le entità

Scrittura di un client

```
public class ClientPagam { // versione 0.1

    public static void main(String[] args) {

        Pagamenti p = new Pagamenti(); // p è una istanza di Pagamenti
        try {
            p.leggiFile("csvfiles", "Importi.csv"); // lettura primo file
            // p contiene tutti i valori letti dal file
        } catch (IOException e) {
        }
        System.out.println("totale: " + p.calcolaSomma());
        System.out.println("max: " + p.calcolaMassimo());
    }
}
```

```
public class ClientPagam { // versione 0.2
    public static void main(String[] args) {
        Pagamenti p = new Pagamenti(); // p e' l'unica istanza
        try {
            p.leggiFile("csvfiles", "Importi.csv"); // lettura primo file
        } catch (IOException e) { }
        System.out.println("file 1 totale: " + p.calcolaSomma());
        System.out.println("file 1 max: " + p.calcolaMassimo());
        try {
            p.leggiFile("csvfiles", "PagMarzo.csv"); // lettura secondo file
            // p adesso contiene tutti i valori letti da entrambi i file
        } catch (IOException e) { }
        System.out.println("file 1 e 2 totale: " + p.calcolaSomma());
        System.out.println("file 1 e 2 max: " + p.calcolaMassimo());
    }
}
```

```

public class ClientPagam { // versione 0.3
    public static void main(String[] args) {
        Pagamenti p1 = new Pagamenti(); // prima istanza
        Pagamenti p2 = new Pagamenti(); // seconda istanza

        try {
            p1.leggiFile("csvfiles", "Importi.csv"); // lettura primo file
            p2.leggiFile("csvfiles", "PagMarzo.csv"); // lettura secondo file
        } catch (IOException e) {
        }
        System.out.println("file 1 totale: " + p1.calcolaSomma());
        System.out.println("file 1 max: " + p1.calcolaMassimo());
        System.out.println("file 2 totale: " + p2.calcolaSomma());
        System.out.println("file 2 max: " + p2.calcolaMassimo());
    }
}

```

Incapsulamento Ereditarietà Polimorfismo

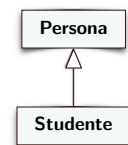


Riuso di classi

- Spesso si ha bisogno di classi simili
 - Si vuole cioè **riusare** classi esistenti per implementare attributi e metodi leggermente diversi
- Riuso implementazioni esistenti, **SENZA RICOPIARE LA CLASSE**
- Il riuso delle classi esistenti deve avvenire
 - Senza dover modificare codice esistente (e funzionante)
 - In modo semplice per il programmatore
- Due le strade possibili: statica (ereditarietà) e dinamica (composizione)

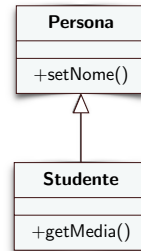
class inheritance and subtyping

- Class inheritance defines an object's implementation in terms of another object's implementation.
- Abstract feature inheritance (or subtyping) describes when an object can be used in place of another.
- No distinction in Java/C++ (use Abstract classes)
- Vedi codice di esempio "animas"



Ereditarietà e classi

- Attraverso l'ereditarietà è possibile
 - Definire una nuova classe indicando solo cosa ha in più rispetto ad una classe esistente: ovvero attributi e metodi nuovi, e modificando i metodi esistenti
 - Esempio: una classe Persona ha nome e cognome (più vari metodi)
 - La classe Studente dovrebbe avere tutto ciò che Persona ha (attributi e metodi) e nuovi attributi e metodi
 - Studente aggiunge esami, voti, etc.
 - La classe Studente eredita da Persona
- ```
public class Studente extends Persona { ... }
```
- Studente è sottoclasse di Persona e Persona è superclasse di Studente



```

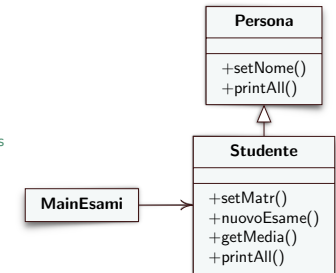
public class Persona {
 public void setNome(String n, String c) { ... }
 public void printAll() { ... }
}

public class Studente extends Persona {
 public void setMatr(String m) { ... }
 public void nuovoEsame(String m, int v) { ... }
 public float getMedia() { ... }
 public void printAll() { ... }
}

public class MainEsami {
 public static void main(String[] args) {
 Studente s = new Studente();
 s.setNome("Alan", "Rossi"); // metodo della superclasse di s
 s.setMatr("M12345");
 s.nuovoEsame("Italiano", 8); // metodo della classe di s
 s.printAll(); // metodo della classe di s

 s.nuovoEsame("Fisica", 7);
 Persona p = s; // p e' dichiarato di tipo Persona
 p.printAll(); // a runtime p punta all'istanza s
 //p.nuovoEsame("Geometria", 9); SEGNALATO IN COMPILAZIONE!
 }
}

```



```

public class Persona {
 private String nome, co;
 public void setNome(String n, String c) {
 nome = n; co = c;
 }
 public void printAll() {
 System.out.println(nome + " " + co);
 }
}

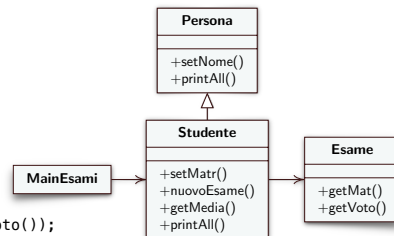
public class Studente extends Persona {
 private String matr;
 private List<Esame> esami = new ArrayList<>();
 public void setMatr(String m) { matr = m; }
 public void nuovoEsame(String m, int v) {
 Esame e = new Esame(m, v);
 esami.add(e);
 }
 public float getMedia() {
 if (esami.isEmpty()) return 0;
 float sum = 0;
 for (Esame e : esami) sum += e.getVoto();
 return sum / esami.size();
 }
 public void printAll() {
 super.printAll();
 System.out.println("matr: " + matr);
 for (Esame e : esami)
 System.out.println(e.getMat() + ": " + e.getVoto());
 System.out.println("media: " + getMedia());
 }
}

```

```

public class Esame {
 private String mat;
 private int voto;
 public Esame(String n, int v) {
 mat = n; voto = v;
 }
 public String getMat() {
 return mat;
 }
 public int getVoto() {
 return voto;
 }
}

```



## Modificatori di accesso in Java

### Access Modifiers

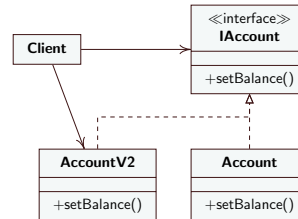
| Modifier  | Class | Package | Subclass | Global |
|-----------|-------|---------|----------|--------|
| Public    | ✓     | ✓       | ✓        | ✓      |
| Protected | ✓     | ✓       | ✓        | ✗      |
| Default   | ✓     | ✓       | ✗        | ✗      |
| Private   | ✓     | ✗       | ✗        | ✗      |

## Classi ed interfacce

- Un client che usa un'interfaccia rimane immutato quando l'implementazione dell'interfaccia cambia

```
public class AccountV2 implements IAccount {
 public void setBalance() { }
}
```

```
public class Client {
 public void main(String[] args) {
 IAccount a = new AccountV2();
 a.setBalance();
 }
}
```



## Classi Astratte in Java

```
public abstract class Libro {
 private String autore;

 // Niente (obbligo di) implementare un costruttore...

 public abstract void view(); // forzo le derivate ad implementarla

 public String getAutore() {
 return autore;
 }
}
```

## Interfaces:

Five blind philosophers touch different parts of an elephant



## Interface inheritance

- Interfaces only partially describes an object's features
- An object can inherit (implement) multiple interfaces
- No relationship between interfaces
  
- Interface inheritance should not be used to represent structural subtype (is-a) relations between objects

## Interface

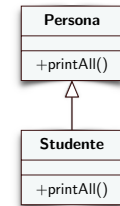
```
public interface IAccount {
 public void setBalance();
}
```

Da java 8:

- metodi static/default/private
- attributi static/final

## Late binding e polimorfismo

```
public void m() {
 Persona p = new Persona();
 Studente s = new Studente();
 Persona px;
 if (i > 10)
 px = p;
 else
 px = s;
 px.printAll();
}
```

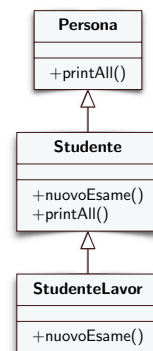


## Polimorfismo

- Più metodi con la stessa signature in classi diverse
- la chiamata ad un metodo può avere effetti diversi, ovvero il comportamento è polimorfo

```
Studente s;
// ...
s.nuovoEsame("Maths", 8);
s.printAll();
```

- quale metodo nuovoEsame() sarà eseguito è deciso a runtime, in base all'istanza puntata dalla variabile s.
- Lo stesso vale quando si ha una variabile di tipo Persona e per il metodo printAll()

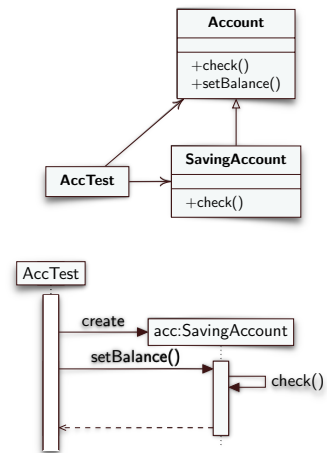


## Sottoclassi e dispatch

```
public class Account {
 protected float balance;
 public void setBalance(float amount) {
 System.out.println("in account set-balance");
 if (check(amount))
 balance += amount;
 }
 public boolean check(float amount) {
 System.out.println("in account check");
 return (balance + amount) >= 0;
 }
}

public class SavingAccount extends Account {
 public boolean check(float amount) {
 System.out.println("in saving-account check");
 return (balance + amount) >= 1000;
 }
}

public class AccTest {
 public static void main(String[] args) {
 Account acc = new SavingAccount();
 acc.setBalance(1234);
 }
}
```



## Dynamic casting: Tipi di variabili e tipi a runtime

- Una variabile di un tipo può tenere il riferimento ad un'istanza del sottotipo (la classe `Studiante` è sottotipo di `Persona`)

```
Persona p = new Studiante();
```

- Tramite cast forzo temporaneamente `p` di tipo `Persona` ad un altro tipo

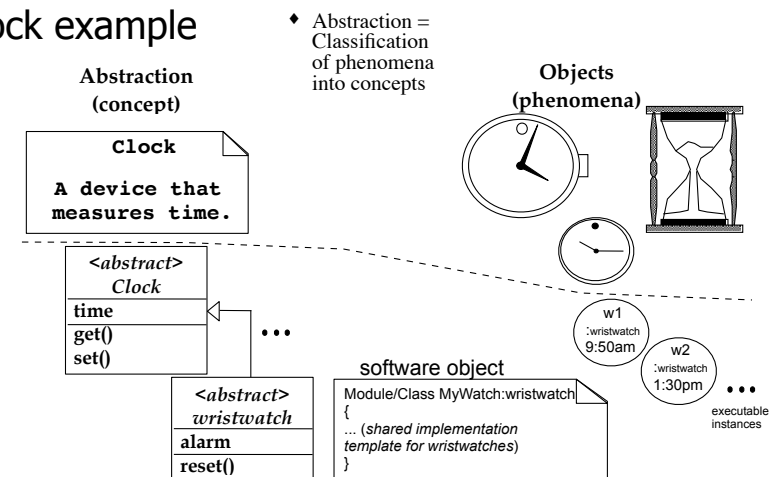
```
Studiante s = (Studiante) p;
```

- OK per il compilatore, ma OK a runtime solo se l'istanza corrente di `p` è effettivamente compatibile (sottotipo) con il tipo cui lo assegna: cioè `studiante` (covarianza)
- ovvero `s` di un tipo compatibile (supertipo) con il tipo che gli assegna: cioè `persona` (controvarianza)
- Con `s = (Studiante) new Persona()` si avrebbe a runtime `ClassCastException`

# Designing Reusable Code



## Clock example



## Design Principle

### Principio della singola responsabilità

Design methods to perform a single specific task, related to its defining class/ADT

Design Classes/ADT to clearly represent one single concept



## Design Principle

### Program to an interface, not an implementation:

Use abstract classes (and/or interfaces in Java) to define common interfaces for a set of classes

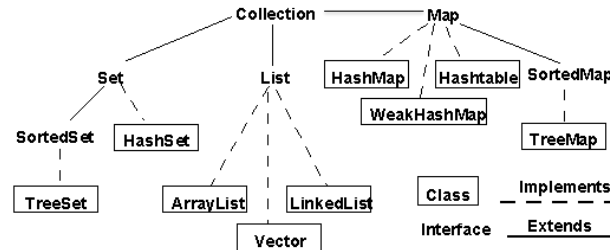
Declare variables to be instances of the abstract class or interface, not instances of particular classes



## Programming to an Interface: Java Collections

```
Collection students = new XXX;
students.add(aStudent);
```

- students can be any collection type
- We can change our mind on what type to use



## Inheritance vs Composition

### Inheritance

```
class A {
 Foo x
 public int complexOperation() { ... }
}

class B extends A {
 public void bar() { ... }
}
```

### Composition

```
class B {
 A myA;
 public int complexOperation() {
 return myA.complexOperation();
 }
 public void bar() { ... }
}
```

## Design Principle

### Favour object composition over class inheritance

Composition...

- Allows behaviour changes at run time
- Helps keep classes encapsulated and focused on one task (coesione)
- Reduce implementation dependencies (accoppiamento)



## Design Principle

### Use Parametrised types

- Parameterized types give a third way to compose behavior in an object-oriented system
- It gives the flexibility of dynamic (run-time) binding, just as like composition does



## Parameterized Types

- Generics in Ada, Eiffel, Java
- Templates in C++
- Allows you to make a type as a parameter to a method or class

```
template <class TypeX>
TypeX min(TypeX a, TypeX b)
{
 return a < b ? a : b;
}
```

## Design... patterns?

- Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over

[Christopher Alexander on architecture patterns]

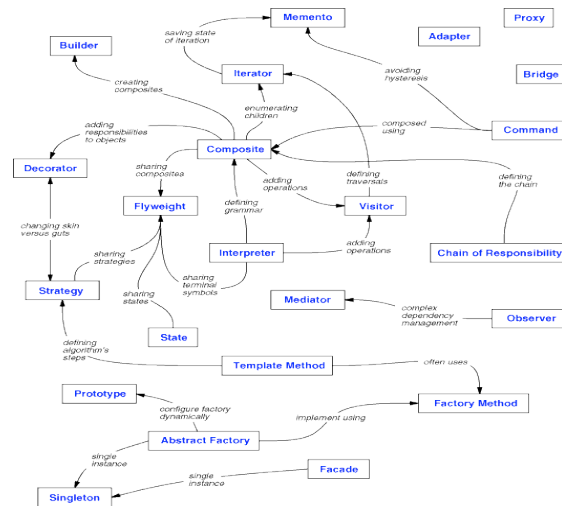


## Four parts...

- The **pattern name** Finding good names has been one of the hardest parts of developing our catalog.
- The **problem** describes when to apply the pattern. It explains the problem and its context.
- The **solution** describes the elements that make up the design, their relationships, responsibilities, and collaborations.
- The **consequences** are the results and trade-offs of applying the pattern., they are critical for evaluating design alternatives and for understanding the costs and benefits of applying the pattern.

## GoF pattern catalog

|       |        | Purpose                                                                    |                                                                                                    |                                                                                                                                                                                            |
|-------|--------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |        | Creational                                                                 | Structural                                                                                         | Behavioral                                                                                                                                                                                 |
| Scope | Class  | Factory Method (107)                                                       | Adapter (139)                                                                                      | Interpreter (243)<br>Template Method (325)                                                                                                                                                 |
|       | Object | Abstract Factory(87)<br>Builder (97)<br>Prototype (117)<br>Singleton (127) | Adapter (139)<br>Bridge (151)<br>Composite (163)<br>Decorator (175)<br>Facade (185)<br>Proxy (207) | Chain of Responsibility (223)<br>Command (233)<br>Iterator (257)<br>Mediator (273)<br>Memento (283)<br>Flyweight (195)<br>Observer (293)<br>State (305)<br>Strategy (315)<br>Visitor (331) |



## Design for change

- **Creating an object by specifying a class explicitly**

Abstract factory, Factory Method, Prototype

- **Dependence on specific operations**

Chain of Responsibility, Command

- **Dependence on hardware and software platforms**

Abstract factory, Bridge

- **Dependence on object representations or implementations**

Abstract factory, Bridge, Memento, Proxy

- **Algorithmic dependencies**

Builder, Iterator, Strategy, Template Method, Visitor

- **Tight Coupling**

Abstract factory, Bridge, Chain of Responsibility, Command, Facade, Mediator, Observer

- **Extending functionality by subclassing**

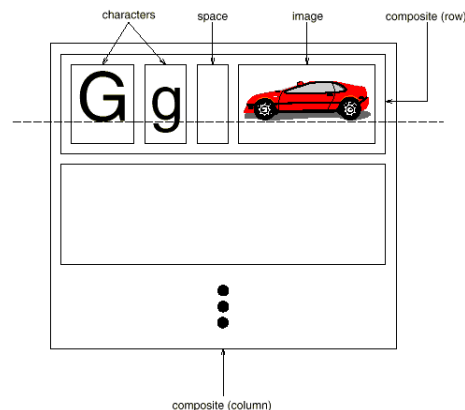
Bridge, Chain of Responsibility, Composite, Decorator, Observer, Strategy

- **Inability to alter classes conveniently**

Adapter, Decorator, Visitor

## Patterns case study: Document editor

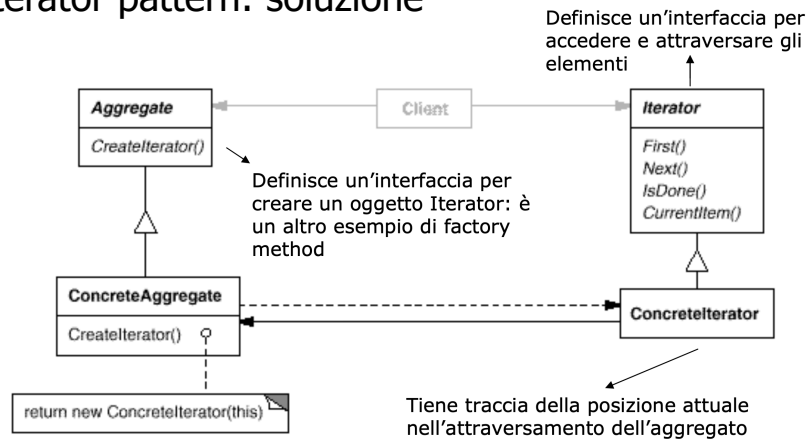
- design problem: how to design a heterogeneous data structure so to allow for easy navigation of its elements?



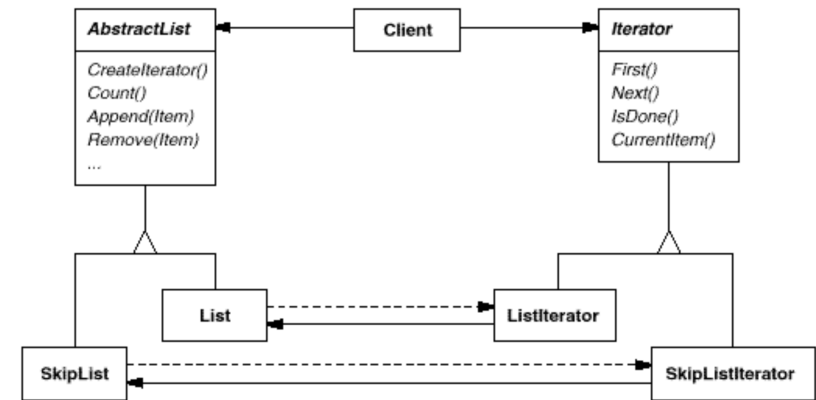
## Pattern Iterator

- **Intento:** Attraversare gli elementi di un aggregato senza esporre la sua struttura (iterazione polimorfica)
  - l'iterator conosce i dettagli della classe che visita, mentre al client restano nascosti
- E' possibile definire più *concrete iterator* che visitano lo stesso contenitore, anche con politiche diverse
- Sia Java che la STL del linguaggio C++ fanno ampio uso del pattern iterator

## Iterator pattern: soluzione



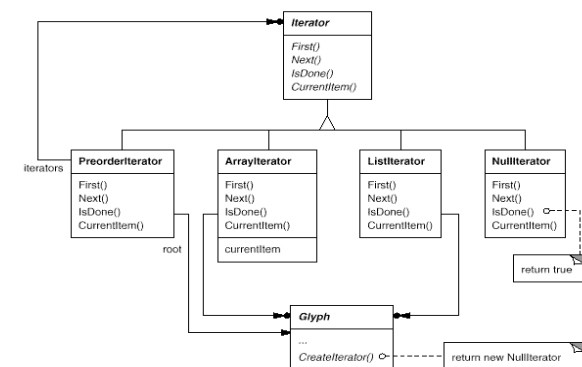
## Iterator example



## Iterator pattern: considerazioni

- Fornisce una interfaccia **uniforme** per accedere agli elementi di un **aggregato astratto** in maniera **sequenziale**
  - Toglie le funzionalità per l'attraversamento dall'aggregato stesso e le incapsulo in un oggetto **Iterator, da istanziare separatamente**
  - L'iteratore, e non più il client, ha **accesso privilegiato** a (ovvero, deve conoscere) l'esatta l'implementazione dell'aggregato
- Il codice del **client** diventa facilmente **riusabile** in quanto generico
  - Svincolato dall'implementazione dell'aggregato, che è visto come astratto
- Posso avere contemporaneamente **più attraversamenti** (iteratori) indipendenti dello stesso aggregato
  - Ogni istanza di Iterator è responsabile di tener traccia dell'**elemento corrente**
  - In alcuni casi serve tener **traccia** di quali elementi sono già stati attraversati
- Posso supportare **più modalità differenti di attraversamento** (ad. es. reverse o filtrato)
  - implemento un ConcreteIterator separato per ogni modalità
  - Evita di appesantire l'interfaccia dell'aggregato (e la sua implementazione) con tali operazioni

## Iterator Pattern applied



# Java Collection framework

Program to an interface



## JAVA Collections Framework

- <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html>

The collections framework is a unified architecture for representing and manipulating collections, enabling collections to be manipulated independently of implementation details.

- **Reduces programming effort** by providing data structures and algorithms so you don't have to write them yourself.
- **Increases performance** by providing high-performance implementations of data structures and algorithms. Because the various implementations of each interface are interchangeable, programs can be tuned by switching implementations.
- **Fosters software reuse** by providing a standard interface for collections and algorithms with which to manipulate them.

OVERVIEW PACKAGE **CLASS** USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3  
java.util

**Interface Collection<E>**

Type Parameters:  
E - the type of elements in this collection

All Superinterfaces:  
Iterable<E>

All Known Subinterfaces:  
BeanContext, BeanContextServices, BlockingDeque<E>, BlockingQueue<E>, Deque<E>, List<E>, NavigableSet<E>, Queue<E>, Set<E>, SortedSet<E>, TransferQueue<E>

All Known Implementing Classes:  
AbstractCollection, AbstractList, AbstractQueue, AbstractSequentialList, AbstractSet, ArrayBlockingQueue, ArrayDeque, ArrayList, AttributeList, BeanContextServicesSupport, BeanContextSupport, ConcurrentHashMap.KeySetView, ConcurrentLinkedDeque, ConcurrentLinkedQueue, ConcurrentSkipListSet, CopyOnWriteArrayList, CopyOnWriteArraySet, DelayQueue, EnumSet, HashSet, JobStateReasons, LinkedBlockingDeque, LinkedBlockingQueue, LinkedHashSet, LinkedList, LinkedTransferQueue, PriorityBlockingQueue, PriorityQueue, RoleList, RoleUnresolvedList, Stack, SynchronousQueue, TreeSet, Vector

## JAVA Collections Framework

| Interface | Hash Table | Resizable Array | Balanced Tree | Linked List | Hash Table + Linked List |
|-----------|------------|-----------------|---------------|-------------|--------------------------|
| Set       | HashSet    |                 | TreeSet       |             | LinkedHashSet            |
| List      |            | ArrayList       |               | LinkedList  |                          |
| Deque     |            | ArrayDeque      |               | LinkedList  |                          |
| Map       | HashMap    |                 | TreeMap       |             | LinkedHashMap            |

...(r)esistono ancora Vector (sync., List), Hashtable (synchronized, Map)

## ArrayList vs Vector

ArrayList and Vector both implements List interface and maintains insertion order.

However, there are many differences between ArrayList and Vector classes that are given below.

### ArrayList

ArrayList is **not synchronized**.

ArrayList **increments 50%** of current array size if the number of elements exceeds from its capacity.

ArrayList is **not a legacy** class. It is introduced in JDK 1.2.

ArrayList is **fast** because it is non-synchronized.

ArrayList uses the **Iterator** interface to traverse the elements.

### Vector

Vector is **synchronized**.

Vector **increments 100%** means doubles the array size if the total number of elements exceeds than its capacity.

Vector is a **legacy** class.

Vector is **slow** because it is synchronized, i.e., in a multithreading environment, it holds the other threads until current thread releases the lock of the object.

A Vector can use the **Iterator** interface or **Enumeration** interface to traverse the elements.

## Collection e il pattern Iterator

### Java Vector iterator

```
Vector listOfStrings = new Vector();
```

```
// code to add Strings not shown
```

```
Iterator list = listOfStrings.iterator();
while (list.hasNext()){
 Student x = list.next();
 System.out.println(x);
}
```

## Uso dell'iteratore

```
void print(ArrayList list)
{
 for(int k = 0; k < list.size(); k++)
 System.out.println(list.get(k));
}
```

```
void print(LinkedList list)
{
 Node current = list.first();
 System.out.println(current);
 while (current.hasNext())
 {
 current = current.next();
 System.out.println(current);
 }
}
```

//Contro...

```
void print(Collection list)
{
 Iterator items = list.iterator();
 while (items.hasNext())
 {
 System.out.println(items.next());
 }
}
```

- Iterator abstracts out underlying representation of collection
- Programmer does not have to know implementation details of each type of collection
- Can write code that works for wide range of collects
- Do not have to change code if change the type of collection used

## Ways to traverse collections in Java:

1. By using Iterable interface (enhanced for)
2. By using Iterable interface (forEach method)
3. By using Iterator object explicitly (iterator method)
4. By using ListIterator object (List only)
5. By using Enumeration (legacy only)
6. By using for loop (no iterator)
7. By using for-each (stream interface)

## Enumeration, Iterator, and Streams in Java are iterators

```
List listOfStrings = new Vector<String>();
```

```
//con for loop che "entra" nella collection...
for (int i=0; i<listOfStrings.size();i++)
 System.out.println(listOfStrings.get(i));
```

```
//con Iterator hasNext() e next()... (iterator esplicito)
```

```
Iterator<String> list = listOfStrings.iterator();
while (list.hasNext())
 System.out.println(list.next());
```

```
//con Enhanced for loop (ogni Collection è Iterable)
```

```
for(String s: listOfStrings) System.out.println(s);
```

```
//con metodo forEach di Iterable... (iterator implicito)
```

```
sameList.forEach(System.out::println);
listOfStrings.forEach(System.out::println);
```

- An Iterable is a simple representation of a series of elements that can be iterated over.
- It does not have any iteration state such as a "current element".
- Instead, it has one method that produces an Iterator.
- An Iterator is the object with iteration state.

## Enumeration, Iterator, and Streams in Java are iterators

```
//con Enumeration interface (faster implem.)...
```

```
Vector v = new Vector<String>(List.of("ciao", "mamma", "guarda", "come", "mi", "diverto"));
Enumeration<String> e = v.elements(); // oggetto analogo all'iterator
```

```
while(e.hasMoreElements()) System.out.println(e.nextElement());
```

```
//con ListIterator interface (backward and replace)...
```

```
List<String> a = new ArrayList<String>(List.of("ciao", "mamma", "guarda", "come", "mi", "diverto"));
ListIterator i = a.listIterator(a.size());
```

```
while(i.hasPrevious()) System.out.println(i.previous());
```

```
//con metodo forEach() dell'interfaccia Stream...
```

```
import java.util.stream
// package con la definizione di: Interface Stream<T>
```

```
listOfStrings.stream().forEach((c) -> System.out.println(c));
```

- (1) metodo stream() definito nell'interfaccia Collection, torna uno Stream
- (2) List.of() metodo utility, da Java 9

| Property       | Enumeration                                     | Iterator                                           | ListIterator                                        |
|----------------|-------------------------------------------------|----------------------------------------------------|-----------------------------------------------------|
| Applicable for | Only legacy classes                             | Any Collection classes                             | Only List classes                                   |
| Movement       | Only forward direction(single direction)        | Only forward direction(single direction)           | Both forward and backward direction(bi directional) |
| Accessibility  | Only read access                                | Both read and remove                               | Read ,remove, replace and addition of new objects   |
| How to get it? | By using elements() method of Vector class      | By using iterator() method of Collection interface | By using listIterator() method of List interface    |
| Methods        | 2 methods<br>hasMoreElements()<br>nextElement() | 3 methods<br>hasNext ()<br>next()<br>remove()      | 9 methods                                           |
| Is it legacy   | "yes" (1.0v)                                    | "no" (1.2V)                                        | "no" (1.2V)                                         |

## Esercitazione per casa

- implementare un ADT albero binario di ricerca contenente interi.
- Inserimento di un nuovo dato.
- Ricerca booleana di un dato
- scrivere anche un programma client di test delle funzionalità

# Null Object

object oriented null



## Binary Search Tree

```
public class BinaryNode {
 Node left = null;
 Node right = null;
 int key;

 public boolean includes(int value) {
 if (key == value)
 return true;
 else if ((value < key) & left == null))
 return false;
 else if (value < key)
 return left.includes(value);
 else if (right == null)
 return false;
 else
 return right.includes(value);
 }
 etc.
}
```

## Refactoring...

- Introduce Null Object
- You have repeated checks for a null value?
- Replace the null value with a null object

If (customer==null) plan = new BasicBillingPlan();

else plan = customer.getPlan();

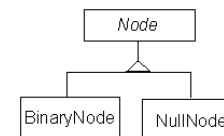
Becomes:

plan = customer.getPlan();

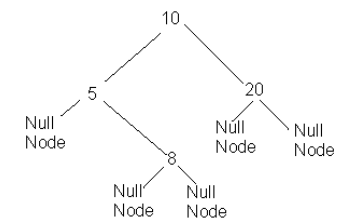
```
Class Nullcustomer {
 getPlan() { return new BasicBillingPlan(); }
}
```

## Binary Search Tree Example

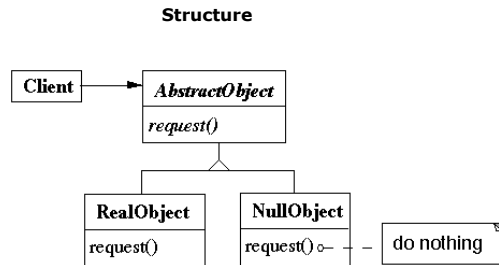
Class Structure



Object Structure



## Null Object



NullObject implements all the operations of the real object,  
These operations do nothing or the correct thing for nothing

## Applicabilità

Use the Null Object pattern when:

- Some collaborator instances should do nothing
- You want clients to ignore the difference between a collaborator that does something and one that does nothing

Client does not have to explicitly check for null or some other special value

- You want to be able to reuse the do-nothing behavior so that various clients that need this behavior will consistently work in the same way

## Binary Search Tree With Null Object

```
public class BinaryNode extends Node {
 Node left = new NullNode();
 Node right = new NullNode();
 int key;

 public boolean includes(int value) {
 if (key == value)
 return true;
 else if (value < key)
 return left.includes(value);
 else
 return right.includes(value);
 }
 etc.
}

public class NullNode extends Node {
 public boolean includes(int value) {
 return false;
 }
 etc.
}
```

## Considerazioni

Vantaggi:

- **Simplifies** client code
- **Incapsula** il “do nothing” behaviour e lo rende riusabile
- spesso usabile insieme al design pattern “**Singleton**”

Svantaggi:

- **Forces uniformity** :Different clients may have different idea of what “do nothing” means  
Rimedio: definisci più tipi nullObject  
May cause class explosion
- **Is non-mutable**  
NullObject objects cannot transform themselves into a RealObject  
Rimedio: dagli un metodo per sostituirsi (in modo trasparente) con un realObject

# Singleton



## Design pattern Singleton

- Intento

- Assicurare che una classe abbia una sola istanza e fornire un punto di accesso globale all'istanza

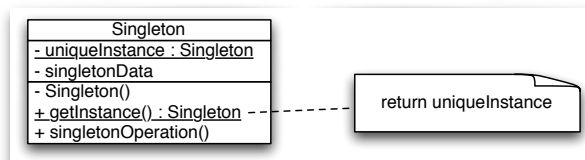
- Motivazione

- Alcune classi dovrebbero avere esattamente una istanza in tutta l'applicazione, es. uno spooler di stampa, un file system, un window manager, una lista clienti, etc.

- Una variabile globale rende un oggetto accessibile ma non proibisce di avere più oggetti per una classe. Si vuole invece proibirlo

- La classe stessa dovrebbe essere responsabile di tener traccia del suo unico punto di accesso

## Singleton



- Soluzione

- La classe che deve essere un Singleton dovrà implementare un'operazione getInstance() sulla classe (ovvero, in Java è un metodo static) che ritorna l'unica istanza creata

- La classe Singleton è responsabile per la creazione dell'istanza

- Il costruttore della classe Singleton è privato, così da non permettere la creazione tramite new ad altre classi

## Esempio classe Singleton: Fibonacci

```
// Classe Singleton che tiene una lista
// di interi
public class Fib {
 // l'unica istanza e' tenuta da obj
 private static Fib obj = new Fib();

 private int[] x = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144};
 private int i = 3;

 private Fib() {
 }
 public static Fib getInstance() {
 return obj; // restituisce l'istanza
 }
 public int getValue() {
 if (i < 11) i++;
 return x[i-1];
 }
 public void revert() {
 i = 0;
 }
}
```

```
public class MainFib {
 public static void main(String[] args) {
 // richiede una istanza di Fib
 Fib f = Fib.getInstance();
 System.out.print("f "+f.getValue());
 System.out.println(" "+f.getValue());

 // richiede una nuova istanza
 Fib f2 = Fib.getInstance();
 System.out.print("f2 "+f2.getValue());
 System.out.println(" "+f2.getValue());

 // Si ha un errore a compile-time con:
 // Fib f3 = (Fib) f2.clone();
 // Fib f4 = new Fib();
 }
}
```

Output dell'esecuzione  
f 5 8  
f2 13 21

```

public class Logs { // Classe Singleton

 private static Logs obj; // obj tiene l'istanza
 private List<String> l; // tiene i dati da registrare

 private Logs() { empty(); // costruttore è privato }

 public static Logs getInstance() { // restituisce l'unica istanza
 if (null == obj) obj = new Logs(); // crea l'istanza se non presente
 return obj;
 }

 public void record(String s) { l.add(s); // accoda il dato }

 public String dumpLast() { return l.get(l.size()-1); // restituisce l'ultimo }

 public String dumpAll() { // restituisce tutti i dati
 String acc = "";
 for (String s : l) acc = acc.concat(s);
 return acc;
 }

 public void empty() { l = new ArrayList<>(); }

}

```

## Esempio: logger

```

public class TestLogs {
 private Logs lg = Logs.getInstance();

 public void testSingl() {
 initLogs();
 Logs lg2 = Logs.getInstance();
 lg2.record("uno");
 lg2.record("due");
 if (lg.dumpLast().equals("due"))
 System.out.println("OK test logs singl");
 else
 System.out.println("FAILED test logs singl");
 }

 public void testLast() {
 initLogs();
 if (lg.dumpLast().equals("three "))
 System.out.println("OK test logs last");
 else
 System.out.println("FAILED test logs last");
 }

 public void testAll() {
 initLogs();
 if (lg.dumpAll().equals("one two three "))
 System.out.println("OK test logs all");
 else
 System.out.println("FAILED test logs all");
 }
}

```

## Test per classe Logs

```

private void initLogs() {
 lg.empty();
 lg.record("one ");
 lg.record("two ");
 lg.record("three ");
}

public static void
main(String[] args) {
 TestLogs tl = new TestLogs();
 tl.testSingl();
 tl.testAll();
 tl.testLast();
}

```

```

Output dell'esecuzione
OK test logs singl
OK test logs all
OK test logs last

```

## Conseguenze del Singleton

- La classe che è un Singleton ha pieno controllo di come e quando i client accedono alla sua unica istanza
- Evita che esistano variabili globali che tengono la sola istanza condivisa
- Permette di controllare il numero di istanze create in un programma, facilmente ed in un solo punto
- La soluzione è più flessibile rispetto a quella di usare `static` per tutte le operazioni e le variabili, poiché si può cambiare facilmente il numero di istanze consentite
- L'oggetto condiviso è comunque una normale istanza di classe, non l'ho resa statica.
- L'unico frammento di codice da modificare quando si vuol variare il numero di istanze create è quello della classe che è Singleton, mentre usando `static` si dovrebbero modificare tutte le invocazioni

# Modeling with UML



## Why model software?

Software is getting increasingly more **complex**

Windows 10 > 40 millions lines of code

A single programmer cannot manage this amount of code in its entirety.

Code is not easily understandable by developers who did not write it

We need simpler representations for complex systems

Modeling is a mean for dealing with **complexity**

## Dealing with Complexity

Object oriented design give us these tools:

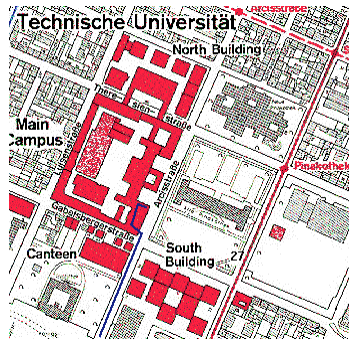
1. Abstraction
2. Decomposition
3. Hierarchy

## 1. Abstraction

- Inherent human limitation to deal with complexity
  - The 7 +- 2 phenomena
- Chunking: Group collection of objects
- Ignore unessential details: => define Models

## What is modeling?

- Modeling consists of building an abstraction of reality.
- Abstractions are **simplifications** because:
  - They ignore irrelevant details and
  - They only represent the relevant details
- What is relevant or irrelevant **depends on the purpose** of the model.

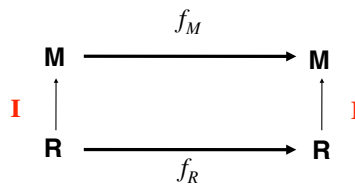


## OO modelling methodologies

- Late 80's early 90: several OO methodologies developed
  - different notations
  - different processes
- Main approaches
  - Booch
  - Rumbaugh
  - Jacobson
- Unificati in UML...

## What is a "good" model?

- $I$ : Mapping of real things in reality  $R$  to abstractions in the model  $M$  abbildet (Interpretation)
- $f_M$ : relationship between abstractions in  $M$
- $f_R$ : relationship between real things in  $R$



Relationships, which are valid in reality  $R$ , must also be valid in model  $M$ .

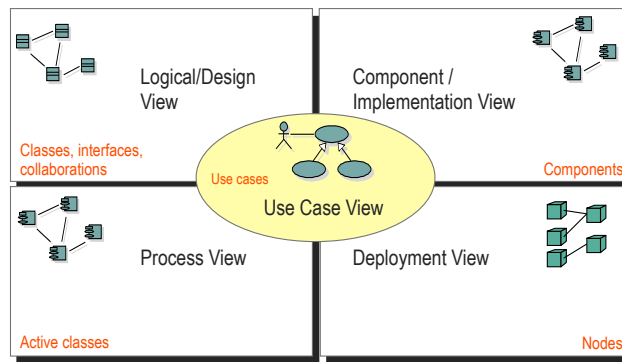
## Systems, Models and Views

- ♦ A **model** is an abstraction describing a subset of a system
- ♦ A **view** depicts selected aspects of a model
- ♦ A **notation** is a set of graphical or textual rules for depicting views
- ♦ Views and models of a single system may overlap each other

Examples:

- ♦ System: Aircraft
- ♦ Models: Flight simulator, scale model
- ♦ Views: All blueprints, electrical wiring, fuel system

## UML ha 4+1 'viste'



## Diagrammi UML

- **Diagramma strutturale (structure diagram)**
  - Diagramma delle classi (class diagram)
  - Diagramma dei componenti (component diagram)
  - Diagramma di struttura composita (composite structure diagram)
  - Diagramma di deployment (deployment diagram)
  - Diagramma degli oggetti (object diagram)
  - Diagramma dei package (package diagram)
- **Diagramma comportamentale (behavior diagram)**
  - Diagramma delle attività (activity diagram)
  - Diagramma dei casi d'uso (use case diagram)
  - Diagramma di macchina a stati (state machine diagram)
  - Diagramma di interazione (interaction diagram)
    - Diagramma di sequenza (sequence diagram)
    - Diagramma di comunicazione (communication diagram)
    - Diagramma di interazione generale (interaction overview diagram)
    - Diagramma di temporizzazione (timing diagram)

## Models are used to provide abstractions

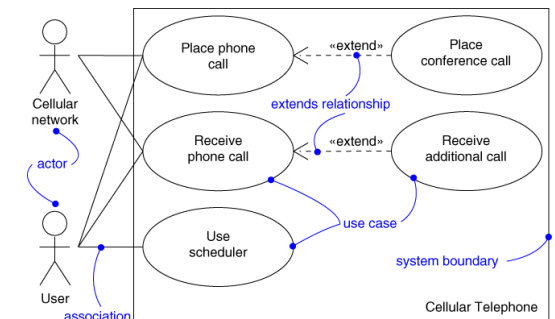
- System Models:
  - **Functional model:** What are the functions of the system? How is data flowing through the system? ==>(use case diagrams)
  - **Object Model:** What is the structure of the system? What are the objects and how are they related? ==>(class diagrams)
  - **Dynamic model:** How does the system react to external events? How is the event flow in the system? ==>(sequence/activity/state diagrams)

## Use Case Diagram: Functional modelling

An actor models an external entity which communicates with the system:

User  
External system  
Physical environment

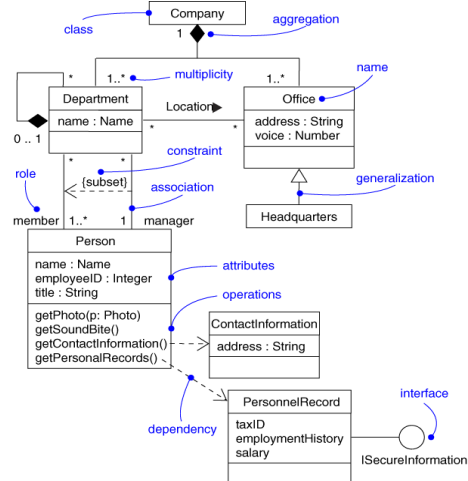
Use Cases are the user-tangible functionalities



- Bridging the gap between user and developer:
  - *Scenarios:* Example of the use of the system in terms of a series of interactions with between the user and the system (one story card)
  - *Use cases:* Abstraction that describes a class of scenarios

## Object Modeling: Class Diagram

Captures the  
vocabulary of  
a domain or a  
system

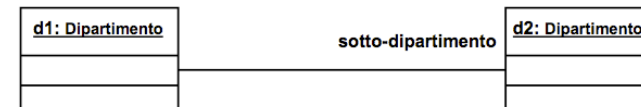


## Ruoli

### Autorelazione nel diagramma delle classi



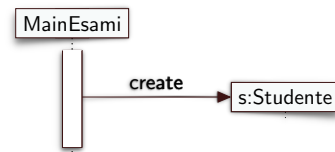
### I ruoli discriminano tra oggetti dello stesso tipo



## Diagramma UML di sequenza

- Esempio di creazione di un'istanza della classe **Studente**

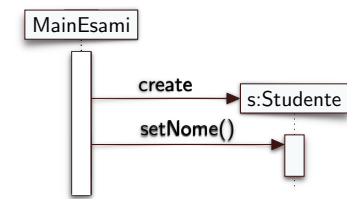
```
public class MainEsami {
 public static void main(String[] args)
 {
 Studente s = new Studente();
 }
}
```



## Diagramma UML di sequenza

- Esempio di chiamata di metodo su un'istanza di **Studente**

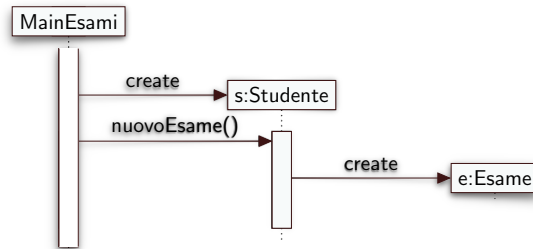
```
public class MainEsami {
 public static void main(String[] args)
 {
 Studente s = new Studente();
 s.setNome("Alan", "Rossi");
 }
}
```



## Diagramma UML di sequenza

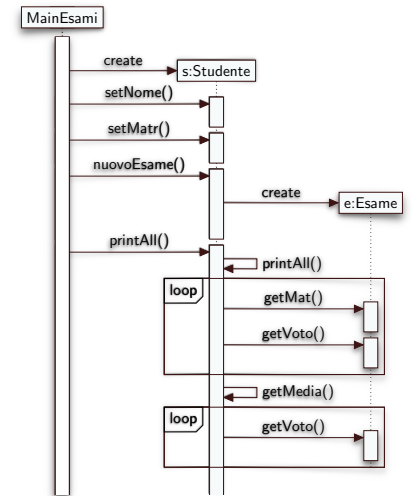
- Esempio di chiamate di metodo a cascata

```
public class MainEsami {
 public static void main(String[] args)
 {
 Studente s = new Studente();
 s.nuovoEsame("Italiano", 8);
 }
}
```



## Diagramma UML di sequenza

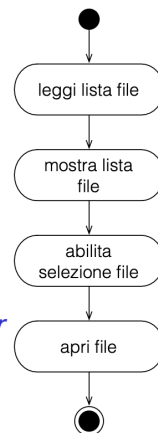
- Mostra interazioni fra istanze di oggetti
- L'asse temporale è inteso in verticale verso il basso
- In alto in orizzontale ci sono vari oggetti
- In ciascuna colonna se l'oggetto esiste è indicato con una linea tratteggiata, detta linea della vita, e se è attivo con una barra di attivazione
- Una chiamata di metodo è indicata da una freccia piena che va dalla barra di attivazione di un oggetto ad un altro



## Diagramma delle attività

- Utilizzato per la modellazione di processi e workflow
- Mostra dettagli di funzionamento interno del sistema software da realizzare
- E' una vista sull'esecuzione delle attività
- Mostra dipendenze tra attività (o passi)

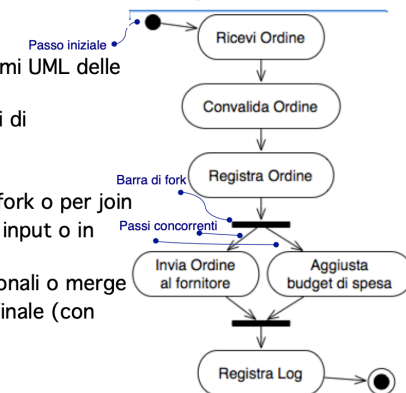
*Attività: Apri file da browser*



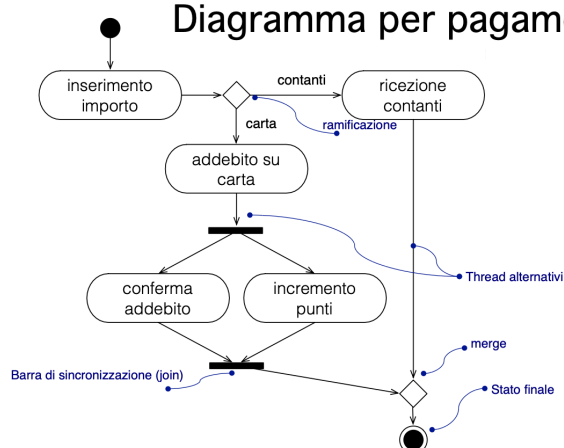
## Diagramma attività UML per ordini

Notazione grafica per diagrammi UML delle attività

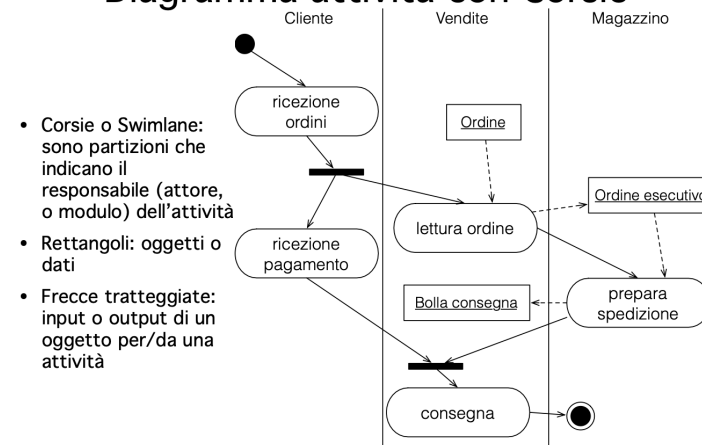
- Rettangoli arrotondati: passi di elaborazione
- Frecce continue: flussi
- Barre: sincronizzazione per fork o per join
- Rettangoli: oggetti o dati in input o in output
- Rombi: ramificazioni condizionali o merge
- Cerchi pieni: stati iniziale e finale (con circonferenza)



## Diagramma per pagamento



## Diagramma attività con Corsie



- Corsie o Swimlane: sono partizioni che indicano il responsabile (attore, o modulo) dell'attività
- Rettangoli: oggetti o dati
- Freccie tratteggiate: input o output di un oggetto per/da una attività

## Collaboration (interaction) Diagram

- Mostra interazioni tra oggetti
  - Il flusso dei messaggi è indicato da frecce accanto alle associazioni fra istanze che partecipano all'interazione
  - I messaggi sono mostrati da etichette sulle frecce ed hanno
    - Un numero sequenziale che indica l'ordine temporale con cui avvengono
    - Il metodo chiamato
    - Un valore di ritorno (opzionale)

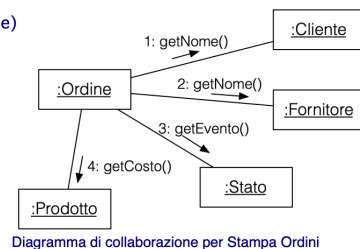
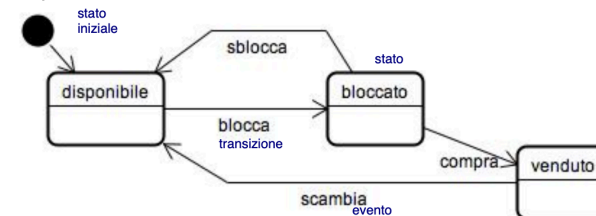


Diagramma di collaborazione per Stampa Ordini

## Diagramma UML degli Stati

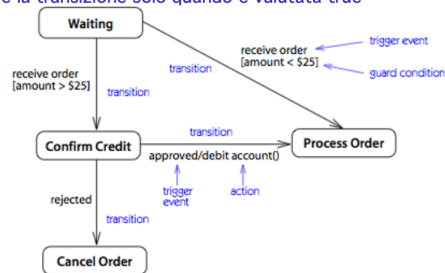
- Diagramma UML degli stati che mostra la vita di un biglietto per uno spettacolo



- Rettangoli con angoli arrotondati: stati
- Freccie: transizioni tra stati
- Cerchi pieni: stati iniziale e finale (con circonferenza)

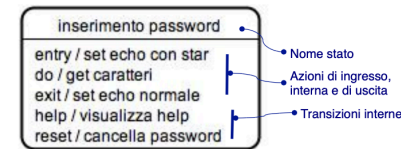
## Transizione

- Permette di lasciare uno stato in risposta ad un certo evento
- Un oggetto gestisce un solo evento alla volta
- È caratterizzata da: **event-trigger [guard] / action**
  - Evento di inizio (trigger)
  - Condizione di guardia (espressione boolean), valutata quando l'evento avviene e che fa avvenire la transizione solo quando è valutata true
  - una azione
  - uno stato target



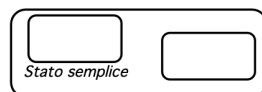
## Stato

- Descrive un intervallo di tempo durante la vita di un oggetto
- È caratterizzato da
  - Valori di oggetti, o
  - Intervallo in cui un oggetto aspetta certi eventi, o
  - Intervallo in cui un oggetto fa certe azioni

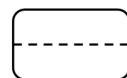


## Stati composti

- Uno stato composto è uno stato che consiste di vari sottostati sequenziali o concorrenti
  - Uno stato semplice non consiste di sottostati
- Solo uno dei sottostati sequenziali può essere attivo in un certo momento
- Lo stato esterno rappresenta la condizione di essere in uno qualsiasi degli stati interni
- Una transizione verso o da uno stato composto può invocare varie azioni di entry (dalla più esterna) o exit (dalla più interna)



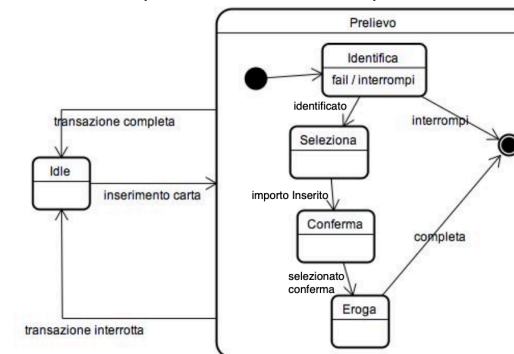
Stato composto sequenziale



Stato composto concorrente

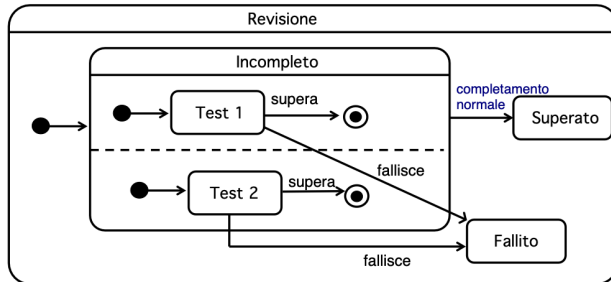
## Diagramma UML degli Stati per ATM

- Stato composto con sottostati sequenziali



## Digramma degli Stati per Revisioni

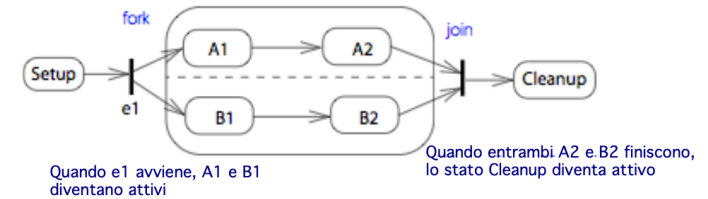
- Revisione è uno stato composto con sottostati sequenziali
  - I sottostati di Revisione sono: Incompleto, Superato e Fallito
- Incompleto è uno stato composto con sottostati concorrenti
  - I sottostati di Incompleto sono: Test 1 e Test 2



16

## Diagramma degli Stati con fork e join

- Stato composto concorrente



# Factory method & Abstract Factory



# Factory Method

- Problema: In un algoritmo, so **quando** deve essere creato un oggetto di servizio, ma non so o non voglio stabilire (e/o voglio poter cambiare) **quale** classe (implementazione) istanziare.
- Primo principio: programma usando *interfacce*: so **cosa** devo fare e **quando** farlo adesso ma non voglio ancora decidere **come**
- Creo le condizioni affinché questa decisione sia fatta altrove: ho molti modi

# FM vs Polimorfismo

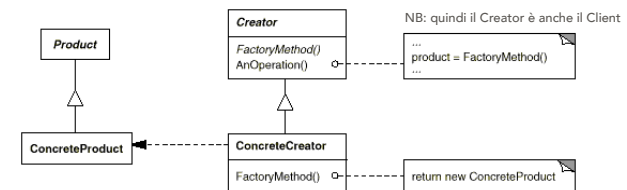
```
public class Sospeso extends BasicStud {
 public void nuovoEsame(String m, int v) {
 System.out.println("Non possibile");
 }
}

public class Studente extends BasicStud{
 private List<Esame> esami = new ArrayList<>();
 public void nuovoEsame(String m, int v) {
 Esame e = new Esame(m, v);
 esami.add(e);
 }
}

public class Client {
 BasicStud s = new Studente();
 public void registra(){
 s.nuovoEsame("Maths", 8);
 }
}
```

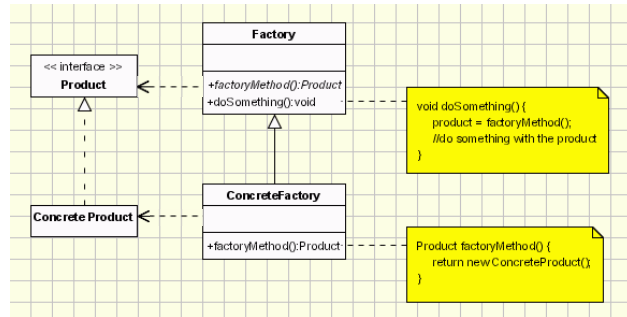
# Factory Method

- Schema del pattern:
  - **Product** è l'interfaccia comune degli oggetti creati da factoryMethod()
  - **ConcreteProduct** è un'implementazione di Product
  - **Creator** dichiara il factoryMethod(), quest'ultimo ritorna un oggetto di tipo Product. Usa l'oggetto Product nel suo codice
  - **ConcreteCreator** implementa il factoryMethod(), o ne fa override, sceglie quale ConcreteProduct istanziare e ritorna tale istanza:



NB: quindi il Creator è anche il Client

# Factory Method



```

public interface Product {
 void request();
}

abstract class Creator {
 abstract public Product getProduct();

 public void AnOperation(){
 Product p = this.getProduct();
 p.request();
 }
}

class CProduct implements Product {
 public void request() {
 System.out.println("CProduct");
 }
}

public class CCreator extends Creator {
 @Override
 public Product getProduct() {
 return new CProduct();
 }
}

public class Client { //client application
 public static void main(String[] args) {
 Creator myApp = new CCreator(); // incapsulation!
 myApp.AnOperation(); //use the product service indirectly
 ...

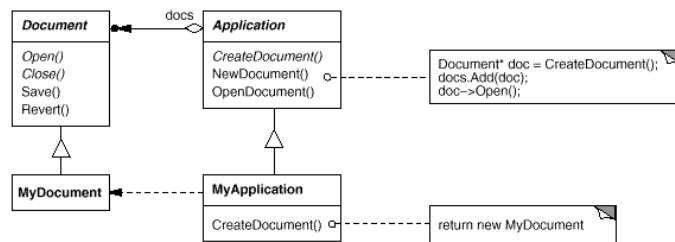
 Product myProduct = myapp.getProduct(); // incapsulation!
 myProduct.request(); // use the product service directly
 }
}

```

Sottoclassando Creator  
fornisco versioni  
alternative di  
AnOperation/request

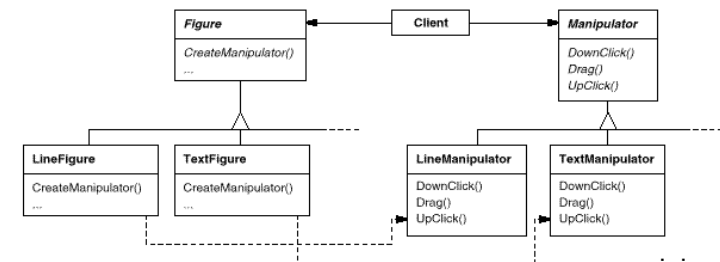
## Factory Method: esempio

- Incapsulo la creazione dentro un metodo (dà l'oggetto come valore di ritorno): saranno le sottoclassi a decidere la classe da istanziare
- Il mio algoritmo (astratto) può essere scritto conoscendo (e/o usando) solo l'interfaccia dell'oggetto di servizio: *primo principio*



## Gerarchie di classi parallele

- Il factory method può essere fornito anche a client esterni al Creator.
- Altri clients possono trovare i factory methods utili, specialmente nei casi di gerarchie di classi parallele



...ci ricorda qualcosa?

## Il Creator come servente di client esterni

factory method reso pubblicamente accessibile  
Il Creator è anche fornitore di servizio: una Factory di oggetti astratti...

```
public class Creator {
 public IStudiante getStudiante() {
 return new Studente();
 }
}

public class SoCreator extends Creator {
 public IStudiante getStudiante() {
 return new Sospeso();
 }
}
```

```
public class Client {
 public void registra(Creator sc) {
 if (sc==null) sc = new SoCreator();
 . . .
 IStudiante s = sc.getStudiante();
 s.nuovoEsame("Maths", 8);
 }
}
```

## Variante: Creator generico

- Implemento il Creator astratto e generico
- Nelle sottoclassi lo collego al prodotto concreto usando la sintassi dei generics
- il client seleziona indirettamente il prodotto da usare
- Non c'è dipendenza diretta del client col prodotto
- (Quasi) identica alla versione classica
- Vediamo esempio...

N.B.: sbagliato fare `Creator myApp = new Creator<CProduct>();`

## Variante: factory method parametrico

Più varianti di uno stesso oggetto, o oggetti diversi, dallo stesso metodo factory?

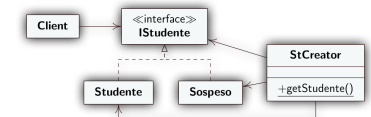
- Derivo un nuovo concrete creator con override del factory method
  - Il prodotto è scelto indirettamente, scegliendo il concrete creator
- Oppure...
- Parametrizzo il metodo factory per supportare più oggetti con un solo Creator
- No Creator subclassing: il client seleziona il prodotto direttamente

## Esempio

```
public interface IStudiante {
 public void nuovoEsame(String m, int v);
 public float getMedia();
}

public class Studente implements IStudiante {
 private List<Esame> esami = new ArrayList<>();
 public void nuovoEsame(String m, int v) {
 Esame e = new Esame(m, v);
 esami.add(e);
 }
 public float getMedia() {
 if (esami.isEmpty()) return 0;
 float sum = 0;
 for (Esame e : esami) sum += e.getVoto();
 return sum / esami.size();
 }
}

public class Sospeso implements IStudiante {
 private float media;
 public Sospeso(float m) {
 media = m;
 }
 public void nuovoEsame(String m, int v) {
 System.out.println("Non e' possibile
sostenere esami");
 }
 public float getMedia() {
 return media;
 }
}
```



```
public class StCreator {
 public static IStudiante getStudiante(boolean a) {
 if (a) return new Studente();
 return new Sospeso(0);
 }
}

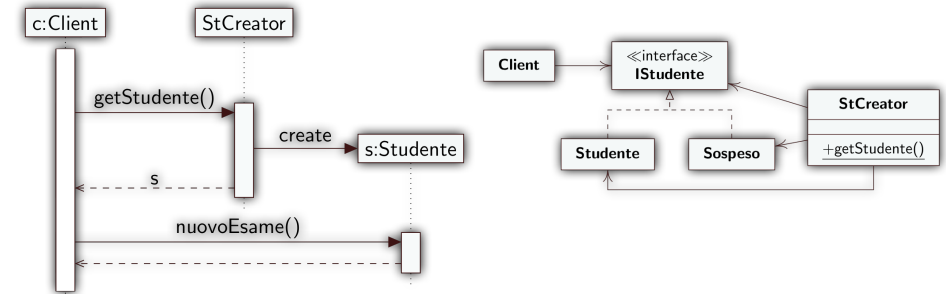
public class Client {
 public void registra() {
 IStudiante s = StCreator.getStudiante(true);
 s.nuovoEsame("Maths", 8);
 }
}
```

## Variante: static factoryMethod()

- factory method metodo di classe, globale
- Ok le scelte su cosa generare sono definitive
- Non estendibile
  - Non possibile l'override di metodi statici
- Non serve dichiarare alcun abstract creator
- Uso il fm senza dover istanziare il Concrete Creator

## Esempio

- l'interfaccia IStudente svolge il ruolo Product, le classi Studente e Sospeso svolgono il ruolo ConcreteProduct, e la classe StCreator svolge il ruolo ConcreteCreator



```
public interface IStudente {
 public void nuovoEsame(String m, int v);
 public float getMedia();
}

public class Studente implements IStudente {
 private List<Esame> esami = new ArrayList<>();
 public void nuovoEsame(String m, int v) {
 Esame e = new Esame(m, v);
 esami.add(e);
 }
 public float getMedia() {
 if (esami.isEmpty()) return 0;
 float sum = 0;
 for (Esame e : esami) sum += e.getVoto();
 return sum / esami.size();
 }
}

public class Sospeso implements IStudente {
 private float media;
 public Sospeso(float m) {
 media = m;
 }
 public void nuovoEsame(String m, int v) {
 System.out.println("Non e' possibile sostenere esami");
 }
 public float getMedia() {
 return media;
 }
}
```

```
public class StCreator {
 private static boolean a = true;

 public static IStudente getStudente() {
 if (a) return new Studente();
 return new Sospeso(0);
 }
}

public class Client {
 public void registra() {
 IStudente s = StCreator.getStudente();
 s.nuovoEsame("Maths", 8);
 }
}
```

## Variante: Creator con riflessione

- Implemento un solo Creator usando le Reflection API
- parametrico <per nome classe> nei concrete product
- Il Client specifica il prodotto "simbolicamente" o come stringa
- Non serve più derivare sottoclassi:

```
Creator myApp = new Creator(CProduct.class);
```

Vediamo codice d'esempio...

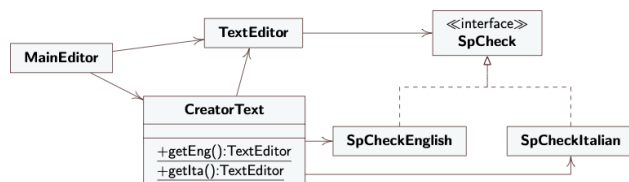
# Factory Method

- Applicabilità:
  - Una classe non è in grado di sapere in anticipo la classe dell'oggetto che deve creare
  - oppure vuole **delegare alle sottoclassi future** la scelta di quale oggetto creare e utilizzare
  - Una classe (creator/client) delega responsabilità (metodi) a una (o più) classe di supporto (product) e vuole **localizzare in un punto** (il factory method) la scelta/conoscenza di quali siano queste classi (dipendenza) Il Client conosce solo l'interfaccia Product e può funzionare con qualsiasi Concrete Product.
- Conseguenze:
  - I Concrete Product sono facilmente intercambiabili
  - Se per ogni Concrete Product va derivato di nuovo il Creator, si ha proliferazione di classi
  - Fornisce un metodo "gancio" alle classi derivate per fornire una versione estesa di un dato oggetto
  - molto più flessibile che creare un oggetto esplicitamente.

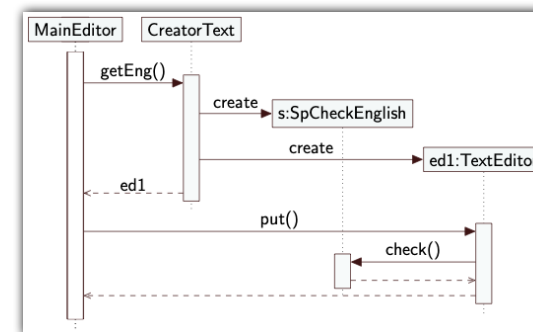
# Dependency Injection

- Consiste nell'inserire le dipendenze necessarie ad altri oggetti (istanze di ConcreteProduct) passandole come parametri di istanza : Una classe C usa un servizio fornito da S (ovvero C dipende da S)
  - Esistono varie implementazioni di S (siano esse: S1, S2), la classe C non deve dipendere dalle implementazioni S1, S2
  - In un terzo contesto, creo l'istanza di C e la configuro con l'implementazione di S con cui desidero che operi: gli passo una istanza già creata di S1 o S2. Ho legato C ad S1 o S2
    - `C myC = new C1( new S1() );`
- La dipendenza tra i due oggetti non è statica, viene "iniettata" attraverso un parametro, nel costruttore al momento della creazione
- creo una composizione di oggetti dinamica, senza dipendenze tra le loro classi
- È una tecnica utile anche per implementare il creator del pattern factory method...

## Esempio: Applicazione Editor



## Sequence diagram



Vediamo il codice di App Editor...

# Object Pool

- Un object pool è un deposito di istanze già create, una istanza sarà estratta dal pool quando una classe client ne fa richiesta
  - Il pool può crescere o può avere dimensioni fisse
    - Dimensioni fisse: se non ci sono oggetti disponibili al momento della richiesta, non ne creo di nuovi
  - Il client restituisce al pool l'istanza usata quando non più utile
- Il design pattern Factory Method può implementare un object pool
  - I client fanno richieste, come visto prima per il Factory Method
  - I client dovranno dire quando l'istanza non è più in uso, quindi riusabile
  - Lo stato dell'istanza da riusare potrebbe dover essere riscritto
  - L'object pool dovrebbe essere unico -> lo rendo un Singleton : Multiton

# Esempio di Object Pool

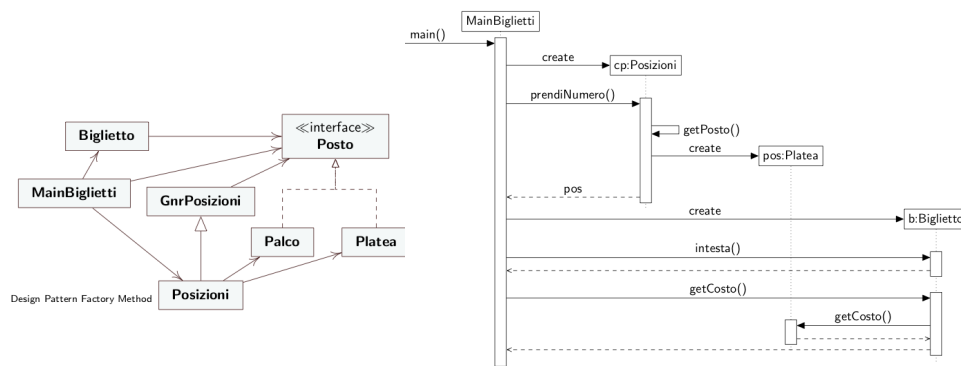
```
import java.util.LinkedList;
// CreatorPool è un ConcreteCreator e implementa un Object Pool

public class CirclePool extends ShapeCreator {
 private LinkedList<Shape> pool = new LinkedList<Shape>();

 // getShape() è un metodo factory che ritorna un oggetto prelevato dal pool
 public Shape getShape() {
 Shape s;
 if (pool.size() > 0) s = pool.removeFirst();
 else s = new Circle(); //concrete product
 return s;
 }

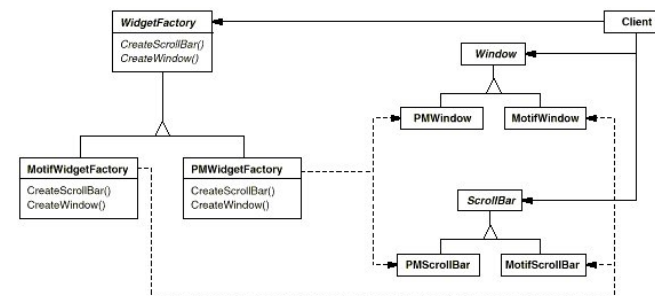
 // releaseShape() inserisce un oggetto nel pool
 public void releaseShape(Shape s) {
 pool.addLast(s);
 }
}
```

# Esempio app Biglietti: Factory Method con Object Pool



Visibile su GitHub

# Abstract Factory

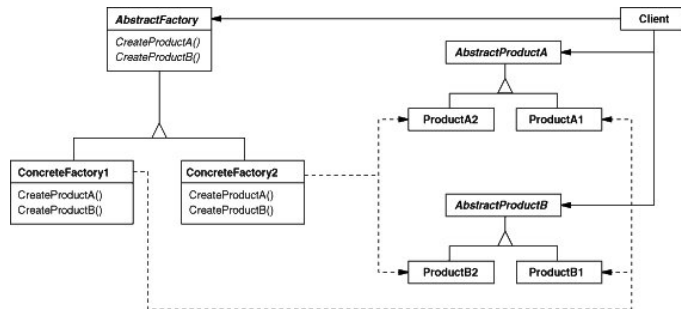


Voglio un applicazione grafica multiplatforma: Mac, PC e Unix

Devo disegnare Menu, Finestre e Pulsanti

Devono apparire nello stile della piattaforma usata

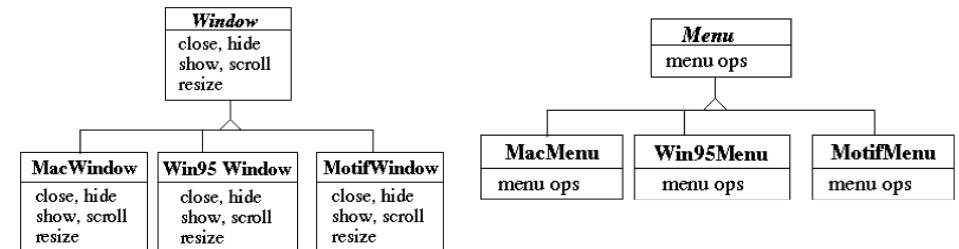
## Abstract Factory (Kit): soluzione



- Intento: Fornire una interfaccia per la creazione di **famiglie di oggetti correlati** o dipendenti senza specificare le loro classi concrete

## Esempio

- Una serie di oggetti di servizio (prodotti) in tre varianti ciascuno



- Program to an interface...Menu
- Ok, ma...

```

public void installDisneyMenu()
{
 Menu disney = Crea un menu' ...in qualche modo
 disney.addItem("Disney World");
 disney.addItem("Donald Duck");
 disney.addItem("Mickey Mouse");
 disney.addGrayBar();
 disney.addItem("Minnie Mouse");
 disney.addItem("Pluto");
 etc.
}

```

- Come creare il menù in modo che: sia della piattaforma giusta?
- e...non dovere scrivere tre volte ma una sola volta il client?
- Ridurre al minimo il numero di posti in cui rivelo la scelta del suo tipo?

## Soluzione

- Passo al client un factory object...

```

public void installDisneyMenu(WidgetFactory myFactory) {
 Menu disney = myFactory.createMenu();
 disney.addItem("Disney World");
 disney.addItem("Donald Duck");

}

```

## Varianti di implementazione

- 1) classica: derivo e faccio override dell'intero set di fm. Tali concrete factory sono dette interne...
- 2) con factory esterne (ad es., preesistenti). Uso la composizione (2° principio): invoco i fm dei creator esterni (derivo ma non faccio override);
- 3/3b) come set di factory methods overridden dalla sottoclasse, ma che ritornano Class meta-objects da istanziare (reflection API)
- 4) basata su implementazioni prototipali

## Classica: set di Factory Methods interni

```
abstract class WidgetFactory {
 public Window createWindow();
 public Menu createMenu();
 public Button createButton();
}

class MacWidgetFactory extends WidgetFactory {
 public Window createWindow() { return new MacWindow(); }
 public Menu createMenu() { return new MacMenu(); }
 public Button createButton() { return new MacButton(); }
}

class WinWidgetFactory extends WidgetFactory {
 public Window createWindow() { return new WinWindow(); }
 public Menu createMenu() { return new WinMenu(); }
 public Button createButton() { return new WinButton(); }
}
```

## Variante 2: Factory methods esterni

```
abstract class WidgetFactory {
 //Creator composti
 WindowFactory wf;
 MenuFactory mf;
 ButtonFactory bf;

 //deleghe ai Creator composti
 public Window createWindow() { return windowFactory.createFancyWindow(); } // posso invocare metodi specifici di WindowFactory
 public Menu createMenu() { return menuFactory.createMenu(); }
 public Button createButton() { return buttonFactory.createMenu(); }
}

//derivo per selezionare il delegati
class MacWidgetFactory extends WidgetFactory {
 public MacWidgetFactory() {
 wf = new MacWindowFactory();
 mf = new MacMenuFactory();
 bf = new MacButtonFactory();
 }
}

//Concrete Creator (delegato) preesistente
// anche una per ogni prodotto concreto, per ogni versione

class MacWindowFactory extends WindowFactory {
 public MacWindow() { code to build a MacWindow... }
 public Window createWindow() { blah }
 public Window createFancyWindow() { blah }
 public Window createPlainWindow() { blah }
 etc.
}
```

Vediamo esempio su GitHub...

## Variante 3: Reflection API

```
public abstract class AbsFact {
 MenuFactory menuFactory;
 WindowFactory windowFactory;
 ButtonFactory buttonFactory;

 Menu getMenu() { return menuFactory.getMenu(); }
 Window getWindow() { return windowFactory.getWindow(); }
 Button getButton() { return buttonFactory.getButton(); }
}

public class WinStyleFactory extends AbsFact {
 public WinStyleFactory() { //versione con reflection
 menuFactory = new MenuFactory(WinMenu.class); //Ho un solo Creator/Factory per Prodotto
 windowFactory = new WindowFactory(WinWindow.class);
 buttonFactory = new ButtonFactory(WinButton.class);
 }
}
```

esempio su GitHub...

## Variante 3b: Reflection API

```
public abstract class AbsFact {
 abstract Class<? extends Window> windowClass();
 abstract Class<? extends Menu> menuClass();
 abstract Class<? extends Button> buttonClass();

 // senza try/catch per semplificare...

 public Button getButton() { return (Button) buttonClass().getConstructor().newInstance(); }
 public Menu getMenu() { return (Menu) menuClass().getConstructor().newInstance(); }
 public Window getWindow() { return (Window) windowClass().getConstructor().newInstance(); }
}

//non ho alcun Creator/Factory di Prodotto

public class MacStyleFactory extends AbsFact {
 Class<? extends Window> windowClass() {return MacWindow.class;}
 Class<? extends Menu> menuClass() {return MacMenu.class;}
 Class<? extends Button> buttonClass() {return MacButton.class;}
}
```

esempio su GitHub...

## Versione con delega diretta a Prototipi

```
class WidgetFactory { //non più abstract
 Window window;
 Menu menu;
 Button button;

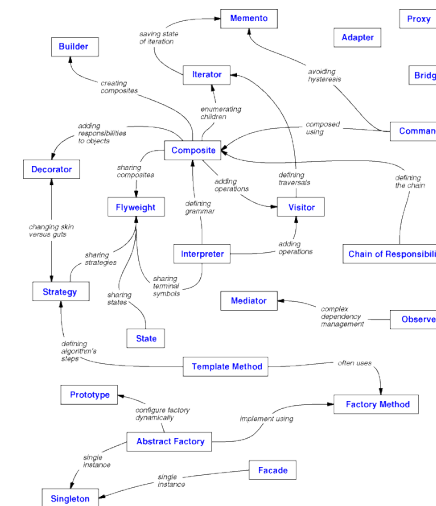
 public WidgetFactory(Window windowPrototype, Menu menuPrototype, Button buttonPrototype){
 this.window= windowPrototype;
 this.menu= menuPrototype;
 this.button= buttonPrototype;
 }

 public Window createWindow() { return window.clone() }
 public Menu createMenu() { return menu.clone() }
 public Button createButton() { return button.clone() }
}
```

Non serve sottoclassare ma inizializzare con le istanze prototipali dei prodotti

## Abstract Factory: applicabilità

- Voglio indipendenza dal tipo concreto di prodotti che creo e uso
- Possibilità di configurare il sistema con una tra varie famiglie di prodotti
- famiglie di prodotti correlati sono state progettate per essere usati insieme e **si vuole imporre** questo vincolo di coerenza
- fornire librerie di classi intercambiabili rivelando solo le interfacce (API)
- **Difficile aggiungere in seguito nuovi prodotti** (vincolati al set della AF)



# Prototype

Pattern "creazionale"



## Prototype

- Un oggetto a run-time funge da classe, affinché nulla dipenda da quella classe
- Non alloco nuove istanze dalla classe ma duplicando una istanza originale

Clono una istanza base (il prototipo) pre-esistente

avrò : `Product obj = prototype->clone();`

La clonazione è una operazione di duplicazione di un oggetto

La creazione dei prototipi è una operazione una-tantum, fatta altrove

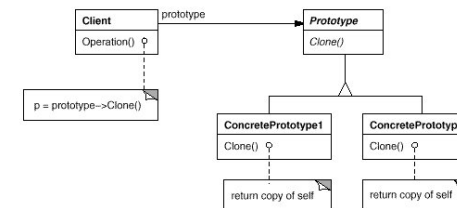
Il clone viene editato dal client per adattarlo al bisogno specifico

## Applicabilità

- Quando un sistema dovrebbe essere indipendente dal prodotto concreto che istanzia, e in più...
  - la classe concreta da usare **è nota solo a run-time:**
    - impossibile scrivere a priori metodi factory per incapsularne la creazione
- oppure, la classe **ha solo pochi stati possibili**, quindi ho poche versioni di istanze tutte dello stesso ADT:
  - conviene avere un prototipo da clonare per ogni versione dello stato invece di classi da istanziare e poi inizializzare.
- oppure, se la creazione di nuove istanze di classe **è costosa**
  - es.: il costruttore fa varie query a DB. Copiare una istanza già pronta è più performante

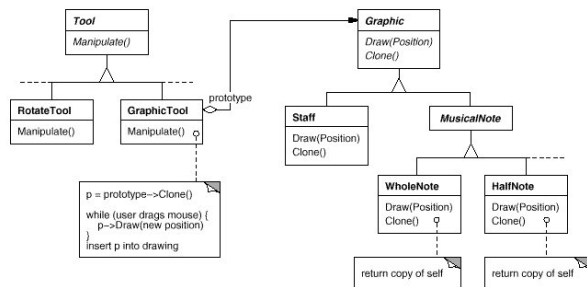
## Prototype

- Coding di un client, ma il design dell'oggetto di servizio è ancora incompleto
  - un set di funzionalità utili al client è fornito da oggetti con implementazioni prototipali
  - Il Client ne decide la creazione, ignorandone il tipo attuale



- Prototipo: un ADT non ancora definitivo, né di nome né di fatto (corpo)
- Il factory method non è applicabile: va applicato a design time del client
  - aggiungere nuovi tipi, va bene. Ma, una volta derivati i relativi Concrete Creator, non sono previste loro "modifiche"

## Classi con poche istanze distinte



WholeNote, HalfNote, etc...  
cambiano solo nel glifo da disegnare

Mi bastano solo due istanze  
ciascuno (prototipi) da clonare a  
volontà

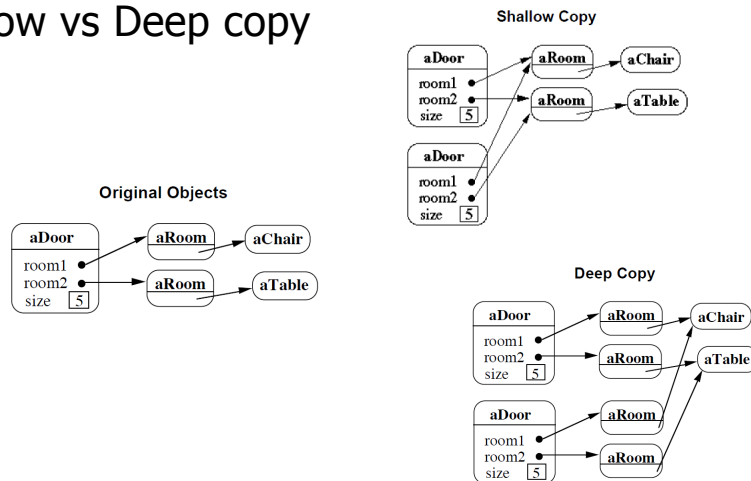
Inoltre, potrei avere prototipi non stateless, ma configurabili solo in pochi modi distinti:

- Istanze distinte dello stesso prototipo in alternativa all'istanziare classi distinte
- Si riduce il numero di classi nel sistema

## Prototipi in Java

- public interface **Cloneable** (dal JDK 1.0)
  - È vuota
  - Se la implemento, eredito il metodo `Object.clone()` :
    - esegue una shallow copy (copia valore per valore tutte le variabili di istanza)
    - altrimenti, **CloneNotSupportedException**
  - fare override di `Clone()` per implementare una deep copy

## Shallow vs Deep copy



## Conseguenze

Pro:

- posso sostituire i prototipi facilmente a run-time senza modifiche nel client
- tengono celata la struttura (eventualmente) complessa al loro interno
  - Es. Una composizione eterogenea di oggetti che non ha ancora ADT

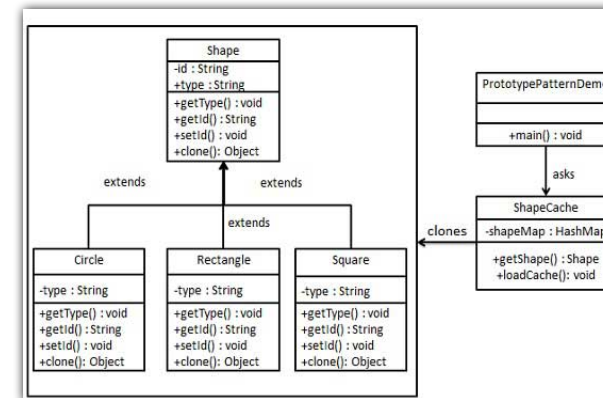
Contro:

- devo implementare il metodo `Clone()`
- devo inizializzarli, se necessario
- a volte comodo un prototype manager

# Prototype Manager

- In alcuni casi il numero di prototipi non è noto o fisso
- Uso un Catalogo (o registro) di prototipi clonabili
- E' una memoria associativa dove posso registrare (o cancellare) nuovi prototipi sotto un etichetta o un codice simbolico
- Restituisce il riferimento ad un prototipo associato ad una data chiave
- `prototypeManager.get("ProtoXY123").clone()`
- La chiave potrebbe essere il nome della classe cui appartiene

## Esempio Prototype Manager



VS CODE SHAPE PROTOTYPE

## Abstract Factory con Prototipi

```
class ProtoFactory{ //non più abstract
 WindowPrototype window;
 MenuPrototype menu;
 ButtonPrototype button;

 public ProtoFactory(WindowPrototype wp, MenuPrototype mp, ButtonPrototype bp){
 this.window = wp;
 this.menu = mp;
 this.button = bp;
 }

 public Window createWindow() { return (Window) window.clone(); }
 public Menu createMenu() { return (Menu) menu.clone(); }
 public Button createButton() { return (Button) button.clone(); }
}
```

## Adapter

Pattern "strutturale"

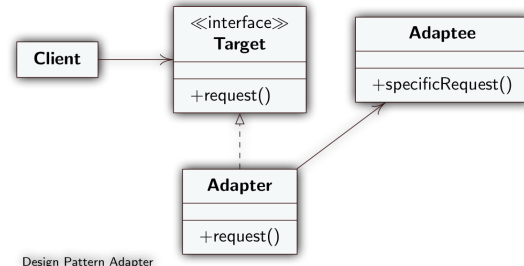


# Adapter

**Intento:** Convertire l'interfaccia di una classe in un'altra interfaccia che i client si aspettano. Adapter permette ad alcune classi di interagire, eliminando il problema di interfacce incompatibili

## Soluzione:

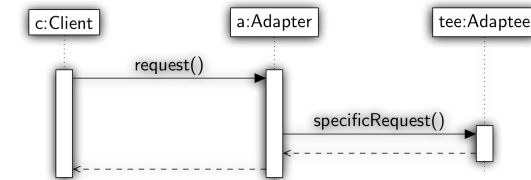
- **Target** è l'interfaccia che il chiamante si aspetta
- **Client** usa oggetti che sono conformi all'interfaccia Target
- **Adaptee** è l'oggetto di libreria che necessita l'adattamento
- **Adapter** converte, ovvero adatta, la chiamata che fa una classe client all'interfaccia della classe di libreria.



Design Pattern Adapter

# Adapter

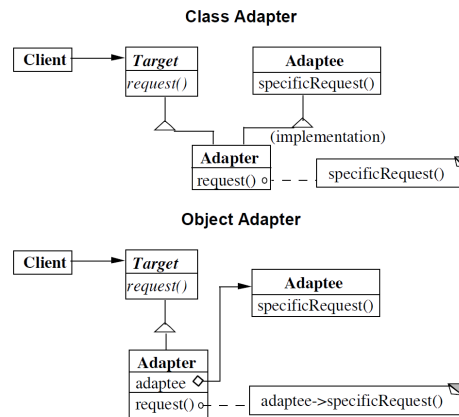
## Sequence diagram



- Non è possibile cambiare l'interfaccia dell'Adaptee, poiché non si ha il sorgente (comunque non conviene cambiarla)
- Non è possibile cambiare l'applicazione (il Client), e si può voler cambiare il metodo chiamato, senza renderlo noto al client

# Object vs Class Adapter

- Il **class adapter** consente ridefinire (override) parte dei metodi dell'Adaptee
- Quando vuoi riusare molte classi esistenti è poco pratico sottoclassarle tutte
- Un **object adapter** può adeguare l'interfaccia di più classi a quella del client senza sottoclassare
- Rende problematico ridefinire il comportamento degli Adaptee perchè...
- L'object adapter non sa esattamente con quale sottoclasse specifica di Adaptee sta lavorando



# Object Adapter example

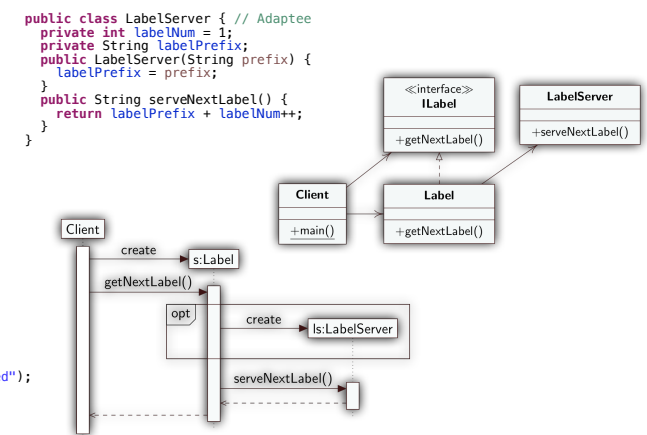
```

public interface ILabel { // Target
 public String getNextLabel();
}

// Adapter
public class LabelServer implements ILabel {
 private LabelServer ls;
 private String p;
 public LabelServer(String prefix) {
 p = prefix;
 }
 public String getNextLabel() {
 // lazy initialisation
 if (null == ls)
 ls = new LabelServer(p);
 return ls.serveNextLabel();
 }
}

public class Client {
 public static void main(String args[]) {
 ILabel s = new Label("LAB");
 String l = s.getNextLabel();
 if (l.equals("LAB1"))
 System.out.println("Test 1:Passed");
 else
 System.out.println("Test1:Failed");
 }
}

```



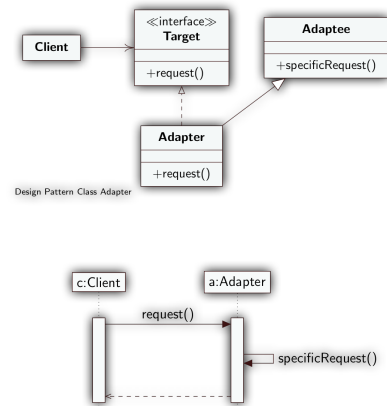
## Versione Class Adapter

- La classe con il ruolo Adapter è sottoclasse di Adaptee

```
public class Label extends LabelServer
 implements ILabel { // Adapter

 public Label(String prefix) {
 super(prefix);
 }

 public String getNextLabel() {
 return serveNextLabel();
 }
}
```



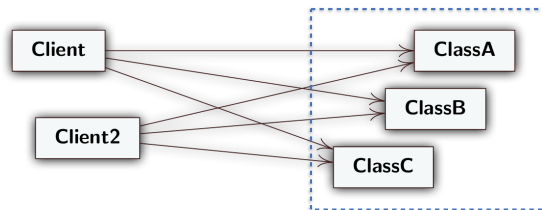
## Facade

Pattern "strutturale"



## Design pattern Facade

- **Intento:** Fornire un'interfaccia unificata al posto di un insieme di interfacce in un sottosistema (consistente di un insieme di classi). Definire un'interfaccia di alto livello (semplificata) che rende il sottosistema più facile da usare

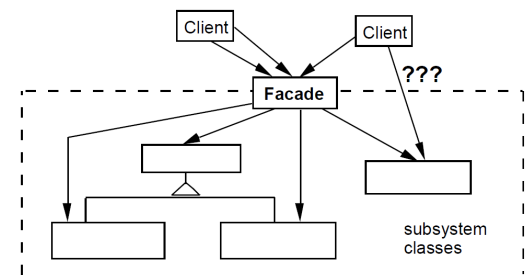


## Design pattern Facade

- Soluzione

- **Facade** fornisce un'unica interfaccia semplificata ai client e nasconde gli oggetti del sottosistema, questo riduce la complessità dell'interfaccia e quindi delle chiamate. **Facade** invoca i metodi degli oggetti che nasconde

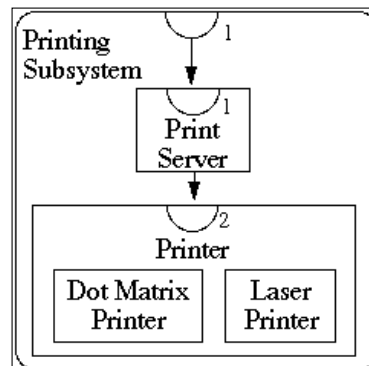
- E' una composizione di una o più istanze di ogni classe del sottosistema usate dai client



## FACADE

### Si applica ad un Sottosistema:

Gruppi di classi, ed eventualmente altri sottosistemi, che collaborano tra loro per supportare la realizzazione di un set di compiti specifici



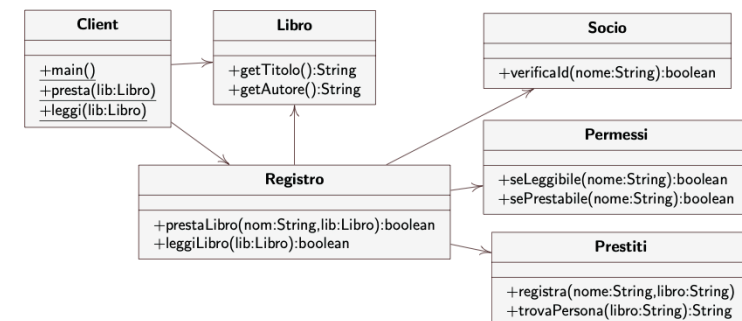
## Conseguenze

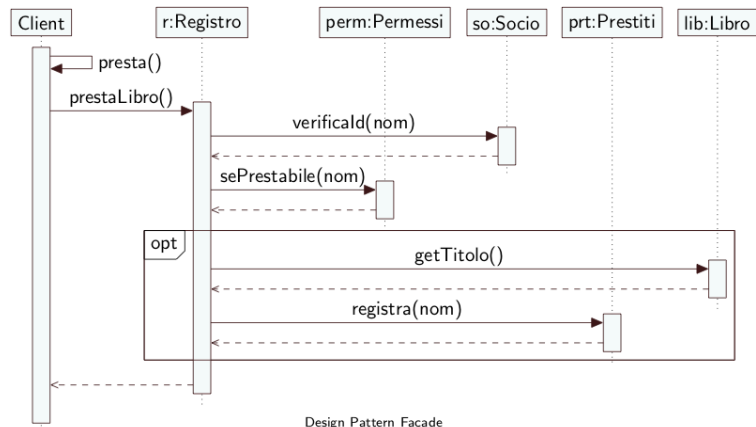
- Nasconde ai client l'implementazione del sottosistema
- Riduce l'accoppiamento tra sottosistemi
  - Riduce le dipendenze di compilazione in sistemi grandi.
  - Se si cambia una classe del sottosistema, si può ricompilare la parte di sottosistema fino al facade, quindi non i vari client
- Se era direttamente il client a istanziare gli oggetti, ora ci deve pensare il facade
- Non previene l'accesso diretto, quando occorre, agli oggetti del sottosistema

## Implementazione

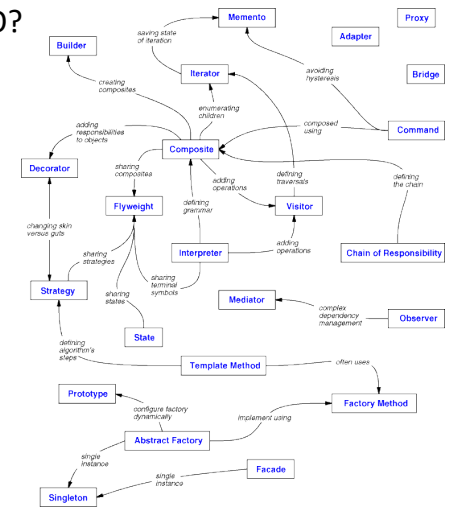
- Non deve aggiungere funzionalità non presenti già nel sottosistema
  - può rielaborare e comporre funzionalità esistenti, solo per semplificare l'uso e ridurre il numero di metodi dell'interfaccia rispetto al totale delle singole classi
- Per rendere gli oggetti del sottosistema non accessibili all'esterno:
  - le classi possono essere annidate dentro il Façade (nested/inner class)
  - posso usare modificatori di accesso che limitino al package
  - In Java: Default class Access Modifier per le classi del sottosistema
  - Solo per Façade public class e/o membri

## Esempio: Libreria





## A CHE PUNTO SIAMO?



# Template method

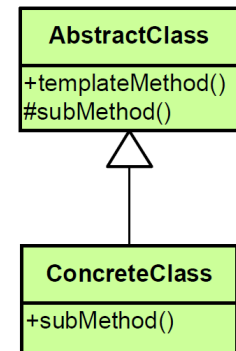
Comportamentale



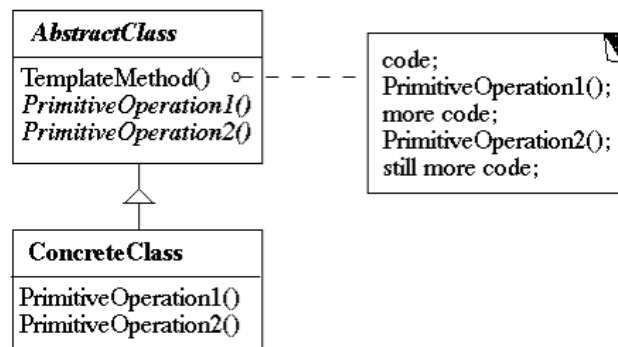
# Template method

## • Intento:

- Consente di definire in modo definitivo l'implementazione di un metodo, tralasciando il dettaglio specifico di uno o più passi interni
- Delega alle sottoclassi l'implementazione specifica di tali passi rimasti astratti, ottenendo più varianti di implementazione dello stesso algoritmo

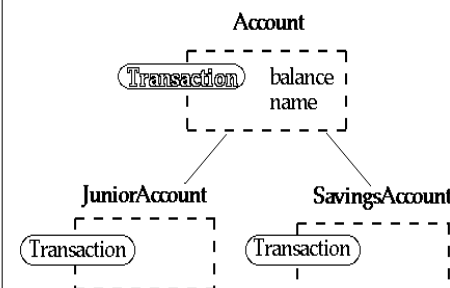


## Struttura



NB Si inverte la struttura di controllo: è la classe padre che esegue metodi di una sottoclasse

## Metodi «differiti»



```

abstract class Account {
 public abstract void Transaction();
}

```

```

class JuniorAccount extends Account {
 public void Transaction() { //put code here }
}

```

```

class SavingsAccount extends Account {
 public void Transaction() { //put code here }
}

```

Quando faccio uso del polimorfismo l'intera implementazione del metodo viene sostituita ("override"), non solo in parte

## Template method

```
class Account {
 public void TransactionSubpartA();
 public void TransactionSubpartB();
 public void TransactionSubpartC();

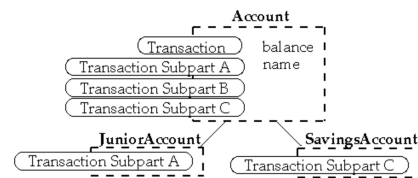
 Public void Transaction(float amount) {
 TransactionSubpartA();
 TransactionSubpartB();
 TransactionSubpartC();
 // EvenMoreCode;
 }
}
```

...nel client....

```
Account customer;
customer = createNewAccount();
customer->Transaction(amount);
```

```
class JuniorAccount extends Account {
 public void TransactionSubpartA() { //
code};
}
```

```
class SavingsAccount extends Account {
 public void TransactionSubpartC(){//
code};
}
```



## In Java (>8) : metodi default

- A volte i metodi da fissare sono funzionalità non specifiche di un ADT
- posso usare i default methods delle interfacce java
- Creo classi specifiche che ne completano in qualche modo l'implementazione

```
public interface algorithmTemplate {

 void partA();
 void partB();
 void partC();

 default void algorithm(){
 partA();
 System.out.print("invariable part");
 partB();
 partC();
 }
}
```

```
class MyWayAlgorithm implements algTemplate {

 //mandatory
 public void partA(){
 System.out.print("my partA\n");
 }

 @Override
 public void partB() {
 System.out.print("my partB\n");
 }

 @Override
 public void partC() {
 System.out.print("my partC\n");
 }

 public static void main (String[] args){
 new PartA().algorithm();
 }
}
```

## Esempio di utilizzo in Java: paint() in AWT

**public void paint(Graphics g)** definito in *java.awt.Component*

Il metodo Java paint è una operazione primitiva che devo implementare ma viene chiamata da un metodo della sua classe padre, mai direttamente da me (*callback function*)

```
class HelloApplication extends Frame {
 public void paint(Graphics display){
 int startX = 30;
 int startY = 40;
 display.drawString("Hello World", startX, startY);
 }
}
```

Si inverte la struttura di controllo: È la classe padre che si ritrova ad eseguire metodi della sottoclasse

## paint() in swing

- javax.swing.JComponent implementa il metodo paint dividendolo in tre metodi separati, invocati nell'ordine seguente:
  - protected void paintComponent(Graphics g)
  - protected void paintBorder(Graphics g)
  - protected void paintChildren(Graphics g)

– Metto il mio codice in paintComponent() e lo invoca paint():

```
public void paintComponent(Graphics g) {
 g.drawString("This is my custom Panel!", 10, 20);
 redSquare.paintSquare(g);
}
```

## Refactoring per introdurre template method

- 1) scrivo tutto il codice in un unico grande metodo che diverrà il template
- 2) Lo divido in passi successivi usando ad es. i commenti
- 3) Incapsulo ogni passo in un metodo separato
- 4) Riscrivo il template invocando i metodi estratti
- Ripeto da 1) su ognuno dei metodi estratti, finché:
  - Tutti i metodi primitivi (ridefinibili) **hanno la stessa granularità** (dichiararli **protected**)
  - Tutte le parti costanti sono incapsulate nei rispettivi metodi (dichiararli **private**)
- dichiaro il template method come **final** affinché sia inalterabile dalle sottoclassi

## Caso particolare: sostituzione di valori costanti

- Applicazione molto comune del template
- Non hanno decisioni: tornano sempre lo stesso valore
- Supponiamo di avere una *lazy initialization*:

```
public class Foo {
 private Bar field;
 public final Bar getField() {
 if (field == null) field = new Bar(10);
 return field;
 }
}
```
- Potrebbe anche essere un metodo costruttore
- Come cambiare il valore (oggetto) di default a cui è inizializzato il campo field, senza esporlo (a qualsiasi valore) come parametro?

## Sostituzione di valori costanti

- Applico il template pattern al metodo:

```
public final Bar getField({
 if (field == null) field = defaultField();
 return field;
})

protected Bar defaultField() {
 return new Bar(10);
}
```

```
public clientCode() {
 field := getField(); // Il cambio risulta trasparente al codice dei client
}
```
- Ora le sottoclassi possono cambiare il valore/oggetto restituito
- L'alternativa era passare parametri nell'invocazione, dando la scelta (a caso?) al client

## Applicabilità

- Applica questo pattern....
  - Per implementare le parti invarianti di una algoritmo una sola volta per tutte:
    - Le sottoclassi implementano le parti variabili
  - Per fattorizzare in una sola classe dei comportamenti comuni a varie sottoclassi ed eliminare la loro duplicazione
  - Per avere controllo di cosa sia modificabile in una classe attraverso la derivazione:
    - Passi dei metodi template uniche parti estendibili dalle sottoclassi

# Strategy

Comportamentale



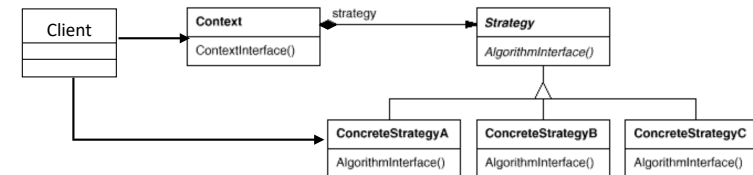
## STRATEGY

Intento:

- definire una famiglia di algoritmi correlati, incapsularli singolarmente e renderli intercambiabili...
- e anche consentire la futura modifica di un algoritmo (strategia) indipendentemente dai contesti che lo usano

ContextInterface(){ ...; strategy.AlgorithmInterface(); ...}

Il contesto non sa quale strategia verrà applicata, **la scelta è del client**



## Conseguenze

- Avrò una famiglia (espandibile) di varianti di un algoritmo a disposizione
- Situazione alternativa al sottoclassare: uso la composizione
- I client devono essere a conoscenza delle strategie disponibili: ulteriore dipendenza
- Elimino istruzioni condizionali multiple sostituendole con **strategy.do()**
  - Invece che:

```
switch (flag) {
 case A: doA(); break;
 case B: doB(); break;
 case C: doC(); break;
}
```

 **strategy.do()**
- Contesto e strategia sono oggetti distinti composti, invece che uno solo: c'è un overhead per questa indicazione.
- Notare come nei fatti uso la delega su un metodo di un oggetto di servizio, con uno scopo diverso ma similmente che in altri pattern già studiati

## Applicabilità

- Uso lo strategy pattern quando...
  - Mi servono versioni diverse di uno stesso algoritmo, anche future
  - Una classe definisce i suoi comportamenti attraverso metodi strutturati basati su switch multipli
  - Ho molte classi correlate che differiscono solo nel modo in cui attuano il loro scopo
  - Un algoritmo deve manipolare dati, in modalità scelte dai suoi client, senza che questi possano accedervi
- Come limitare l'accesso alla classe strategy? uso di *inner class*

## Inner class

- Classi definite come membri di altre classi

```
class MyOuter {
 class MyInner {
 ...
 }
 ...
}
```

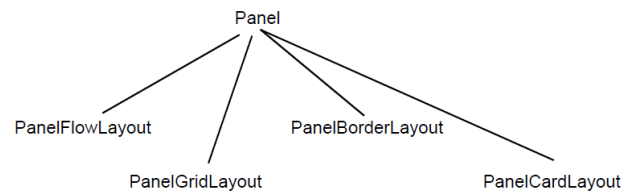
- In quanto membri di una classe, possono essere rese private (unico caso)
- hanno privilegio di accesso agli altri membri privati della classe outer
- Sono istanziabili solo nel contesto della classe outer
- Avete un esempio di inner class da studiare [su GitHub](#)

## Esempio: allineamento variabile degli oggetti grafici (Java Layout Managers)

```
import java.awt.*;
class FlowExample extends Frame {
 public FlowExample(int width, int height) {
 setTitle("Flow Example");
 setSize(width, height);
 setLayout(new FlowLayout(FlowLayout.LEFT));
 for (int label = 1; label < 10; label++)
 add(new Button(String.valueOf(label)));
 show();
 }

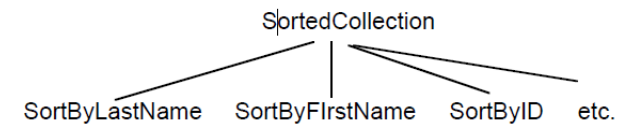
 public static void main(String args[]) {
 new FlowExample(175, 100);
 }
}
```

## Perché non derivare semplicemente sottoclassi?



- Ci sono circa 20 diversi layout
- Circa 40 sottoclassi di Component che possono usarle
- Dovrei derivare circa 800 classi!

## Altro esempio: algoritmi di ordinamento



- Si vuole supportare l'ordinamento delle collezioni
- Si può voler ordinare in qualsiasi modo arbitrariamente e con vari algoritmi
  - Avremmo una sottoclasse per ogni ordinamento di ogni tipo di Collection
  - Con i comparatori non derivo e posso combinare e variare a piacere i criteri con l'algoritmo di ordinamento

## Java: Comparable interface

```
public class Player implements Comparable<Player> {
 private int ranking;
 private String name;
 private int age;
 ...
 @Override public int compareTo(Player otherPlayer) {
 return Integer.compare(getRanking(), otherPlayer.getRanking());
 }
 ...
}
```

## Java Comparator vs Comparable

- L'interfaccia **Comparator** definisce un metodo **compare(arg1, arg2)** con due argomenti che rappresentano gli oggetti confrontati e funziona in modo simile alla **Comparable.compareTo()**.

```
public class PlayerRankingComparator implements Comparator<Player>{
 @Override
 public int compare(Player firstPlayer, Player secondPlayer){
 return Integer.compare(firstPlayer.getRanking(), secondPlayer.getRanking());
 }
}
```

- Comparable deve implementarla un oggetto del tipo Player
- Comparator è una interfaccia funzionale, può implementarla un oggetto o un metodo a parte,
  - Posso definire tutti i criteri di comparazione possibili, oltre quello *naturale (interno)*, per Player.

## Lambda comparators

- posso definire **Comparators** usando **Lambda expressions**

```
Comparator byRanking =
 (Player player1, Player player2) ->
 Integer.compare(player1.getRanking(), player2.getRanking());
```

Espressioni Lambda: funzioni *anonime* (senza nome)

- Creo a run time oggetti senza classe, contenenti un solo metodo anonimo
- Posso usare questi oggetti come tutti gli altri, come fossero "normali" oggetti,
- assegnarli ad attributi, variabili, parametri etc...

indico simbolicamente gli argomenti, e dichiaro un'espressione (o un blocco di codice) che gli abbina un risultato

Ispirate al Lambda calcolo di Church: Nel 1936, il matematico Alonzo Church, nell'ambito della sua ricerca sulle fondamenta della matematica, sviluppò il lambda calcolo come teoria per lo studio del calcolo simbolico.

## Comparator.Comparing(mr)

- metodo **statico di Comparator**
- prende come parametro una lambda expr. oppure un riferimento ad un metodo
- questo deve restituire la proprietà da usare per il confronto
- ritorna una istanza di un oggetto che implementa il corrispondente **Comparator** :

```
Comparator<Player> byRank = Comparator.comparing(Player::getRanking);
```

```
Comparator<Player> byAge = Comparator.comparing(Player::getAge);
```

Vediamo esempio completo su vscode

## Varianti di Implementazione

- Definisco come strategia e contesto interagiscono:
  - Il contesto può passare i dati alla strategia, come parametro
  - oppure, la strategia ha accesso privilegiato al contesto (inner class)
- implemento le strategie da passare (come parametri) ai contesti, come:
  - istanze oggetti, anche anonimi, (dependency injection), se voglio cambiarle a run-time
  - usando i generics, se sono in grado di indicarla a compile-time
    - es.: `SortedList<ShellSort> studentRecords`
  - funzioni anonime (lambda) che implementano *Comparator* (per uso nelle collection). I Java Streams sono una applicazione più generale del pattern.

## Esempio: strategia come istanza anonima

```
interface Strategy {
 String appliedStrategy(int x);
}

public class Context{
 // dati privati da non rivelare se non alla strategia
 private int data = 10;

 public void executeTask(Strategy s) { // strategia ricevuta come parametro
 System.out.println(s.appliedStrategy(data) + ", has been used to execute requested task");
 }

 public static void main(String args[]) {
 Context c = new Context();

 // passo come inner class una istanza anonima sottoclasse di Strategy
 c.executeTask(new Strategy() {
 public String appliedStrategy(int x) {
 return x+" squared";
 }
 });
 }
}
```

Una istanza anonima del tipo Strategy  
Viene passata al Context  
Riceve i dati dal Context come parametro

## Template vs Strategy

- Entrambi pattern comportamentali: mirano all'alterazione del comportamento
- Con il **Template** definisco ad alto livello un algoritmo, lasciando uno o più suoi passi aperti a varie possibili implementazioni (con l'override)
  - Creo varie sottoclassi (ADT), coesistenti: una "famiglia" di varianti dell'algoritmo base (**class scope**)
  - La variante è un ADT indicato nel contesto d'uso a compile-time: devo essere in fase di design per usarlo
  - la modifica del comportamento non compromette la funzionalità di alto livello
- Con lo **Strategy** creo una famiglia di **oggetti** intercambiabili che implementano varianti interne di un algoritmo (**object scope**)
  - Le applico ad un contesto (un algoritmo di giù alto livello), alterandone l'esecuzione ma non lo scopo, ovviamente
  - La variante da usare posso indicarla a run-time: è applicabile anche a contesti già compilati
- In entrambi i casi le varianti sono equivalenti per i client: **sono tutte le varianti di una stessa funzionalità**
- In entrambi i casi il contesto usa **una sola variante per volta**, ed è il client che la sceglie esplicitamente.

## Esercitazione a casa

- Applicare il pattern all'implementazione di una collection di tipo albero binario di ricerca, **generico nel tipo di dato interno**.
- Si implementi un metodo di **stampa** del suo contenuto in un **ordine sconosciuto** dato da una valutazione esterna (strategia) degli attributi del dato.

# State pattern

comportamentale

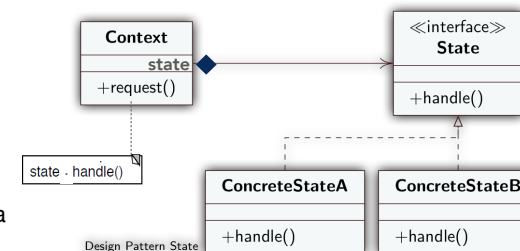


## Design pattern State

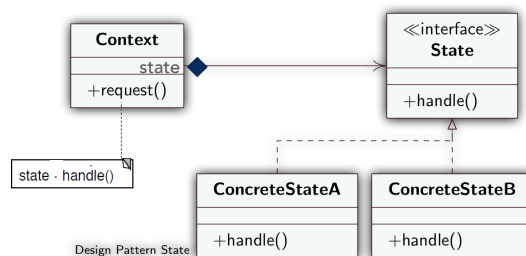
- **Intento:** Permettere ad un oggetto di alterare il suo comportamento quando il suo stato interno cambia. Sembrerà che l'oggetto abbia cambiato classe.

**Applicabilità:** request() è un algoritmo complesso con molte varianti condizionali, legate allo stato del contesto...

**Soluzione:** Inserire ogni ramo condizionale in una classe separata



## Design pattern State

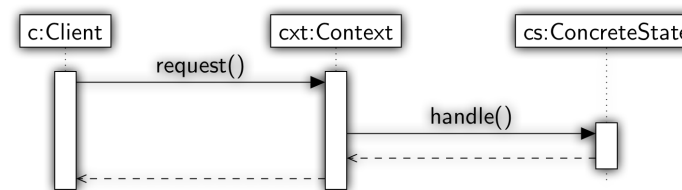


**Context** definisce l'interfaccia che interessa ai client, e mantiene un'istanza **state** di una classe **ConcreteState** che definisce lo stato corrente

L'interfaccia **State** definisce il nome del servizio (o servizi) astratto la cui implementazione deve dipendere dallo stato del Context

I **ConcreteState** sono sottoclassi di State e implementano ciascuna il diverso comportamento da associare ad un diverso stato del Context

## Design Pattern State



Collaborazioni:

Il Context è l'interfaccia per le classi client  
rimbalza le invocazioni all'oggetto ConcreteState corrente

**può passare se stesso come argomento**  
all'oggetto ConcreteState può servire **accedere al contesto...**

Il Context o i ConcreteState decidono **quale stato è il successivo** ed in quali circostanze

## State vs Strategy

• Anche state ha **object scope**, ma cambiano **frequenza** e **visibilità** dei cambi

Come distinguere strategy da state?

Dalla **frequenza** dei cambi:

Nello strategy, applico al contesto una sola strategia (tra varie possibili) per ogni suo ciclo di vita.

Nello state, il contesto cambia più volte lo stato concreto nell'arco dello stesso ciclo di vita.

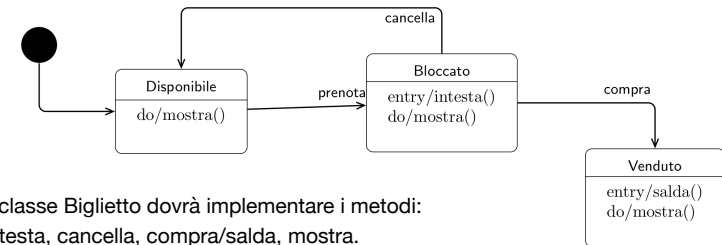
Dalla **visibilità** dei cambi:

Nello strategy, tutte le strategie fanno la stessa cosa, ma in modo diverso: per i client non fa alcuna differenza visibile se cambia il risultato della funzionalità invocata sul contesto

Nello state, gli stati concreti producono risultati visibilmente differenti nel client, per cui i client vedono il contesto reagire in modo cangiante

## Esempio: Biglietteria

Stati di Biglietto: disponibile, bloccato (ovvero prenotato), venduto



Quindi, la classe Biglietto dovrà implementare i metodi: prenota/intesta, cancella, compra/salda, mostra.

Ciascuna operazione controllerà lo stato in cui si trova il biglietto prima di eseguire le azioni necessarie

```
public class Client {
 private Biglietto b = new Biglietto();
 public static void main(String[] args) {
 usaBiglietto();
 nonUsaOk();
 }

 private static void usaBiglietto() {
 b.prenota("Ciccio Falsaperla");
 b.mostra();
 b.compra();
 b.mostra();
 }

 private static void nonUsaOk() {
 b.compra();
 b.cancella();
 b.prenota("Mario Biondi");
 }
}
```

Output dell'esecuzione di MainBiglietto

```
Prezzo: 100 codice: XYZ
Cambia stato da Disponibile a Bloccato
Inserito nuovo intestatario
Prezzo: 100 codice: XYZ
Nome: Mario Tokoro
Cambia stato da Bloccato a Venduto
Pagamento effettuato
Prezzo: 100 codice: XYZ
Nome: Mario Tokoro
Il biglietto era già stato venduto
Non può cambiare stato da Venduto a Disponibile
Non può cambiare il nome nello stato Venduto
```

```
// Codice che implementa i suddetti requisiti (prima versione)
public class Biglietto {
 private String codice = "XYZ", nome;
 private int prezzo = 100;
 private enum StatoBiglietto { DISP, BLOC, VEND }
 private StatoBiglietto stato = StatoBiglietto.DISP;

 // ogni operazione deve controllare in che stato si trova il biglietto
 public void prenota(String s) {
 switch (stato) {
 case DISP:
 System.out.println("Cambia stato da Disponibile a Bloccato");
 nome = s;
 System.out.println("Inserito nuovo intestatario");
 stato = StatoBiglietto.BLOC;
 break;
 case BLOC:
 nome = s;
 System.out.println("Inserito nuovo intestatario");
 break;
 case VEND:
 System.out.println("Non può cambiare il nome nello stato Venduto");
 break;
 }
 }
}
```

```

public void cancella() {
 switch (stato) {
 case DISP:
 System.out.println("Lo stato era gia' Disponibile");
 break;
 case BLOC:
 System.out.println("Cambia stato da Bloccato a Disponibile");
 nome = "";
 stato = StatoBiglietto.DISP;
 break;
 case VEND:
 System.out.println("Non puo' cambiare stato da Venduto a Disponibile");
 break;
 }
}

public void mostra() {
 System.out.println("Prezzo: " + prezzo + " codice: " + codice);
 if (stato == StatoBiglietto.BLOC || stato == StatoBiglietto.VEND)
 System.out.println("Nome: " + nome);
}

```

```

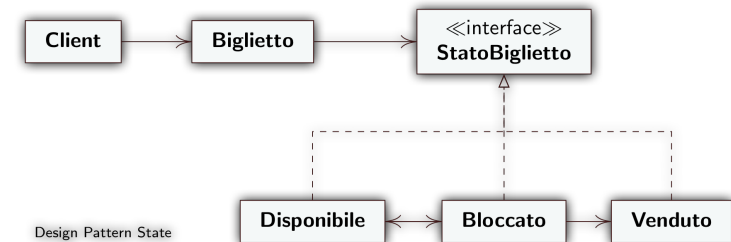
public void compra() {
 switch (stato) {
 case DISP:
 System.out.println("Non si puo' pagare, bisogna prima intestarlo");
 break;
 case BLOC:
 System.out.println("Cambia stato da Bloccato a Venduto");
 stato = StatoBiglietto.VEND;
 System.out.println("Pagamento effettuato");
 break;
 case VEND:
 System.out.println("Il biglietto era gia' stato venduto");
 break;
 }
}

```

## Analisi del codice

- La classe ha circa 70LOC, metodo più lungo 15LOC, solo 32 linee con ";"
- Ogni metodo ha vari rami condizionali, uno per ogni stato. La logica condizionale rende il codice difficile da modificare
- Il comportamento in ciascuno stato non è ben separato, poiché lo stesso metodo implementa più comportamenti
- Si può arrivare a un design e un codice più semplice, e che separa i comportamenti? Sì, tramite indiretteeze
- Una tecnica di refactoring in genere applicabile è **Replace Conditional with Polymorphism**,
  - Le condizioni possono essere trasformate in messaggi, questo riduce i duplicati, aggiunge chiarezza e aumenta la flessibilità del codice
- Ma qui i cambiamenti sono legati allo stato: uso il pattern State, ovvero la tecnica di refactoring **Replace Type Code with State**

## Nuovo design, con il pattern State



Ci sono **dipendenze** tra gli stati perché le **transizioni** di stato sono state cablate al loro interno, non nel contesto (biglietto)

Ciascuno stato avvia, quando occorre, una transizione, questo ha permesso di **eliminare** l'avvio delle transizioni **da Context**, e quindi **gli switch** su esso

L'interfaccia usata dai ConcreteState **ritorna il riferimento** a un nuovo stato (è detta **fluent**)

## Con pattern...

```
// Biglietto e' il Context
public class Biglietto {
 private String codice = "XYZ";
 private int prezzo = 100;

 private StatoBiglietto sb = new Disponibile();

 public void mostra() {
 System.out.println("Prezzo: " + prezzo + " codice: " + codice);
 sb.mostra();
 }

 public void prenota(String s) {
 sb = sb.intesta(s);
 }

 public void cancella() {
 sb = sb.cancella();
 }

 public void compra() {
 sb = sb.paga();
 }
}

// StatoBiglietto e' uno State
public interface StatoBiglietto {
 public void mostra();
 public StatoBiglietto intesta(String s);
 public StatoBiglietto paga();
 public StatoBiglietto cancella();
}
```

## Con pattern...

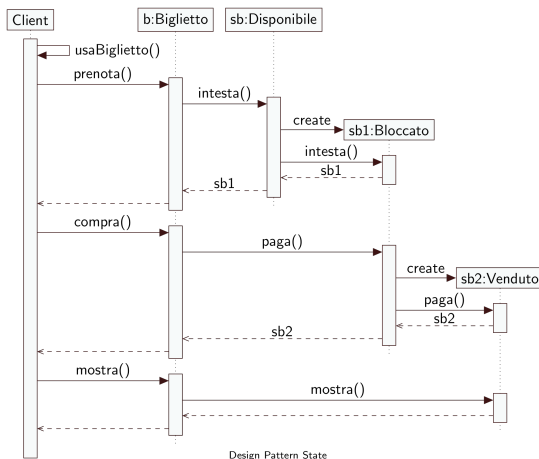
```
// Disponibile e' un ConcreteState
public class Disponibile implements StatoBiglietto {
 @Override public void mostra() { }

 @Override public StatoBiglietto intesta(String s) {
 System.out.println("DISP Cambia stato da Disponibile a Bloccato");
 return new Bloccato().intesta(s);
 }

 @Override public StatoBiglietto paga() {
 System.out.println("DISP Non si puo' pagare, bisogna prima intestarlo");
 return this;
 }

 @Override public StatoBiglietto cancella() {
 System.out.println("DISP Lo stato era gia' Disponibile");
 return this;
 }
}
```

## Vista a run-time



L'interfaccia del context può non coincidere con l'interfaccia State: questa può avere signature diverse, su cui però si devono poter mappare correttamente le funzionalità

in questo caso il context è *affine* ad un adapter *multiplo* adatta il client al riuso di multiple funzionalità, vista la similarità delle interfacce

Su GitHub: APPBiglietti

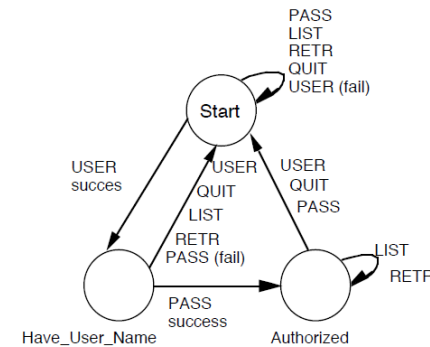
## Analisi del codice

- Le LOC sono 2 o 3 per metodo, ci sono 4 classi, e 1 interfaccia
  - Totale LOC 140 circa (compresi commenti e linee vuote), solo 53 linee con ";"
  - Sono state **eliminate le istruzioni condizionali**: i metodi non devono controllare in quale stato si trovano, poiché la classe si riferisce ad un singolo stato. **Non si ha codice duplicato** per i test condizionali su ciascun metodo
  - **I metodi sono più semplici** da comprendere e modificare
  - Il codice per ciascuno stato può implementare altre attività senza complicarsi
- 
- Sono **conseguenze** dell'applicazione del design pattern State

## Esempio più complesso

- Applicazione "Simple Post Office Protocol"
- Un semplice mailbox (solo ricezione) che supporta i seguenti comandi:
  - USER <username>
  - PASS <password>
  - LIST
  - RETR <message number>
  - QUIT
- Comandi USER e PASS :
  - Se l'username e la password sono validi l'utente può accedere agli altri comandi
- Comando LIST :
  - Parametro (opzionale) message-number
  - Se presente, ritorna la dimensione del messaggio indicato
  - Altrimenti, ritorna la dimensione di tutti i messaggi nel mailbox

## Logica di funzionamento



L'applicazione ha una serie di funzionalità non indipendenti: legate dallo stato

Esistono delle regole precise che stabiliscono le transizioni (esecuzioni di comandi) che sono ammesse

Posso formalizzarle con una macchina a stati finiti (automa)

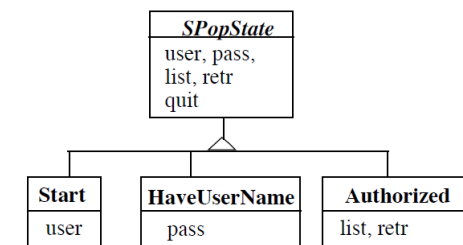
Codifico un protocollo d'uso corretto per l'oggetto

## Implementazione con switch

- Codice confuso e poco chiara la logica su GitHub....SPOPapp
- Ripetizioni e duplicazioni
- Difficile da modificare in seguito
- La presenza di switch è un sintomo che suggerisce di usare il pattern State (quantomeno, il polimorfismo)

## SPOP con Pattern

- Sfrutto il polimorfismo
- No duplicazione
- Comportamenti default



su GitHub....SPOPappState

## Chi definisce le transizioni? Lo stato

```
class SPop {
 private SPopState state = new Start();

 public void user(String userName){
 state = state.user(userName);
 }
 public void pass(String password){
 state = state.pass(password);
 }
 public void list(int messageNumber){
 state = state.list(messageNumber);
 }
}
```

È la scelta che dà più flessibilità a run-time

Nuovi stati (e nuove logiche) possono essere incluse a run-time

Il contesto (SPop) resta minimale e riusabile

## Chi definisce le transizioni? Il contesto

```
class SPop{
 private SPopState state = new Start();

 public void user(String userName){
 state.user(userName);
 state = new HaveUserName(userName);
 }
 public void pass(String password){
 if (state.pass(password))
 state = new Authorized();
 else
 state = new Start();
 }
}
```

- La **logica** che gestisce i cambiamenti di stato è **separata dalle azioni** corrispondenti e **raccolta** in una sola classe (Context)

- Più facile comprenderla o modificarla: anche se sparpagliata nei metodi del context, non è distribuita su più oggetti
- permette di **riusare gli stati** (ora tra loro indipendenti) in contesti (automi) diversi
- Va bene se le transizioni sono già note a compile-time
- Gli stati si semplificano ulteriormente

- Altra versione di SPOP con logica cablata nel contesto:

[SPOPAppStateBis su GitHub](#)

## Condivisione degli stati

- Nell'esempio visto, tutto il comportamento e di dati del contesto sono spostati negli oggetti stato: **è una situazione estrema**
  - In generale, il contesto può avere anche altri dati o metodi che non influenzano lo stato complessivo
- L'altro estremo solo gli **stati senza attributi di istanza**:
  - Sono proprio stateless o lavorano su dati che devono restare conservati (privati) nel contesto
  - Sono stati che posso condividere/riusare in molteplici contesti
    - Posso usare il pattern **singleton** per crearli una volta sola e non distruggerli mai
    - Oppure posso crearli quando servono ed eliminarli quando inutilizzati
    - Refactoring** di un concrete state affinché non abbia più variabili di istanza
      - le sposto nel contesto. Poi...
      - le passo allo stato come parametri (vedi prec. slide), oppure...
      - Passo il contesto e gli stati vi accedono direttamente...

## Condivisione degli stati

```
class SPop {
 private SPopState state = new Start();
 String userName;
 String password;
 public String userName() { return userName; }
 public String password() { return password; }
 public void user(String newName) {
 this.userName = newName ;
 state.user(this);
 }
 ...etc.
}
```

```
class HaveUserName implements SPopState {
 public SPopState pass(SPop mailServer) {
 validate(mailServer.password());
 ...etc.
 }
}
```

Passo il contesto e gli stati vi accedono direttamente...

Devo prevedere metodi getter-setter per consentire e disciplinare l'accesso

# Composite

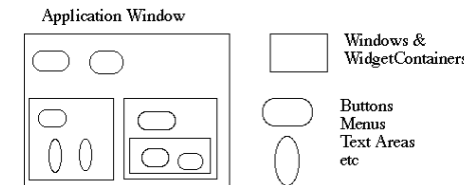
Object Structural



## COMPOSITE

•Intento:

- manipolazione uniforme di strutture ad oggetti legate da relazioni del tipo tutto-parte: composizione/aggregazione.
- I client devono essere in grado di ignorare la differenza tra composizioni di oggetti e oggetti singoli. I client tratteranno gli oggetti in modo uniforme.



•Avrò oggetti «semplici» (foglie) ed oggetti «compositi», contenitori di altri oggetti

•Voglio poter invocare l'operazione che ridisegna (update()) complessivamente tutta l'interfaccia utente come una singola operazione su un unico oggetto

```
class Window {
 Buttons[] myButtons;
 Menus[] myMenus;
 TextAreas[] myTextAreas;
 WidgetContainer[] myContainers;

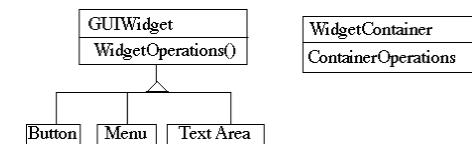
 public void update() {
 if (myButtons != null)
 for (int k = 0; k < myButtons.length(); k++) myButtons[k].refresh();
 if (myMenus != null)
 for (int k = 0; k < myMenus.length(); k++) myMenus[k].display();
 if (myTextAreas != null)
 for (int k = 0; k < myTextAreas.length(); k++) myTextAreas[k].refresh();
 if (myContainers != null)
 for (int k = 0; k < myContainers.length(); k++) myContainers[k].updateElements();
 // etc.
 }

 public void fooOperation() {
 if (myButtons != null);
 // etc.
 }
 //etc...
}
```

Ogni operazione avra' la stessa struttura duplicata: n cicli, uno per tipologia di oggetto.  
Uso direttamente le interfacce degli oggetti: la classe dipenderà da tutti i tipi esistenti

## Idea: uso il polimorfismo

- Con una interfaccia rappresento in astratto tutti i tipi semplici (foglie)
- un solo tipo li rappresenta tutti
- Distinguo ancora tra due categorie: oggetti contenitori e semplici
- Devo comunque iterare esplicitamente gli oggetti delle due tipologie
- Idea migliore: vedere il contenitore come una foglia "speciale"

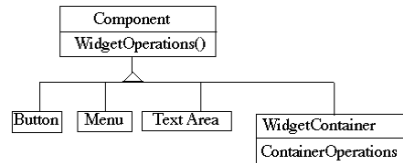


```
class Window {
 GUIWidgets[] myWidgets;
 WidgetContainer[] myContainers;

 public void update() {
 if (myWidgets != null)
 for (int k = 0; k < myWidgets.length(); k++)
 myWidgets[k].update();
 if (myContainers != null)
 for (int k = 0; k < myContainers.length(); k++)
 myContainers[k].updateElements();
 // etc.
 }
}
```

## Composite pattern

- **Una sola interfaccia (Component)** definisce in modo uniforme come interagire con tutti i tipi di oggetto: sia semplici che composti
- può anche implementare dei comportamenti di default
- Gli oggetti semplici (**Leaf**) fanno override delle operazioni di component per specializzarle
- Il **Composite** (widgetContainer) referencia i suoi molteplici oggetti come istanze tutte di tipo Component
- In Composite specializzo ogni operazione in Component con un semplice ciclo, che invoca l'operazione su tutti i suoi componenti



```

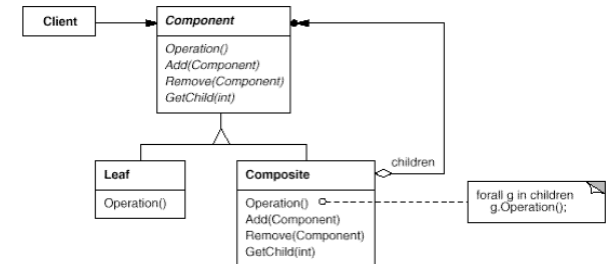
class WidgetContainer extends Component {
 Component[] myComponents;

 //ricorsione object oriented
 public void update() {
 if (myComponents != null)
 for (int k = 0; k < myComponents.length(); k++)
 myComponents[k].update();
 }
 // add container specific operations here
}

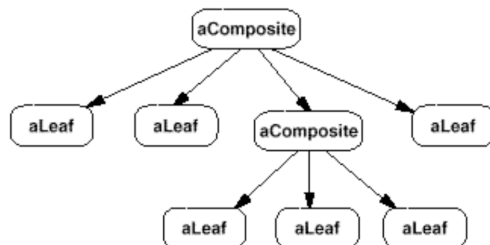
```

## Struttura del pattern Composite

- L'interfaccia **Component** rappresenta in modo uniforme le **operazioni applicabili all'intera struttura**, ovvero, comuni a tutti gli oggetti semplici
- Il codice client è indipendente dai tipi **Leaf**: Leaf1, Leaf2...LeafN usati nella struttura
- Solo il Composite implementa dei metodi in più per aggiungere o togliere oggetti al suo interno (e similari)
- Composite implementa le operazioni di applicandole **ricorsivamente** agli oggetti al suo interno;



## Esempio di struttura a run-time



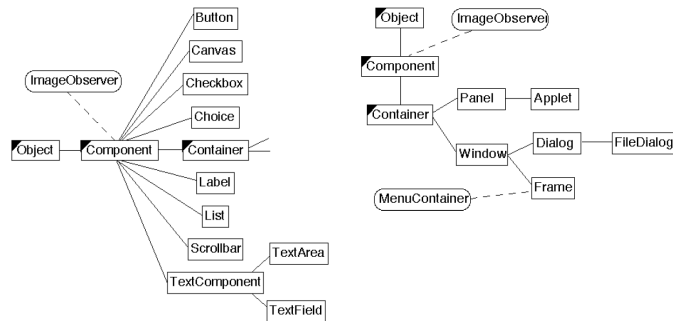
NB la struttura è eterogenea, posso aggregare oggetti di tipi (classi) diversi, sia come Leaf che come Composite

## Composite

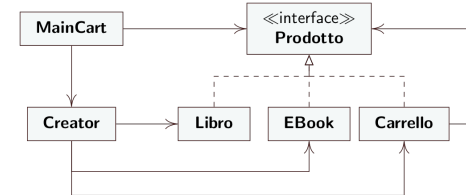
- **Motivazione**
  - Necessità di **raggruppare** elementi semplici tra loro per formare elementi composti
  - Non **distinguere** tra classi per elementi semplici e classi per contenitori nei client
- **Collaborazioni**
  - I client usano **l'interfaccia Component** per interagire con elementi della struttura composta.
  - Se il ricevente del messaggio è Leaf, la richiesta è gestita direttamente.
  - Se il ricevente è un Composite, questo invia la richiesta ai suoi child e possibilmente **avvia operazioni aggiuntive** prima e dopo

## Usato in Java

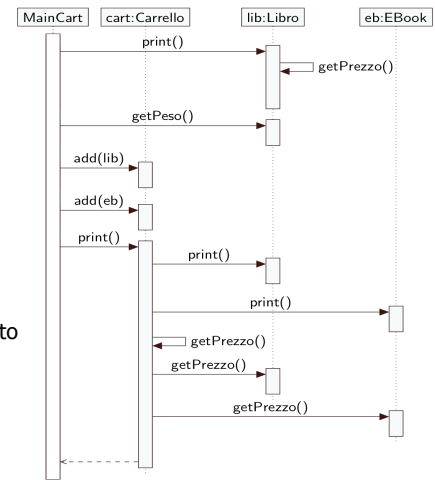
- Lungamente impiegato per le librerie AWT in Java fin dalla versione 1.0



## Esempio: AppCart



- Si vuol gestire un carrello di prodotti vari con la stessa semplicità con cui gestirei un solo prodotto
- Implemento il carrello come una struttura composita
- Mi avvalgo di un Creator (factory method) per riempirlo



## Conseguenze

- Un client che si aspetta un elemento semplice può riceverne uno composto
- I client sono semplici, trattando strutture composte e semplici uniformemente, non sapendo se usano un Leaf o un Composite (polimorfici)
- Nuovi tipi di elementi (Leaf o Composite) possono essere aggiunti e potranno funzionare con la struttura ed i client esistenti (riusabili)
- Supporto qualsiasi tipo di Leaf che implementi l'interfaccia usata dal client (Component), non serve che siano parenti
- Non è possibile a design time vincolare il Composite all'uso solo alcuni tipi di Leaf: posso solo bloccarne l'uso facendo dei controlli a runtime
- Ultimo punto: proprio perché è una struttura di istanze, i tipi di foglie non devono per forza essere parenti tra loro. Questo può essere non voluto: a volte potrei volere garantire che i componenti siano di una sola e ben precisa gerarchia. Con questo pattern è volutamente inevitabile il contrario: si applica al dominio strutturale degli OGGETTI non delle classi.

## Implementazione

- **Il Composite può essere anche un ulteriore tipo di Leaf** (ad es. la classe Window ha un titolo, posizione, bordo, dimensioni, etc...) con suoi dati e operazioni
  - Di base è solo un raccoglitore che implementa la gestione dei suoi children (**add-remove**)
- **In fase di costruzione** della struttura, so quali oggetti sono Composite e possono ricevere messaggi add/remove
- Se dichiarassi add/remove nel Component?
  - Vantaggi: **trasparenza**. Tutti gli oggetti hanno la stessa interfaccia e sono quindi indistinguibili per il client: lo scopo del pattern
  - Svantaggi: **esposizione ad errori a run-time**
    - Consentire di invocare add/remove sulle leaf potrebbe generare un errore a run-time
    - posso gestirlo (**eccezione**), ma avrei potuto segnalarlo in compilazione
    - posso ispezionare il tipo (reflection) dell'oggetto prima di invocarle, ma non ho trasparenza

## Implementazione

- L'attraversamento di una struttura composita non è trasparente: devo poter distinguere i Leaf dai Composite
  - Si può inserire in **Component** una operazione `getComposite()` **che ritorna null**, ridefinita in **Composite** per **ritornare this**.
  - In Java si può usare la reflection
- La **lista che contiene i child** deve essere definita in **Composite**, non in Component. Ie Leaf non ne fanno mai uso
- **L'ordinamento dei child** per un Composite potrebbe essere importante in certe implementazioni
- Il Composite potrebbe **implementare una cache**, per ottimizzare le prestazioni
  - Ad es. nella ricerca di un child tra tutti i child locali
  - I child devono poter accedere ad un'operazione che **invalida la cache**

## Riferimenti espliciti al padre

- In component posso avere il riferimento all'oggetto padre
- Leaf e Composite implementeranno le operazioni per gestirlo
- Facilita la navigazione a ritroso tra gli oggetti della struttura

```
class Composite extends Component {
 Component[] myComponents;

 public void update() {
 if (myComponents != null)
 for (int k = 0; k < myComponents.length(); k++)
 myComponents[k].update();
 }

 public void add(Component aComponent) {
 myComponents.append(aComponent);
 aComponent.setParent(this);
 }
}

abstract class Component {
 protected Component parent;
 public void setParent(Component myParent) {
 parent = myParent;
 }
 // etc.
}
```

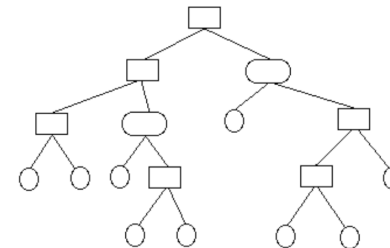
# Visitor

## Object Behavioural



## Albero: esempio di struttura composita

Sappiamo navigarlo, ma in che ordine : Pre-ordine, post-ordine, in-ordine?



E se ci potesse servire anche...

- Navigarlo negli altri modi?
- Poterlo stampare in 2D?
- Una stampa in HTML?
- Saperlo bilanciare?

... come supportare varie necessità future  
Senza doverle prevedere tutte prima?  
Senza affollare l'ADT di cose "opzionali"?

## Pattern Visitor

### Soluzione 1: sottoclassare l'ADT

- Si dà luogo a numerose sottoclassi
- Non posso avere combinazioni di più di due funzionalità in una sola classe

### Soluzione 2 (Visitor): incapsulo le funzionalità in oggetti

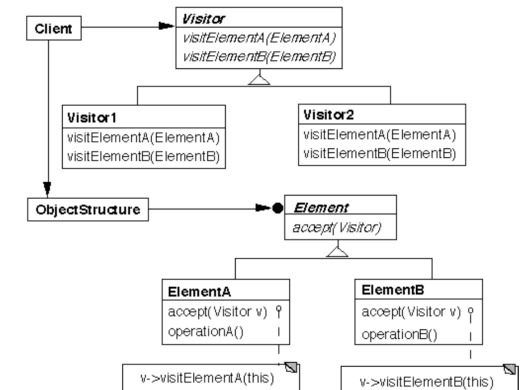
- Metto le operazioni in un oggetto separato (il "visitor")
- Passo il visitor a ciascun elemento della struttura
- L'elemento invoca la funzionalità del visitor, e quindi...
- Il visitor esegue la sua operazione sull'elemento → **Double Dispatch**

## Struttura del pattern

La struttura, un iterator, o il visitor possono saper navigare gli elementi

- Tipicamente è la struttura
- Avere la navigazione nel visitor può essere una duplicazione di codice

Il visitor conosce il tipo degli elementi concreti su cui si applica, per cui usare un visitor sbagliato è un compile-time error



## Esempio

```
class BinaryTreeNode extends Node {
 public void accept(Visitor aVisitor) {
 aVisitor.visitBinaryTreeNode(this);
 }
 etc.
}

class BinaryTreeLeaf extends Node {
 public void accept(Visitor aVisitor) {
 aVisitor.visitBinaryTreeLeaf(this);
 }
 etc.
}

abstract class Visitor {
 abstract void visitBinaryTreeNode(BinaryTreeNode);
 abstract void visitBinaryTreeLeaf(BinaryTreeLeaf);
}

class HTMLPrintVisitor extends Visitor {
 public void visitBinaryTreeNode(BinaryTreeNode){
 HTML print code here
 }
 etc.
}
```

## Double Dispatch

La visita di un elemento richiede due chiamate di metodo

- 1) `Node example = new BinaryTreeNode();`
- 2) `Visitor traveler = new HTMLPrintVisitor();`
- 3) `example.accept( traveler );`

`example.accept(traveler)` invoca al suo interno `aVisitor.visitBinaryTreeNode(this);`

Nella prima invocazione (`accept`) selezioni la classe Visitor (la funzionalità da applicare)

Nella seconda selezioni il metodo corretto, per l'elemento corrente (`this`), da invocare nel Visitor (come applicarla)

È un legame definito staticamente (noto a compile time)

## Java (8+): Consumer functional interface

```
@FunctionalInterface
public interface Consumer<T> {
 void accept(T t);
}
```

This is a functional interface whose functional method is `accept(Object)`.

functional interface = Interfaccia con esattamente un solo metodo astratto:

Le interfacce funzionali permettono di definire singoli metodi, anche anonimi a run-time (lambda expressions), come oggetti "di prima classe":

Metodi che puoi assegnare a variabili o passare come valori attuali ai parametri di altri metodi...

`Consumer` represents an operation that accepts a single input argument and returns no result.

**i consumer corrispondono a dei visitor**

## default void `forEach (Consumer<? super T> action)`

L'applicazione del pattern di Java è meno macchinosa e più flessibile (ma meno generale) del pattern originale:

```
Collection <String> sc = new ArrayList<String>();

for(String s: sc) {System.out.println(i)} // println used as a Consumer (con Iterator)
sc.forEach(println); // println is a method reference of a Consumer
sc.forEach((String i) -> System.out.println(i)); // lambda expression
sc.forEach(HTMLPrintVisitor::visit); // method reference
```

• **ForEach(Consumer action)** è il metodo **accept(Visitor visitor)** predefinito di tutti gli aggregati (interfaccia `Iterable`): non serve implementare altro, solo i visitor

• invoca il visitor (consumer) su tutto l'aggregato: **for(T t : this) action.accept(t);**

• Con le lambda expression il metodo visitor è definibile a run-time, come oggetto anonimo

• l'**accept(T t)** del consumer corrisponde al **visit(Element e)** del pattern

## Applicabilità

- Quando una struttura di oggetti contiene molte classi di oggetti con interfacce diverse e si desidera eseguire su questi oggetti operazioni che dipendono dalle loro classi concrete
- Quando è necessario eseguire molte operazioni distinte e non correlate sugli oggetti in una struttura di oggetti e si desidera evitare di sovraccaricare le classi con queste operazioni
- Quando le classi che definiscono la struttura cambiano raramente, ma si desidera spesso definire nuove operazioni sulla struttura

## Conseguenze

- I visitori semplificano l'aggiunta di nuove operazioni
- I visitori raccolgono le operazioni correlate e separano quelle non correlate
- Se aggiungo un nuovo `ConcreteElement` è necessario modificare tutti i visitori esistenti
- Gli iteratori non possono iterare attraverso una struttura contenente classi non correlate
  - Tuttavia, il visitatore presuppone che ogni elemento nella struttura contenga un metodo di visita, il che implica almeno un'interfaccia di visita comune per tutti gli elementi
- Accumulazione di stato
  - Il visitatore può accumulare informazioni
- Il visitatore potrebbe dover accedere a parti privilegiate degli elementi
  - Anche qui sono utili le inner class per non doverle rendere pubbliche

## Esercizio per casa

- Usare il pattern Visitor per fornire a un client molti modi di attraversare il BST (pre, post, etc...)

# Decorator

Object Structural



## Motivazione

Un oggetto deve implementare una operazione composta da un numero **molteplice e variabile** di singole responsabilità.

- Ad es. disegnare un oggetto grafico **TextView** che può avere anche, opzionalmente:
  - Barra scorrimento verticale
  - Barra scorrimento orizzontale
  - Bordo 3D
  - Bordo piatto

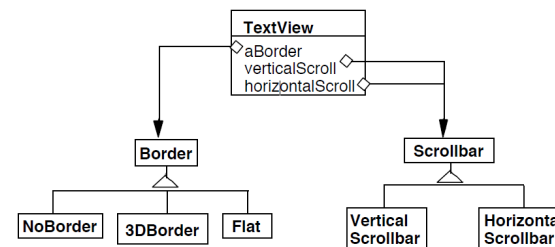
Prevedere nella sua classe l'implementazione di tutte le responsabilità che servono significherebbe avere per essa troppe responsabilità

Soprattutto se è una classe posizionata in alto nella gerarchia

## Estendere la base

- Dichiaro la classe base
  - TextView
- Derivo una sottoclasse per ogni possibile combinazione di opzioni
  - TextViewWithNoBorder&SideScrollbar
  - TextViewWithNoBorder&BottomScrollbar
  - TextViewWithNoBorder&Bottom&SideScrollbar
  - TextViewWith3DBorder
  - TextViewWith3DBorder&SideScrollbar
  - ...
- Prolifera le classi
- codice duplicato
- Alternative?

## Soluzione 1, non ottimale



```
class TextView {
 Border myBorder;
 ScrollBar verticalBar;
 ScrollBar horizontalBar;

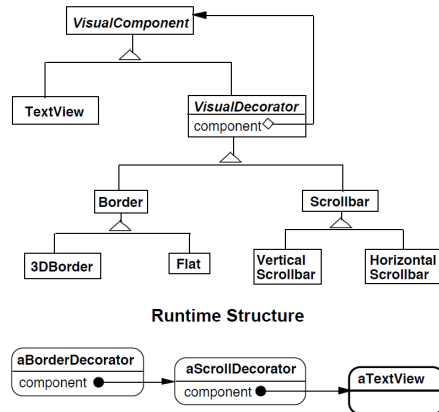
 public void draw() {
 myBorder.draw();
 verticalBar.draw();
 horizontalBar.draw();
 //code to draw self
 }
 etc.
}
```

- Uso la composizione invece dell'ereditarietà
- Ho una sola classe per ogni opzione aggiuntiva

- Ma la classe textview dipende da tutte le tipologie di classi di opzioni
- Aggiungere una opzione richiede la modifica di TextView
- E di ogni altra classe cui l'opzione era applicabile...

## Soluzione 2: Decorator pattern

- Definisco l'interfaccia **Component**, implementata dal **componente base** e da un **Decorator**
- Nel Decorator ho un riferimento ad un componente successivo: un altro decorator o il componente base.
- Tutte le tipologie di opzioni sono sottoclassi di Decorator (sono dei decoratori)
- TextView ha al suo interno solo il suo codice, non ha né bordo né barre
- Creo a run-time una catena di funzionalità successive, conclusa dalla classe base



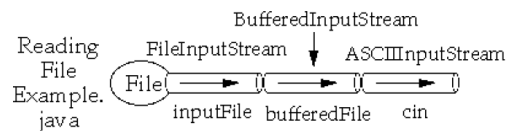
## Decorator

- Con il pattern Decorator
  - Se si vuol aggiungere la proprietà Bordo al componente Testo:
    - Inserisco il componente Testo dentro un altro oggetto, che a sua volta aggiunge il Bordo
  - Il client ottiene un testo incorniciato, **interagendo solo con l'oggetto più esterno**
  - DecoratorBordo usa i servizi della componente Testo e aggiunge la sua attività **prima o dopo** l'invio della richiesta ad esso (obj. oriented recursion)



## Java I/O stream library

Usato nelle librerie Java per l'i/o da file, per la gestione dei fonts, per le interfacce utente AWT/Swing



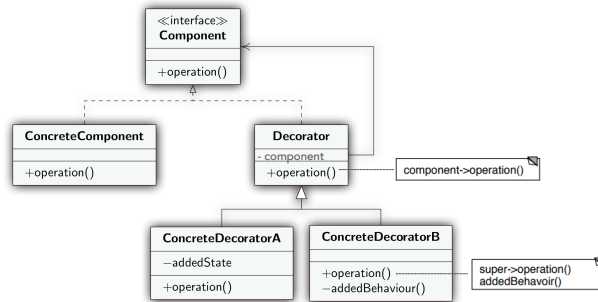
- Da non confondere con i nuovi streams funzionali in java 8
- Esempio: **ReadingFileExample**

## Decorator

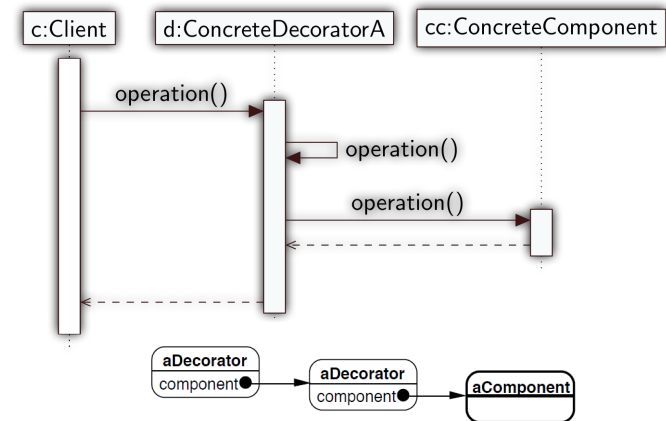
- Intento:** Modificare il comportamento di un oggetto dinamicamente.
  - L'oggetto cambia tipo e comportamento
  - Alternativa all'uso di sottoclassi per estendere implementazioni
  - Flessibile**, perchè definisco la funzionalità complessiva a run-time
  - Avvolgo (wrap) il componente base in un altro oggetto che aggiunge implementazione alla sua.
  - L'oggetto wrapper **è un decorator** e la sua presenza **è trasparente ai client**, che lo usano attraverso l'interfaccia Component

## Decorator: struttura

- **Component**: interfaccia comune per comporre dinamicamente l'implementazione di operation()
- **ConcreteComponent**: l'oggetto base cui aggiungere responsabilità
- Fornisce l'implementazione fissa di operation()
- Termina la catena di oggetti composti
- **Decorator**: un Component con un riferimento a un ulteriore Component.
- Implementa operation() inoltrando la richiesta al suo Component successore
- **ConcreteDecorator**: implementa una responsabilità aggiuntiva per il Component e svolge la sua operazione prima e/o dopo l'inoltro della richiesta



## Interazioni



## Implementazione

```
interface Component {
 public void operation();
}
```

```
class ConcreteComponent implements Component {
 public void operation() {
 // impl. base dell'operazione
 }
}
```

```
class Decorator implements Component {
 private final Component nextComp;

 public Decorator(Component c) {
 nextComp = c;
 }

 @Override public void operation() {
 nextComp.operation();
 }
}
```

```
class ConcreteDecoratorA extends Decorator {
 public ConcreteDecoratorA(Component c) {
 super(c);
 }

 @Override public void operation() {
 super.operation();
 // ...funzionalità aggiuntiva
 }
}
```

Set-up di un oggetto (catena) con funzionalità estesa:

```
Component c = new ConcreteDecoratorA(new ConcreteComponent());
```

Esempio su vscode: Applicazione Messaggi

## Conseguenze

- Scelta forzata se la creazione di sottoclassi non è praticabile, per il numero elevato di varianti, o se si vuole poter estendere anche in seguito
- Si avranno **tante piccole classi**, indipendenti e focalizzate su un solo compito, molto specifico
- A run-time, **interconnetto tanti piccoli oggetti in sequenza**, come i passi di un algoritmo, ottenendo un oggetto complesso che offre una funzionalità **nuova**, più elaborata delle esistenti
- Posso **aggiungere, sottrarre, spostare, ripetere** a piacimento le istanze di micro funzionalità, a tempo d'esecuzione, componendo o **modificando l'algoritmo eseguito**
- Molto più flessibile di un algoritmo da scrivere a priori in una classe
- L'oggetto, la catena di oggetti, che implementa l'algoritmo non ha un tipo (ADT) nominale, ma solo la facciata d'uso (Component): ho creato un **prototipo**

## Decorator vs State

- **Change Skin vs change Guts** : modifico a run-time il comportamento di un oggetto, (l'effetto di un messaggio ad un oggetto) come nel pattern State, ma:
  - L'oggetto con cui il client interagisce cambia tipo (cambio pelle)
  - Con State, il cambio di oggetto è solo interno (cambio viscere)
- Qui aggiungo/sottraggo funzionalità ed ho un comportamento di base
- Con State, *sostituisco* (interamente) la funzionalità tra varie alternative
- Qui, istanzio di volta in volta solo le funzionalità aggiuntive che uso
- Con State, tutti gli stati sono necessari per avere un funzionamento completo

# Observer

Object behavioural



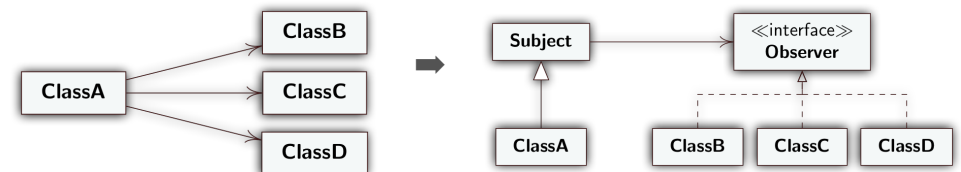
## Observer

- Intento:
  - Definire una **dipendenza uno a molti** fra oggetti
  - quando un oggetto cambia stato tutti i suoi oggetti dipendenti sono notificati e aggiornati automaticamente
- Motivazione:
  - **mantenere la consistenza** fra oggetti che hanno relazioni
- Conosciuto anche come Publish-Subscribe

## Observer

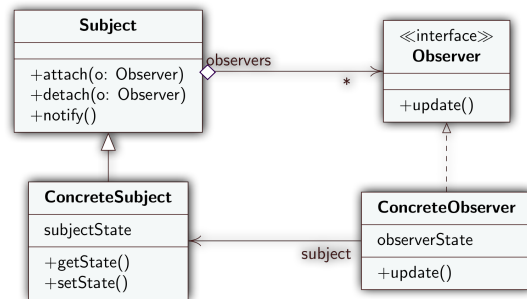
- Applicabilità
  - Un'astrazione ha **due aspetti interdipendenti** (es. dati e presentazione). Incapsulare questi aspetti in oggetti separati permette di **riusarli indipendentemente**
  - Un cambiamento su un oggetto richiede il cambiamento di altri e **non si conosce quanti** oggetti sarà necessario cambiare
  - Un oggetto deve notificare altri oggetti **senza sapere chi sono** tali oggetti, quindi gli oggetti **non devono essere strettamente accoppiati**

## Prima e dopo l'uso di Observer

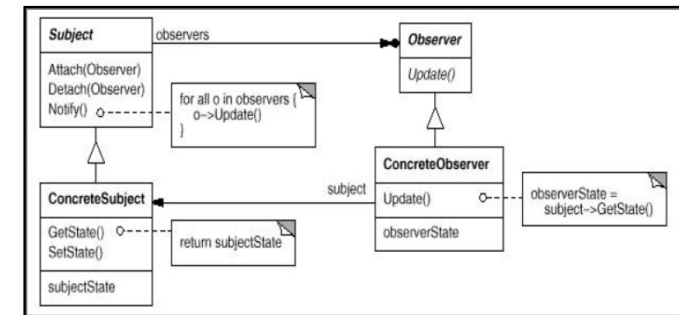


## Observer

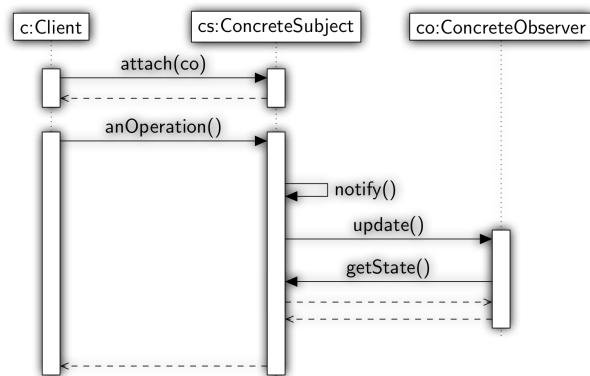
### • Soluzione



## Soluzione (dal libro)



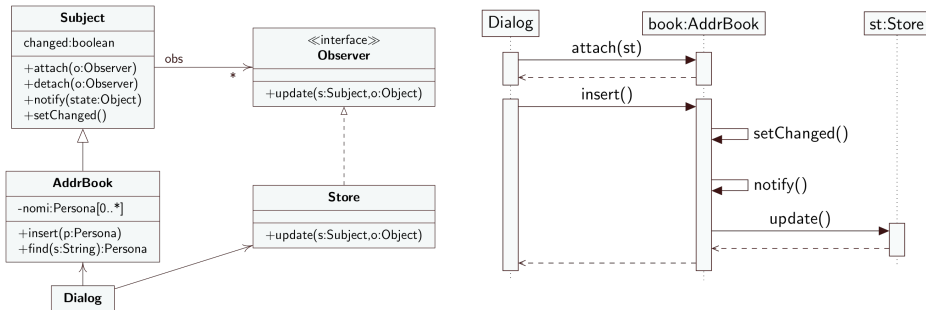
## Observer



## Observer in Java (<=8)

- Incluso nella libreria `java.util`, con la classe **Observable** e l'interfaccia **Observer**
- **Observable** svolge il ruolo di **Subject** e notifica il cambiamento di stato quando il metodo `notifyObservers()` viene chiamato
- ha un attributo (boolean) di stato, impostato dal metodo `setChanged()`.
- **Observer** è una interfaccia che ha solo il metodo `update()` e può avere un argomento che indica quale oggetto ha causato l'aggiornamento
- Alla base dell'implementazione della gestione eventi (`ActionListener`) nelle librerie per le interfacce grafiche (vedi esempio `Buzzer` su GitHub)

## Esempio: sincronizzazione dello storage



```
public class Subject {
 private List<Observer> obs = new ArrayList<>();
 private boolean changed = false;
 public void notify(Object state) {
 if (!changed) return;
 for (Observer o : obs) o.update(this, state);
 changed = false;
 }
 public void setChanged() {
 changed = true;
 }
 public void attach(Observer o) {
 obs.add(o);
 }
 public void detach(Observer o) {
 obs.remove(o);
 }
}

public class AddrBook extends Subject {
 private List<Persona> nomi = new ArrayList<>();
 public void insert(Persona p) {
 if (nomi.contains(p)) return;
 nomi.add(p);
 setChanged(); // la prossima notifica avverrà
 notify(nomi); // notifica i ConcreteObserver
 }
 public Persona find(String cognome) {
 for (Persona p : nomi)
 if (p.getCognome().equals(cognome)) return p;
 System.out.println("AddrBook: find: NOT found");
 return null;
 }
}
```

```
public interface Observer {
 public void update(Subject s, Object o);
}

public class Store implements Observer {
 @Override
 public void update(Subject s, Object o) {
 List<Persona> l = (List<Persona>) o;
 String nom;
 try (FileWriter f = new FileWriter("nomi.txt")) {
 for (Persona p : l) {
 nom = p.getNome() + "\t" + p.getCognome() +
 "\t" + p.getTelefono();
 f.write(nom + "\n");
 }
 } catch (IOException e) {}
 }
}

public class Dialog {
 private static final AddrBook book =
 new AddrBook();
 private static final Store st = new Store();
 private static final Persona p1 =
 new Persona("Oliver", "Stone", "012345", "NY");

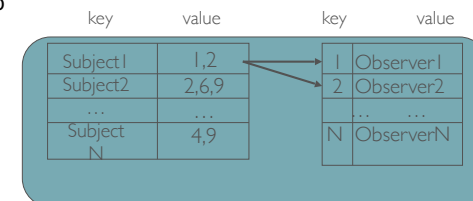
 public static void main(String[] args) {
 book.attach(st);
 book.insert(p1);
 }
}
```

## Conseguenze

- Il Subject conosce solo la classe Observer e non ha bisogno di conoscere le classi ConcreteObserver. **ConcreteSubject e ConcreteObserver non sono accoppiati** quindi più facili da riusare e modificare
- La notifica inviata da un ConcreteSubject è mandata a tutti gli oggetti che si sono iscritti, il ConcreteSubject non sa quanti sono. **Gli osservatori possono essere rimossi in qualunque momento.**
- Quando si vuol **eliminare un Subject**, gli Observer dovrebbero essere avvisati per cancellare eventuali loro riferimenti ad esso!
- La notifica può **non dire agli Observer cosa è cambiato** nel ConcreteSubject, è un evento che **indica solo il completamento di un'operazione** del ConcreteSubject
- In alternativa, il Subject può passare informazioni sullo stato come parametro della `notify()` (modalità push) o è l'Observer che accede allo stato del Subject (modalità pull) ed in questo caso dipende dal Subject.

## Shared look-up table

- Uno stesso Observer potrebbe osservare più subject (**relazione N a N**): per sapere chi ha mandato la notifica, come parametro di `update()` si manda il riferimento al Subject
- Se ho **tanti Subject e pochi Observer**, anziché tenere riferimenti a Observer nei Subject, si può usare una **tabella associativa in comune** fra i Subject per ridurre lo spazio impegnato



- No duplicate references to observers
- No storage for subjects with no observers

## Considerazioni

- Quando il ConcreteSubject chiama `notify()`, questo chiama `update()` che **esegue sul thread del chiamante**, costringendo il subject ad aspettare l'esecuzione di `update()` di ciascun ConcreteObserver
- Le notifiche **sono in serie** e possono essere **costose**, ovvero tante e lunghe, in modo imprevedibile per il subject!
- Subject chiama `notify()` dopo un cambiamento, oppure si aspetta un certo numero di cambiamenti, in modo da **evitare continue notifiche** agli Observer (**`setChanged()`** per almeno uno...)
- Gli Observer che si registrano possono **specificare gli eventi di interesse**, in modo che **la `notify()` non sia un broadcast** e si riducano le update (*Java Event Model*)

## Reactive programming

Adozione del pattern publish/subscribe (interazione reattiva) come paradigma generale di programmazione.

Reactive Streams è un progetto con l'obiettivo di creare uno standard per l'elaborazione asincrona di flussi di oggetti/eventi

Consente il controllo della **back pressure** (quantità di notifiche) sugli observer e **nessun blocco** del chiamante (ovvero del subject).

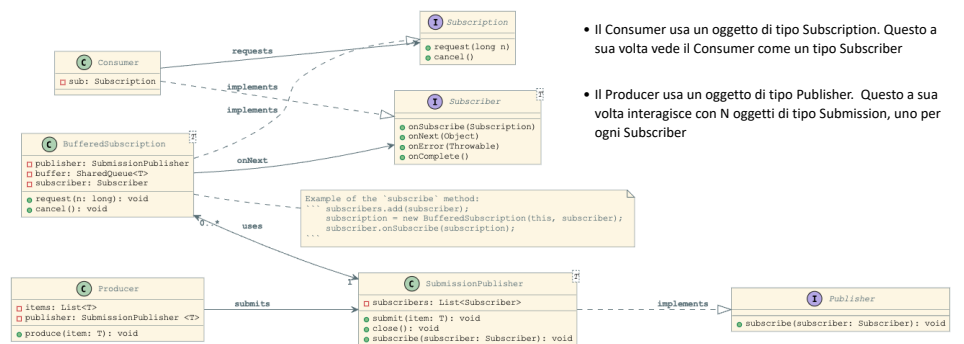
Il progetto ha prodotto una specifica, implementata in varie librerie, mirate sia ai run-time environments (JVM e JavaScript) che ai protocolli di rete, ed è idoneo per scrivere algoritmi che possono sfruttare sia la concorrenza locale che la distribuzione in rete.

Le interfacce disponibili nella classe `java.util.concurrent.Flow` del JDK 9 sono semanticamente equivalenti 1:1 alle loro rispettive controparti del progetto Reactive Streams.

## Observer in Java 9: Publisher/Subscriber

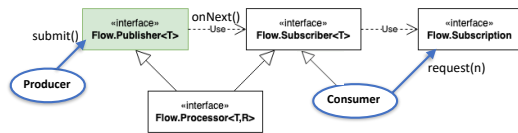
- Astrae il passaggio di un flusso (Flow) di oggetti da un **publisher** a un **subscriber senza bloccare il publisher e senza inondare il subscriber**
- Java 9 fornisce nella libreria `java.util.concurrent` le interfacce `Flow.Publisher<T>`, `Flow.Subscriber<T>`, `Flow.Subscription` ed inoltre `Flow.Processor<R,T>`
- La classe `SubmissionPublisher<T>` è solo una implementazione già disponibile di un `Publisher<T>`, che fa uso di un pool di thread per l'esecuzione asincrona delle notifiche (**`submit(T)`**) ai suoi subscribers.
- Attraverso un oggetto con interfaccia **Subscription**, ciascun Subscriber può controllare in modo autonomo la quantità massima di notifiche (**`onNext(T)`**) ricevibili in serie.
- Una coda (interna al Subscriber, condivisa dalle Subscription) conserva le notifiche prodotte da un Publisher e non ancora ricevute da tutti i Subscriber.

## Publish/Subscribe: soluzione

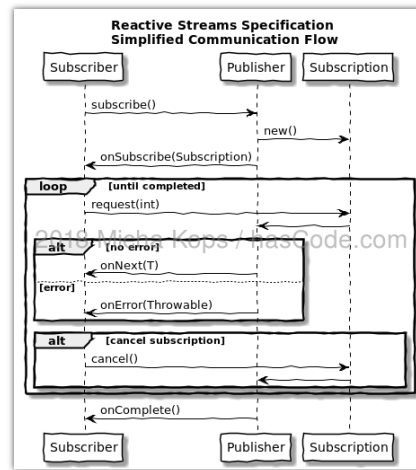


- Producer e Consumer non hanno nessuna dipendenza diretta. Solo l'interfaccia Publisher dipende dalla Subscriber.
- Il Consumer usa un oggetto di tipo Subscription. Questo a sua volta vede il Consumer come un tipo Subscriber
- Il Producer usa un oggetto di tipo Publisher. Questo a sua volta interagisce con N oggetti di tipo Submission, uno per ogni Subscriber

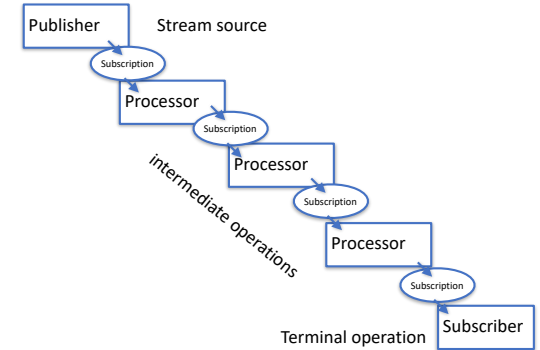
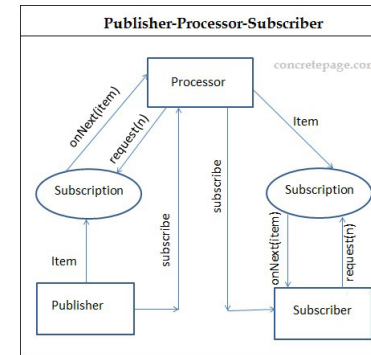
## Publisher-Subscriber



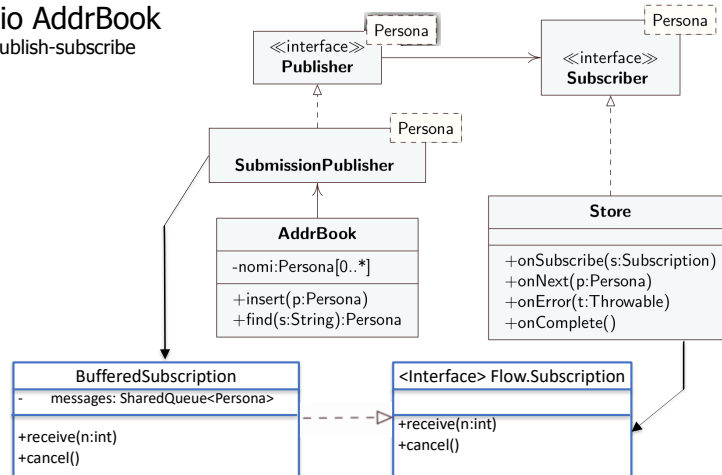
- Non è il publisher a generare i contenuti/notifiche ma un suo client (producer)
- Il publisher ha la lista dei subscriber (i consumatori finali)
- La subscription (concreta, interna al publisher) ha la coda di notifiche push
- La coda viene "sbloccata" direttamente dal consumer
- le notifiche



## Pipeline di processor (stream di elaborazione asincrona)



## Esempio AddrBook versione publish-subscribe



## JDK9 java.util.concurrent.Flow

public final class Flow extends Object

### Nested Classes

| Modifier and Type                                                           | Class               |
|-----------------------------------------------------------------------------|---------------------|
| static interface                                                            | Flow.Processor<T,R> |
| A component that acts as both a Subscriber and Publisher.                   |                     |
| static interface                                                            | Flow.Publisher<T>   |
| A producer of items (and related control messages) received by Subscribers. |                     |
| static interface                                                            | Flow.Subscriber<T>  |
| A receiver of messages.                                                     |                     |
| static interface                                                            | Flow.Subscription   |
| Message control linking a Flow.Publisher and Flow.Subscriber.               |                     |

## Publisher Subscriber Java 9

- **Publisher** è l'astrazione per fornire item, definisce il metodo ***subscribe()*** che prende in input `Subscriber<T>` [analogo di `Observable::addObserver()`]
- **SubmissionPublisher** implementa Publisher e invia item ai Subscriber in maniera asincrona. Quando sul SubmissionPublisher è chiamato il metodo ***submit()***, esso esegue in un thread dedicato (asincrono) la chiamata al metodo `onNext()` dei Subscriber, e si blocca se il subscriber non può ricevere l'item [analogo di `Observable::notify()` ma concorrente]
- **Subscriber** è usato per ricevere item, definisce i metodi ***onComplete()***, ***onError()***, ***onNext()***, ***onSubscribe()*** [analogo di `Observer::update()` ma più completo]
- **Subscription** definisce il collegamento fra Publisher e Subscriber; i metodi `cancel()` e `request(n)` permettono al Subscriber di fermare l'invio di messaggi e di richiedere l'invio dei prossimi n messaggi (*handshake* per adeguare la rapidità del flusso di notifiche). E' una interfaccia, noi non vediamo il tipo concreto (interno a Java)

## Pattern architetturale: Model View Controller (MVC)

- Il pattern Observer (Reactive) applicato all'architettura dell'intero sistema prende il nome di MVC
- Modularità: ogni sottosistema è nettamente focalizzato su un solo scopo funzionale
- Disaccoppiamento: l'interazione avviene per **reazione** a notifiche di "eventi", non tramite messaggi diretti alle rispettive interfacce, che restano sconosciute tra loro.
- Centralità della logica di Controllo: non consente l'interazione diretta tra View e Model.
- È un concetto ripreso anche nell'architettura dei sistemi distribuiti *three-tier*

